Ren'IDE User Guide

The Visual IDE for Ren'Py Development

See Your Story

Blue Moon Foundry

April 2026

Abstract

Welcome to Ren'IDE, the visual development environment designed specifically for Ren'Py visual novel creators. This comprehensive guide covers everything from installation to advanced features, with dedicated sections for writers, artists, and developers.

Whether you're creating your first visual novel or managing a complex branching narrative with hundreds of scenes, Ren'IDE helps you visualize structure, catch errors early, and streamline your workflow.

Contents

1 About This Guide **12**
- 1.1 Who Should Read This Guide . . . 12
- 1.2 What's Covered . . . 12
- 1.3 Document Conventions . . . 13
- 1.4 Getting Help . . . 13
- 1.5 Feedback . . . 13
- 1.6 Acknowledgments . . . 13

2 Quick Start Guide **14**
- 2.1 1. Install Ren'IDE . . . 14
- 2.2 2. Launch and Open a Project . . . 14
- 2.3 3. Explore the Interface . . . 14
- 2.4 4. Navigate Your Story . . . 14
- 2.5 5. Start Creating . . . 15

3 Ren'IDE User Guide **16**
- 3.1 Table of Contents . . . 16

4 Section 1: Introduction **17**
- 4.1 1.1 What is Ren'IDE? . . . 17
  - 4.1.1 The Problem Ren'IDE Solves . . . 17
  - 4.1.2 The Ren'IDE Solution . . . 17
  - 4.1.3 What Ren'IDE Is NOT . . . 18
  - 4.1.4 Core Philosophy . . . 18
- 4.2 1.2 Who Should Use Ren'IDE? . . . 19
  - 4.2.1 For Writers . . . 19
  - 4.2.2 For Artists . . . 20
  - 4.2.3 For Developers (Programmers) . . . 20
  - 4.2.4 For Solo Developers . . . 21
  - 4.2.5 For Teams . . . 22
- 4.3 1.3 Key Benefits . . . 22
  - 4.3.1 See Your Story's Structure . . . 22
  - 4.3.2 Navigate Instantly . . . 23
  - 4.3.3 Catch Problems Early . . . 23
  - 4.3.4 Compose Visually . . . 24
  - 4.3.5 Unified Environment . . . 24

4.4 1.4 System Requirements . . . 24
4.4.1 Minimum Requirements . . . 25
4.4.2 Recommended Requirements . . . 25
4.4.3 Platform-Specific Notes . . . 25
4.4.4 Internet Connection . . . 26
4.4.5 Storage Considerations . . . 26
4.4.6 Performance Considerations . . . 26

5 Section 2: Getting Started 27
5.1 2.1 Installation . . . 27
5.1.1 2.1.1 Windows Installation . . . 27
5.1.2 2.1.2 macOS Installation . . . 29
5.1.3 2.1.3 Linux Installation . . . 30
5.1.4 2.1.4 Building from Source (Optional) . . . 33
5.2 2.2 First Launch . . . 34
5.2.1 The Welcome Screen . . . 34
5.2.2 First Launch Permissions . . . 35
5.3 2.3 Opening an Existing Project . . . 35
5.3.1 What Ren'IDE Looks For . . . 35
5.3.2 Opening a Project . . . 36
5.3.3 Initial Project Analysis . . . 36
5.3.4 Understanding the Canvas View . . . 36
5.3.5 Persistence . . . 37
5.4 2.4 Creating a New Project . . . 37
5.4.1 Starting the Wizard . . . 37
5.4.2 Step 1: Project Name and Location . . . 38
5.4.3 Step 2: Resolution Presets . . . 38
5.4.4 Step 3: Theme and Accent Color . . . 38
5.4.5 Project Generation . . . 39
5.4.6 Next Steps After Creating a Project . . . 40
5.5 2.5 Configuring Ren'Py SDK Path . . . 40
5.5.1 Why You Need the Ren'Py SDK . . . 40
5.5.2 Downloading and Installing the Ren'Py SDK . . . 40
5.5.3 Configuring the Path in Ren'IDE . . . 41
5.5.4 Running Your Game from Ren'IDE . . . 41
5.5.5 Troubleshooting SDK Configuration . . . 42

6 Section 3: Interface Overview 44
6.1 3.1 Application Layout . . . 44
6.1.1 Main Layout Areas . . . 44
6.1.2 Collapsible Panels . . . 45
6.1.3 Responsive Design . . . 45
6.2 3.2 Toolbar Reference . . . 46
6.2.1 Undo/Redo Group . . . 46
6.2.2 Canvas Actions Group . . . 46
6.2.3 Panels Group . . . 46
6.2.4 Canvas Tabs . . . 47
6.2.5 Run Group . . . 47

6.2.6 File Actions Group . . . 47
6.2.7 Additional Toolbar Features . . . 48
6.3 3.3 Project Explorer (Left Sidebar) . . . 48
6.3.1 File Tree Structure . . . 48
6.3.2 Opening Files . . . 48
6.3.3 Creating Files and Folders . . . 48
6.3.4 Renaming Files and Folders . . . 49
6.3.5 Deleting Files and Folders . . . 49
6.3.6 Cut, Copy, and Paste . . . 49
6.3.7 Drag-and-Drop Organization . . . 49
6.3.8 Context Menu Actions . . . 50
6.3.9 "Center on Canvas" Feature . . . 50
6.3.10 File Tree Search (if applicable) . . . 50
6.4 3.4 Story Elements Panel (Right Sidebar) . . . 50
6.4.1 Tab Navigation . . . 51
6.4.2 Characters Tab . . . 51
6.4.3 Variables Tab . . . 51
6.4.4 Images Tab (Image Asset Manager) . . . 51
6.4.5 Audio Tab (Audio Asset Manager) . . . 52
6.4.6 Screens Tab . . . 52
6.4.7 Composers Tab . . . 53
6.4.8 Menus Tab (Visual Menu Builder) . . . 53
6.4.9 Snippets Tab . . . 53
6.5 3.5 Split Pane Editor (Bottom) . . . 54
6.5.1 Opening Files . . . 54
6.5.2 Editor Panes . . . 54
6.5.3 Tab Management . . . 55
6.5.4 Monaco Editor Features . . . 55
6.5.5 Saving Changes . . . 55
6.6 3.6 Status Bar . . . 56
6.6.1 Left Side . . . 56
6.6.2 Center . . . 56
6.6.3 Right Side . . . 56
6.7 3.7 Keyboard Shortcuts . . . 56
6.7.1 Global Shortcuts . . . 56
6.7.2 Canvas Shortcuts . . . 57
6.7.3 Editor Shortcuts . . . 57
6.7.4 File Explorer Shortcuts . . . 57
6.7.5 Customizing Shortcuts . . . 57
6.8 3.8 Themes . . . 58
6.8.1 Available Themes . . . 58
6.8.2 Changing Themes . . . 58
6.8.3 Theme Persistence . . . 58
6.8.4 Dark Mode vs. Light Mode . . . 58

7 Section 4: Core Features Tour **59**
7.1 4.1 The Three Canvas System . . . 59
7.1.1 4.1.1 Story Canvas - File-Level View . . . 59

7.1.2 4.1.2 Route Canvas - Label-Level View . . . 62
7.1.3 4.1.3 Choice Canvas - Player Experience View . . . 64
7.1.4 4.1.4 Canvas Navigation (All Canvases) . . . 65
7.2 4.2 Code Editor . . . 66
7.2.1 4.2.1 Monaco Editor Basics . . . 66
7.2.2 4.2.2 Syntax Highlighting . . . 66
7.2.3 4.2.3 IntelliSense & Autocomplete . . . 67
7.2.4 4.2.4 Snippets . . . 67
7.2.5 4.2.5 Advanced Editor Features . . . 68
7.3 4.3 Project-Wide Search & Replace . . . 68
7.3.1 Opening the Search Panel . . . 68
7.3.2 Performing a Search . . . 69
7.3.3 Search Results . . . 69
7.3.4 Replace . . . 69
7.3.5 Search History . . . 69
7.3.6 Tips for Search & Replace . . . 69
7.4 4.4 Diagnostics Panel . . . 70
7.4.1 Opening Diagnostics . . . 70
7.4.2 Diagnostic Categories . . . 70
7.4.3 Diagnostic Details . . . 70
7.4.4 Jumping to Source . . . 70
7.4.5 Filtering Diagnostics . . . 71
7.4.6 Converting Diagnostics to Tasks . . . 71
7.4.7 Diagnostic Badges . . . 71
7.4.8 Tips for Using Diagnostics . . . 71
7.5 4.5 Project Statistics . . . 71
7.5.1 Opening Stats . . . 72
7.5.2 Deferred Loading . . . 72
7.5.3 Metrics Displayed . . . 72
7.5.4 Interpreting Stats . . . 72
7.5.5 Exporting Stats . . . 73
7.6 4.6 Undo/Redo System . . . 73
7.6.1 What Can Be Undone . . . 73
7.6.2 Using Undo . . . 73
7.6.3 Using Redo . . . 73
7.6.4 History Depth . . . 74
7.6.5 Visual Feedback . . . 74
7.6.6 Limitations . . . 74
7.6.7 Tips for Using Undo/Redo . . . 74

8 Section 5: For Writers **75**
8.1 5.1 Visualizing Your Narrative . . . 75
8.1.1 5.1.1 Using the Story Canvas for Narrative Structure . . . 75
8.1.2 5.1.2 Using the Route Canvas for Plot Structure . . . 76
8.1.3 5.1.3 Using the Choice Canvas for Player Experience . . . 77
8.1.4 5.1.4 Combining All Three Canvases . . . 77
8.2 5.2 Character Management . . . 78
8.2.1 5.2.1 Character Database (Characters Tab) . . . 78

8.2.2 5.2.2 Character Editor . . . 78
8.2.3 5.2.3 Dialogue Tracking . . . 79
8.3 5.3 Menu Builder & Choices . . . 79
8.3.1 5.3.1 Visual Menu Designer . . . 80
8.3.2 5.3.2 Adding and Configuring Choices . . . 80
8.3.3 5.3.3 Generating Menu Code . . . 80
8.3.4 5.3.4 Editing Existing Menus . . . 81
8.3.5 5.3.5 Integration with Choice Canvas . . . 81
8.4 5.4 Go-to-Label Navigation . . . 81
8.4.1 5.4.1 Command Palette (Ctrl+G) . . . 81
8.4.2 5.4.2 Toolbox Label Search . . . 82
8.4.3 5.4.3 Comparison: Palette vs. Toolbox . . . 82
8.5 5.5 Story Flow Visualization . . . 82
8.5.1 5.5.1 Understanding Arrows . . . 83
8.5.2 5.5.2 Block Classification . . . 83
8.5.3 5.5.3 Layout Strategies . . . 83
8.5.4 5.5.4 Using Sticky Notes . . . 84
8.6 5.6 Variable Tracking . . . 84
8.6.1 5.6.1 Variables Tab . . . 84
8.6.2 5.6.2 Understanding Variable Scope . . . 84
8.6.3 5.6.3 Using Variables in Choices . . . 85
8.7 5.7 Writing Tips & Workflow . . . 85
8.7.1 5.7.1 Starting a New Story . . . 85
8.7.2 5.7.2 Organizing Files and Labels . . . 86
8.7.3 5.7.3 Using Sticky Notes for Annotations . . . 86
8.7.4 5.7.4 Collaborative Workflow Considerations . . . 86
8.7.5 5.7.5 Best Practices for Large Projects . . . 87
8.7.6 5.7.6 Writer-Specific Keyboard Shortcuts . . . 87

9 Section 6: For Artists **88**
9.1 6.1 Image Asset Management . . . 88
9.1.1 6.1.1 Image Assets Tab Overview . . . 88
9.1.2 6.1.2 Scanning External Directories . . . 88
9.1.3 6.1.3 Importing Images . . . 89
9.1.4 6.1.4 Image Metadata . . . 89
9.1.5 6.1.5 Quick Code Generation . . . 89
9.1.6 6.1.6 Filtering and Searching Images . . . 90
9.1.7 6.1.7 Drag to Composers . . . 90
9.2 6.2 Audio Asset Management . . . 90
9.2.1 6.2.1 Audio Assets Tab Overview . . . 90
9.2.2 6.2.2 Scanning External Audio Directories . . . 91
9.2.3 6.2.3 Importing Audio . . . 91
9.2.4 6.2.4 Built-in Audio Player . . . 91
9.2.5 6.2.5 Audio Metadata . . . 91
9.2.6 6.2.6 Quick Code Generation . . . 91
9.2.7 6.2.7 Audio Workflow Tips for Artists . . . 92
9.3 6.3 Scene Composer . . . 92
9.3.1 6.3.1 Overview . . . 92

9.3.2 6.3.2 Creating a Composition . . . 92
9.3.3 6.3.3 Stage Controls . . . 93
9.3.4 6.3.4 Sprite Transforms . . . 93
9.3.5 6.3.5 Layer Ordering . . . 93
9.3.6 6.3.6 Code Generation . . . 94
9.3.7 6.3.7 Exporting as PNG . . . 94
9.3.8 6.3.8 Managing Compositions . . . 94
9.4 6.4 ImageMap Composer . . . 94
9.4.1 6.4.1 Overview . . . 95
9.4.2 6.4.2 Setting Ground and Hover Images . . . 95
9.4.3 6.4.3 Drawing Hotspots . . . 95
9.4.4 6.4.4 Configuring Hotspot Actions . . . 95
9.4.5 6.4.5 Code Generation . . . 95
9.4.6 6.4.6 Managing ImageMap Compositions . . . 96
9.5 6.5 Asset Preview Features . . . 96
9.5.1 6.5.1 Image Thumbnails . . . 96
9.5.2 6.5.2 Double-Click to Open in Editor . . . 96
9.5.3 6.5.3 Full-Size Image Preview . . . 97
9.5.4 6.5.4 Dimensions and File Size Display . . . 97
9.6 6.6 Drafting Mode for Artists . . . 97
9.6.1 6.6.1 What is Drafting Mode? . . . 97
9.6.2 6.6.2 Enabling Drafting Mode . . . 97
9.6.3 6.6.3 Why Artists Should Use This . . . 97
9.6.4 6.6.4 Workflow . . . 97
9.6.5 6.6.5 Placeholder Customization (If Available) . . . 98
9.7 6.7 Artist Workflow Tips . . . 98
9.7.1 6.7.1 Organizing Assets by Scene/Character . . . 98
9.7.2 6.7.2 Naming Conventions . . . 99
9.7.3 6.7.3 Using Tags for Filtering . . . 99
9.7.4 6.7.4 Coordinating with Writers . . . 99
9.7.5 6.7.5 Version Control for Assets . . . 99
9.7.6 6.7.6 Optimizing Assets . . . 100
9.7.7 6.7.7 Testing with Scene Composer . . . 100
9.7.8 6.7.8 Leveraging Quick Code Copy . . . 100

10 Section 7: For Developers **101**
10.1 7.1 Code Editor for Developers . . . 101
10.1.1 7.1.1 Monaco Editor Features . . . 101
10.1.2 7.1.2 Multi-File Editing Workflows . . . 101
10.1.3 7.1.3 Advanced Editing Techniques . . . 102
10.1.4 7.1.4 Find and Replace with Regex . . . 102
10.2 7.2 Syntax Highlighting . . . 103
10.2.1 7.2.1 TextMate Grammar . . . 103
10.2.2 7.2.2 Semantic Token Provider . . . 103
10.2.3 7.2.3 Customizing Colors (If Supported) . . . 103
10.3 7.3 Snippets Library . . . 103
10.3.1 7.3.1 Built-in Snippets Overview . . . 104
10.3.2 7.3.2 Using Snippets . . . 104

10.3.3 7.3.3 Creating Custom Snippets . . . 105
10.3.4 7.3.4 Snippet Management . . . 105
10.4 7.4 Screen Layout Composer . . . 106
10.4.1 7.4.1 Overview . . . 106
10.4.2 7.4.2 Available Widgets . . . 106
10.4.3 7.4.3 Building a Screen . . . 106
10.4.4 7.4.4 Asset Integration . . . 107
10.4.5 7.4.5 Code Generation . . . 107
10.4.6 7.9.2 Using Labels Effectively . . . 117
10.4.7 7.9.3 Leveraging Diagnostics for QA . . . 117
10.4.8 7.9.4 Version Control Integration (Git) . . . 117
10.4.9 7.9.5 Collaborative Development . . . 118
10.4.10 7.9.6 Testing Strategies . . . 118
10.4.11 7.9.7 Performance Optimization . . . 118

11 Section 8: Complete Feature Reference 119
11.1 8.1 Canvas Features . . . 119
11.1.1 Story Canvas . . . 119
11.1.2 Route Canvas . . . 120
11.1.3 Choice Canvas . . . 120
11.2 8.2 Code Editor Features . . . 121
11.2.1 Monaco Editor Core . . . 121
11.2.2 IntelliSense Suggestions . . . 121
11.2.3 Snippets (28+ Built-in) . . . 122
11.3 8.3 Story Elements Panel Tabs . . . 122
11.3.1 Characters Tab . . . 122
11.3.2 Variables Tab . . . 123
11.3.3 Images Tab (Image Asset Manager) . . . 123
11.3.4 Snd Tab (Audio Asset Manager) . . . 123
11.3.5 Screens Tab . . . 123
11.3.6 Composers Tab . . . 124
11.3.7 Menus Tab (Visual Menu Builder) . . . 124
11.3.8 Snippets Tab . . . 124
11.4 8.4 Visual Composers . . . 124
11.4.1 Scene Composer . . . 124
11.4.2 ImageMap Composer . . . 125
11.4.3 Screen Layout Composer . . . 125
11.5 8.5 Diagnostics . . . 125
11.5.1 Error Types (Red) . . . 125
11.5.2 Warning Types (Amber) . . . 126
11.5.3 Info Types (Blue) . . . 126
11.5.4 Actions . . . 126
11.6 8.6 Project Statistics . . . 126
11.6.1 Metrics . . . 126
11.6.2 Export Formats . . . 127
11.7 8.7 Project Explorer . . . 127
11.7.1 File Operations . . . 127
11.7.2 Context Menu Actions by File Type . . . 128

11.8 8.8 Search & Replace . . . 128
11.8.1 Features . . . 128
11.9 8.9 Settings . . . 128
11.9.1 General Settings . . . 128
11.9.2 Editor Settings . . . 129
11.9.3 Canvas Settings . . . 129
11.9.4 Ren'Py SDK Settings . . . 129
11.9.5 AI Settings . . . 129
11.9.6 Mouse Preferences . . . 130
11.108.10 Undo/Redo System . . . 130
11.10.1 What Can Be Undone . . . 130
11.10.2 Shortcuts . . . 130
11.10.3 Limitations . . . 130
11.118.11 Drafting Mode . . . 130
11.128.12 Keyboard Shortcuts (Complete List) . . . 131
11.12.1 Global . . . 131
11.12.2 Canvas . . . 131
11.12.3 Editor . . . 131
11.12.4 File Explorer . . . 132
11.12.5 Search Panel . . . 132
11.138.13 Themes (10 Available) . . . 132
11.13.1 Light Themes . . . 132
11.13.2 Dark Themes . . . 132
11.148.14 Auto-Updater . . . 133
11.158.15 New Project Wizard . . . 133
11.168.16 Run Game . . . 134
11.178.17 Markdown Preview . . . 134
11.188.18 AI Story Generator (If Configured) . . . 134

12 Section 9: Appendices **135**
12.1 Appendix A: Keyboard Shortcuts Reference . . . 135
12.1.1 Complete Shortcut Table . . . 135
12.2 Appendix B: Troubleshooting . . . 136
12.2.1 Installation Issues . . . 136
12.2.2 Performance Issues . . . 137
12.2.3 File System Issues . . . 138
12.2.4 Ren'Py SDK Integration . . . 138
12.2.5 Asset Management . . . 139
12.2.6 Code Editor Issues . . . 139
12.2.7 Canvas Issues . . . 140
12.3 Appendix C: Frequently Asked Questions . . . 140
12.3.1 General . . . 140
12.3.2 Features . . . 141
12.3.3 Compatibility . . . 141
12.3.4 Assets . . . 142
12.3.5 Advanced . . . 142
12.4 Appendix D: Glossary of Terms . . . 142
12.5 Appendix E: Resources & Links . . . 144

12.5.1 Official Resources . . . 144
12.5.2 Ren'Py Resources . . . 144
12.5.3 Community & Support . . . 144
12.5.4 Tutorials & Examples . . . 144
12.6 Appendix F: Credits & Acknowledgments . . . 145
12.6.1 Development Team . . . 145
12.6.2 Open Source Libraries . . . 145
12.6.3 Special Thanks . . . 145
12.6.4 License . . . 145
12.7 Document Metadata . . . 145
12.8 End of User Guide . . . 146

Ren'IDE

User Guide

The Visual IDE for Ren'Py Development

See Your Story

Version 0.7.1 Public Beta 4

April 2026

Blue Moon Foundry

Copyright © 2026 Blue Moon Foundry. All rights reserved.

Website: https://github.com/bluemoonfoundry/vangard-renpy-ide
Support: https://github.com/bluemoonfoundry/vangard-renpy-ide/issues

License: MIT License

Chapter 1

About This Guide

This user guide provides comprehensive documentation for Ren'IDE (Vangard Ren'Py IDE), covering all features, workflows, and best practices for visual novel development.

1.1 Who Should Read This Guide

- **Writers** creating branching narratives and managing complex storylines
- **Artists** managing assets, composing scenes, and creating visual content
- **Developers** writing code, building systems, and integrating with Ren'Py SDK
- **Solo Creators** wearing multiple hats and managing all aspects of development
- **Teams** collaborating on visual novel projects

1.2 What's Covered

This guide is organized into nine sections:

1. **Introduction** — Understanding Ren'IDE, its purpose, and benefits
2. **Getting Started** — Installation, first launch, and basic setup
3. **Interface Overview** — Complete UI tour and navigation
4. **Core Features Tour** — In-depth look at the three canvas system, editor, diagnostics, and statistics
5. **For Writers** — Narrative visualization, character management, and story flow tools
6. **For Artists** — Asset management, visual composers, and artwork workflows
7. **For Developers** — Advanced code editor, screen builder, and technical integration
8. **Complete Feature Reference** — Quick-reference tables for all features
9. **Appendices** — Shortcuts, troubleshooting, FAQ, glossary, and resources

1.3 Document Conventions

Throughout this guide, we use the following conventions:

- **Bold text** indicates UI elements, buttons, and menu items (e.g., **Settings**, **Save All**)
- `Monospace text` indicates code, file names, and keyboard shortcuts (e.g., `script.rpy`, `Ctrl+S`)
- *Italic text* indicates emphasis or new terms being introduced
- **Tip:** Helpful hints and best practices
- **Warning:** Important cautions to prevent issues
- **Note:** Additional information and context

1.4 Getting Help

If you encounter issues or have questions not covered in this guide:

- **GitHub Issues:** Report bugs or request features
- **Discussions:** Join the community discussion
- **Ren'Py Community:** Lemma Soft Forums

1.5 Feedback

We're continuously improving Ren'IDE and this documentation. If you have suggestions, corrections, or additions:

- Open an issue on GitHub
- Submit a pull request with improvements
- Contact us directly through the project repository

1.6 Acknowledgments

Ren'IDE is built on the shoulders of giants. We thank:

- **The Ren'Py Community** for creating an amazing visual novel engine
- **Open Source Contributors** for the libraries and tools that power Ren'IDE
- **Beta Testers** for their invaluable feedback
- **You** for choosing Ren'IDE for your visual novel development

---

Chapter 2

Quick Start Guide

New to Ren'IDE? Here's a 5-minute quick start to get you up and running.

2.1 1. Install Ren'IDE

Download the latest release: - Visit GitHub Releases - Download the installer for your platform (Windows .exe, macOS .dmg, Linux .AppImage) - Run the installer and follow the prompts

Detailed installation instructions are in Section 2: Getting Started

2.2 2. Launch and Open a Project

First launch: - Open Ren'IDE from your applications folder or start menu - You'll see the Welcome Screen - Click **"Open Existing Project"** to open a Ren'Py project folder - Or click **"Create New Project"** to start from scratch

2.3 3. Explore the Interface

Main areas: - **Center:** The Story Canvas showing your `.rpy` files as blocks - **Left:** Project Explorer with your file tree - **Right:** Story Elements panel with characters, images, audio, and more - **Bottom:** Code editor (appears when you open a file)

2.4 4. Navigate Your Story

Try these actions: - Drag blocks on the canvas to organize them - Double-click a block to open its file in the editor - Press **Ctrl+G** (or **Cmd+G** on Mac) to quickly jump to any label - Switch between **Story**, **Route**, and **Choice** canvas tabs to see different views

2.5 5. Start Creating

Common tasks: - Press **N** to create a new `.rpy` file - Right-click an image in the Images tab and select "Copy show statement" to insert it into your code - Use the Menu Builder (Menus tab) to visually design branching choices - Check the Diagnostics panel for errors and warnings

---

Ready to dive deeper? Continue to Section 1: Introduction for a comprehensive overview.

Chapter 3

Ren'IDE User Guide

The Visual IDE for Ren'Py Development

Version 0.7.1 Public Beta 4

3.1 Table of Contents

[Table of contents will be auto-generated by Pandoc]

Chapter 4

Section 1: Introduction

4.1 1.1 What is Ren'IDE?

Ren'IDE (short for **Ren'Py IDE**, marketed as **Vangard Ren'Py IDE**) is a desktop application designed to revolutionize how you develop visual novels with the Ren'Py engine. Unlike traditional text editors where your project exists as dozens of scattered `.rpy` files with no visual connection, Ren'IDE gives you a **bird's-eye view** of your entire story structure at a glance.

4.1.1 The Problem Ren'IDE Solves

Creating a visual novel in a plain text editor presents several challenges:

- **Lost in the files:** With dozens or hundreds of `.rpy` files, it's impossible to see the big picture. Where does this jump lead? Which scenes are connected? What's the player's path through the story?
- **Broken connections:** When you rename a label or reorganize your files, jumps can break silently. You might not discover the issue until you test that specific branch hours later.
- **Asset chaos:** Keeping track of which images and audio files are actually used in your project becomes a manual spreadsheet exercise. Missing assets cause runtime errors that are tedious to track down.
- **Collaboration friction:** Team members (writers, artists, programmers) work in silos because there's no shared visual representation of the project structure.

4.1.2 The Ren'IDE Solution

Ren'IDE addresses these challenges by providing:

Visual Story Structure Your `.rpy` files appear as draggable blocks on a canvas. Jump and call connections automatically draw as arrows between blocks. You can see the entire narrative

flow, reorganize your structure by dragging blocks, and understand your branching complexity at a glance.

Three Different Views - **Story Canvas** shows your project at the file level—one block per `.rpy` file - **Route Canvas** shows your project at the label level—one node per label, with every jump and call visualized - **Choice Canvas** shows your project from the player's perspective—what choices will they see, where do they lead, and what conditions guard them?

Integrated Code Editor A full-featured code editor (powered by the same engine as Visual Studio Code) is built right in. You get syntax highlighting designed specifically for Ren'Py, autocomplete for labels and assets, 28+ built-in code snippets, and the ability to edit multiple files side by side.

Intelligent Diagnostics Ren'IDE continuously analyzes your project and flags issues: jumps to labels that don't exist, references to missing images or audio files, unreachable story branches, unused characters, and more. Click any diagnostic to jump straight to the source file and line.

Visual Composers Build scenes, imagemaps, and screen layouts visually with drag-and-drop interfaces, then copy the generated Ren'Py code directly into your project. Perfect for prototyping or for team members who are more comfortable with visual tools.

Comprehensive Asset Management Browse all your project images and audio files with visual thumbnails, scan external directories to reference assets without copying them, and right-click any asset to instantly copy the appropriate Ren'Py code (`scene`, `show`, `play music`, etc.) to your clipboard.

4.1.3 What Ren'IDE Is NOT

It's important to understand what Ren'IDE doesn't change:

Ren'IDE is not a replacement for the Ren'Py SDK. You still need the Ren'Py SDK installed to run and build your game. Ren'IDE is a development environment that works *alongside* the SDK, not instead of it.

Ren'IDE does not use a proprietary format. Your `.rpy` files stay as `.rpy` files. Everything you create in Ren'IDE is standard Ren'Py code. You can open your project in any text editor, and you can open projects created in other editors in Ren'IDE. There is no lock-in.

Ren'IDE does not change how Ren'Py works. It's a visualization and editing tool. All the code it generates and all the files it creates are standard Ren'Py constructs. Your game will run exactly the same way whether you built it in Ren'IDE or in a plain text editor.

4.1.4 Core Philosophy

Ren'IDE's design philosophy is built on three pillars:

1. **Visualization makes complexity manageable.** Seeing your story structure as a graph makes it possible to understand and navigate projects that would be overwhelming as text alone.
2. **Non-destructive tooling.** Ren'IDE works with standard Ren'Py files and never modifies them in ways that break compatibility. You remain in full control of your project.
3. **Flexible workflow.** Whether you're a writer who wants to visualize narrative flow, an artist who needs asset management, or a programmer who wants advanced code editing tools, Ren'IDE adapts to your role and preferences.

4.2 1.2 Who Should Use Ren'IDE?

Ren'IDE is designed for anyone creating visual novels with Ren'Py, whether you're working solo or as part of a team. The features are organized around three primary roles, though many users will wear multiple hats.

4.2.1 For Writers

If you're responsible for crafting the narrative, Ren'IDE offers tools specifically designed for story development:

Visual Narrative Flow See the entire branching structure of your story on the Canvas. Understand which choices lead where, identify dead ends, and verify that every storyline reaches a satisfying conclusion.

Character Management Track all your characters in one place. See how many lines of dialogue each character has, find everywhere a character appears, and ensure no character is defined but never used.

Menu and Choice Builder Design branching choices with a visual menu builder. See all the player-facing text, configure conditional choices (choices that only appear if certain variables are set), and specify where each choice leads without writing code.

Go-to-Label Navigation Jump to any label in your project instantly with the command palette (Ctrl+G). No more scrolling through files trying to find that scene you wrote last week.

Label-Level Visualization Use the Route Canvas to trace the player's path through your story at the label level. Every `label`, `jump`, and `call` becomes a node or connection, making control flow crystal clear.

Dialogue Tracking See per-character dialogue counts and identify characters who might need more screen time or who might be over-represented in certain story sections.

Writers benefit from Ren'IDE because it makes the **structure** of the narrative visible, not just the text. You can plan, revise, and debug your story's flow without losing sight of the big picture.

4.2.2 For Artists

If you're creating the visual and audio assets for a visual novel, Ren'IDE provides dedicated tools for asset management and composition:

Image Asset Manager Browse all project images with visual thumbnails organized by folder. Drag images from external directories to import them, tag images for easy searching, and right-click any image to copy a `show` or `scene` statement to your clipboard.

Audio Asset Manager Manage music and sound effects with the same thumbnail-based browsing. Use the built-in audio player to preview sounds, and right-click to copy `play music` or `play sound` statements.

Scene Composer Lay out backgrounds and sprites visually on a stage. Adjust position, scale, rotation, flip, alpha, and blur for each sprite. When you're satisfied, copy the generated Ren'Py code (`scene` and `show` statements with transforms) directly into your script—or export the composition as a flat PNG for reference or social media.

ImageMap Composer Create clickable imagemaps by drawing rectangular hotspots over a ground image. Configure each hotspot to jump or call to a specific label, then copy the generated `imagemap` or `imagebutton` screen code.

Metadata and Organization Add Ren'Py tags to images (e.g., "eileen happy", "bg office"), organize assets into subfolders, and use custom tags for project-specific categorization. Ren'IDE makes it easy to find the exact asset you need among hundreds of files.

Drafting Mode Enable Drafting Mode to add placeholder images and audio for assets that haven't been created yet. This lets the game run during development even when some assets are still works in progress.

Artists benefit from Ren'IDE because it provides a **visual workspace** for asset management and composition, eliminating the need to manually write `show` statements or memorize file paths.

4.2.3 For Developers (Programmers)

If you're writing code, building systems, or managing technical aspects of the project, Ren'IDE offers advanced development tools:

Full-Featured Code Editor Edit Ren'Py code in a Monaco editor (the same engine as Visual Studio Code) with syntax highlighting specifically designed for Ren'Py, semantic token coloring for labels and variables, and multi-cursor editing for bulk changes.

IntelliSense Autocomplete Type `jump` and get autocomplete suggestions for every label in your

project. Type `show` and see a list of every image. Character names, screen names, and variables all autocomplete as you type.

Snippets Library Use 28+ built-in Ren'Py snippets for common patterns (menu, choice, show, scene, call, jump, etc.) with tab-stop placeholders. Create custom snippets for project-specific boilerplate and trigger them with custom prefixes.

Diagnostics Panel Get real-time error and warning detection: invalid jumps to missing labels, references to undefined characters or screens, missing image or audio assets, unreachable labels, syntax errors, and more. Click any diagnostic to jump to the file and line where the issue occurs.

Screen Layout Composer Build Ren'Py screens visually by dragging widgets (vbox, hbox, text, button, input, etc.) onto a canvas, nesting them to create layout hierarchies, and configuring properties per widget. Copy the generated `screen` code into your project.

Project Statistics Analyze your project with metrics like total word count, estimated play time, lines of dialogue per character (visualized as a bar chart), scene and route counts, and branching complexity scores.

Ren'Py SDK Integration Configure the path to your Ren'Py SDK installation and run your game directly from the Ren'IDE toolbar. No need to switch to a terminal or the Ren'Py launcher.

Advanced Canvas Features Use the Route Canvas to understand control flow at a granular level, identify unreachable code, and visualize cyclic structures (loops and recursion). Use the Choice Canvas to test the player experience and verify conditional logic.

Developers benefit from Ren'IDE because it combines the power of a **professional code editor** with **intelligent project analysis** and **visual debugging tools** designed specifically for Ren'Py.

4.2.4 For Solo Developers

If you're working on a visual novel by yourself and handling all three roles (writer, artist, programmer), Ren'IDE is designed to make your life easier:

- **Context switching is seamless.** Write code in the editor, switch to the Scene Composer to lay out a scene, drag an image from the asset manager, copy the generated code back to the editor—all without leaving the application.
- **The big picture stays visible.** Whether you're deep in dialogue text or debugging a technical issue, the Canvas views keep your narrative structure visible.
- **Asset management is integrated.** No need for separate tools to track which images and sounds you're using. Everything is in one place.
- **Diagnostics catch mistakes early.** When you rename a label, Ren'IDE immediately flags any jumps that are now broken. When you reference an image that doesn't exist, you see a warning before you run the game.

Solo developers benefit from Ren'IDE because it reduces the **cognitive load** of juggling multiple concerns and tools, letting you focus on creating your story.

4.2.5 For Teams

If you're working with other people—whether a small team or a larger studio—Ren'IDE provides a shared visual language for your project:

- **Non-technical team members can navigate.** Writers and artists who aren't comfortable in a text editor can use the Canvas views to understand the project structure and find specific scenes or files.
- **The Canvas is a communication tool.** During planning meetings, open the Story Canvas to discuss narrative flow. Use the Route Canvas to identify problematic branches. Use the Choice Canvas to review the player experience.
- **Asset handoffs are clear.** Artists can see exactly which images and audio files are referenced in the script. Programmers can see diagnostic warnings for missing assets.
- **Everyone works with the same files.** Because Ren'IDE uses standard `.rpy` files, there's no format conversion or export step. Commit your changes to version control and everyone sees the updates.

Teams benefit from Ren'IDE because it creates a **shared understanding** of the project that transcends individual roles and technical skill levels.

4.3 1.3 Key Benefits

4.3.1 See Your Story's Structure

The most immediate benefit of Ren'IDE is visualization. Instead of mentally reconstructing the flow of your story from dozens of text files, you see it as a graph:

- **File-level view (Story Canvas):** Each `.rpy` file is a block. Arrows show jumps and calls. You can arrange blocks spatially to reflect the narrative structure (e.g., main story on the left, side quests on the right).
- **Label-level view (Route Canvas):** Each label is a node. Edges show every possible transition. You can trace the path from `start` to any ending, see which labels are unreachable, and understand cyclical structures (repeated scenes or loops).
- **Player-facing view (Choice Canvas):** Menu nodes show the actual choice text the player will see. Colored pills lead to destination labels. Conditional badges indicate when choices are guarded by `if` statements.

With these three views, you can answer questions that are nearly impossible to answer in a text editor: - "Where does this jump go?" - "What are all the ways to reach this ending?" - "Which story branches are never visited?" - "How complex is my branching structure?" - "What does the player experience at this decision point?"

4.3.2 Navigate Instantly

Large Ren'Py projects can have hundreds of labels spread across dozens of files. Finding a specific label in a text editor means opening file after file and using text search. In Ren'IDE:

- Press **Ctrl+G** to open the Go-to-Label command palette, type a few letters of the label name, and jump directly to that label on the canvas with intelligent zoom.
- Use the label search box in the Canvas toolbox to filter and navigate to labels without a keyboard shortcut.
- Click a block on the Story Canvas to see all the labels it contains, then click a label to open the file in the editor with the cursor positioned on that line.
- Right-click a `.rpy` file in the Project Explorer and select "Center on Canvas" to instantly locate its block on the Story Canvas.

Navigation is instant and visual. You spend less time hunting for code and more time writing.

4.3.3 Catch Problems Early

Ren'IDE's diagnostics system continuously analyzes your project and flags issues before they cause runtime errors:

- **Invalid jumps:** A `jump` or `call` to a label that doesn't exist in your project.
- **Missing assets:** References to image or audio files that aren't found in your project directories.
- **Undefined characters:** Using a character tag in dialogue before defining it with `define [tag] = Character(...)`.
- **Undefined screens:** Calling or showing a screen that hasn't been defined.
- **Unused characters:** Defining a character that never speaks (possible leftover from cut content).
- **Unreachable labels:** Labels that no path from `start` leads to (dead code).
- **Syntax errors:** Code that Ren'Py won't be able to parse.

Each diagnostic includes: - Severity (error, warning, info) - Description of the issue - File path and line number - A clickable link to jump directly to the problem

By catching these issues during development, you avoid discovering them during playtesting or—worse—in a published build.

4.3.4 Compose Visually

Not everyone thinks in code. Ren'IDE provides three visual composers that let you design elements of your game graphically and then generate the corresponding Ren'Py code:

Scene Composer Drag backgrounds and sprites onto a stage. Adjust their position, scale, rotation, alpha, and other transforms. When you're satisfied, copy the generated `scene` and `show` statements into your script. This is ideal for prototyping scenes, creating reference compositions for artists, or for team members who are more comfortable working visually.

ImageMap Composer Draw clickable rectangular hotspots over an image (a map, a UI mockup, a point-and-click scene). Configure each hotspot's action (jump or call) and target label. Copy the generated `imagemap` or `imagebutton` screen code. This eliminates the tedious process of manually determining pixel coordinates.

Screen Layout Composer Build Ren'Py screen layouts by dragging widgets (containers like `vbox` and `hbox`, content like `text` and `image`, interactive elements like `textbutton` and `input`) onto a canvas and nesting them to create hierarchies. Configure widget properties in a side panel, then copy the generated `screen` code. This is much faster than hand-coding screen layouts and makes it easy to experiment with different UI structures.

4.3.5 Unified Environment

Before Ren'IDE, a typical Ren'Py workflow might involve: - A text editor (Atom, Sublime, VSCode) for code - A file browser or asset management tool for images and audio - A spreadsheet or document for tracking characters and variables - The Ren'Py launcher for running the game - Separate tools for testing, debugging, and analyzing the project

With Ren'IDE, all of these functions are integrated into a single application: - Code editor with Ren'Py-specific features - Image and audio asset browsers with thumbnails and playback - Character and variable databases with usage tracking - Canvas views for narrative visualization and navigation - Diagnostics panel for error detection - Statistics panel for project analysis - Run button to launch the game directly from the toolbar

This reduces context switching, speeds up your workflow, and keeps all project information in one place.

4.4 1.4 System Requirements

Ren'IDE is a desktop application built with Electron and React. It runs on Windows, macOS, and Linux. Here are the system requirements:

4.4.1 Minimum Requirements

Operating System: - **Windows:** Windows 10 (64-bit) or later - **macOS:** macOS 10.15 (Catalina) or later (Intel and Apple Silicon supported) - **Linux:** Recent distribution (Ubuntu 20.04+, Fedora 35+, or equivalent) with AppImage support

Hardware: - **Processor:** 64-bit CPU (Intel or AMD; Apple Silicon on macOS) - **RAM:** 4 GB minimum - **Storage:** 500 MB for the application, plus space for your project files - **Display:** 1280×720 resolution minimum

Software: - **Ren'Py SDK** (if you want to run your game from Ren'IDE; not required for editing)

4.4.2 Recommended Requirements

For a smoother experience, especially with larger projects:

Hardware: - **RAM:** 8 GB or more - **Storage:** SSD recommended for faster project loading - **Display:** 1920×1080 resolution or higher (more screen real estate for split pane editing and side panels)

Software: - **Ren'Py SDK 8.0 or later** for full compatibility with modern Ren'Py features

4.4.3 Platform-Specific Notes

Windows: - Windows SmartScreen may show a warning when you first run the installer. This is normal for unsigned applications. Click "More info" and "Run anyway" to proceed. - The application is installed by default in `C:\Program Files\Ren'IDE` but you can choose a custom location during installation.

macOS: - On first launch, macOS Gatekeeper may block the application with a message like "Ren'IDE cannot be opened because it is from an unidentified developer." To bypass this: 1. Right-click (or Control-click) the application in Finder 2. Select "Open" from the context menu 3. Click "Open" in the confirmation dialog - Ren'IDE requires permissions to access your file system. macOS will prompt you to grant access to the project directory when you open a project for the first time. - Both Intel and Apple Silicon (M1/M2/M3) Macs are supported. The application runs natively on Apple Silicon.

Linux: - Ren'IDE is distributed as an AppImage, which is a self-contained executable that runs on most modern Linux distributions without installation. - Make the AppImage executable: `chmod +x Vangard-RenPy-renide-*.AppImage` - Some distributions require FUSE to run AppImages. If the AppImage doesn't run, install FUSE: `sudo apt install fuse` (Ubuntu/Debian) or `sudo dnf install fuse` (Fedora). - For desktop integration (application menu entry, file associations), you can use tools like `appimaged` or manually create a `.desktop` file.

4.4.4 Internet Connection

Ren'IDE does not require an internet connection for core functionality. All editing, visualization, and project management features work offline.

An internet connection is required for: - **Auto-updater:** Checking for and downloading new versions of Ren'IDE - **AI Story Generator:** Using the AI content generation feature (requires API keys for Google Gemini, OpenAI, or Anthropic)

4.4.5 Storage Considerations

The Ren'IDE application itself is approximately 300-500 MB depending on the platform. Your project files are separate and can be stored anywhere on your file system.

Ren'IDE creates a small `.ide` directory in your project folder to store settings, compositions, and cached analysis results. This directory is typically a few hundred KB but can grow to a few MB for very large projects with many saved compositions.

4.4.6 Performance Considerations

For most projects (up to several hundred `.rpy` files and a few thousand labels), Ren'IDE performs well on minimum-spec hardware. However, very large projects may benefit from more RAM and faster storage:

- **Large projects (1000+ labels):** 8 GB RAM recommended for smooth canvas rendering and navigation
- **Complex graphs:** Projects with highly interconnected label graphs (lots of jumps and calls between labels) may experience slower layout computation. The Route Canvas layout algorithm can take several seconds for graphs with thousands of nodes and edges.
- **Many assets (1000+ images/audio):** Asset thumbnails are generated asynchronously, but initial loading may be slower on HDDs. An SSD significantly improves asset browsing performance.

If you experience performance issues with a large project, consider: - Closing unused editor tabs and panes - Filtering the canvas to show fewer blocks (e.g., use the character filter) - Using the Minimap for navigation instead of scrolling the full canvas

---

End of Section 1: Introduction

Chapter 5

Section 2: Getting Started

Welcome to Ren'IDE! This section walks you through installing the application on your platform, launching it for the first time, and opening or creating your first Ren'Py project.

5.1 2.1 Installation

Ren'IDE is distributed as a platform-specific installer or package. The installation process differs slightly depending on your operating system.

5.1.1 2.1.1 Windows Installation

Ren'IDE for Windows is distributed as an NSIS installer (`.exe` file).

Step 1: Download the Installer

1. Visit the releases page on GitHub
2. Scroll down to the "Assets" section
3. Download the file named `Vangard-RenPy-renide-Setup-[version].exe` (e.g., `Vangard-RenPy-renide-Set`
4. Wait for the download to complete (file size is approximately 150-200 MB)

Step 2: Extract the ZIP (if downloaded as .zip)

If you downloaded a `.zip` file containing the installer:

1. Right-click the downloaded `.zip` file
2. Select "Extract All…"
3. Choose a destination folder (e.g., your Downloads folder)
4. Click "Extract"
5. Open the extracted folder to find the `.exe` installer

Step 3: Run the Installer

1. Double-click the `.exe` installer file
2. **Windows SmartScreen warning:** If you see a "Windows protected your PC" message:
   - This is normal for applications without an expensive code-signing certificate
   - Click "More info"
   - Click "Run anyway"
3. If prompted by User Account Control (UAC), click "Yes" to allow the installer to make changes

Step 4: Installer Wizard

The NSIS installer will open with a setup wizard:

1. **Welcome screen:** Click "Next"
2. **License Agreement:** Read the license and click "I Agree" (if you agree)
3. **Installation Folder:**
   - Default: `C:\Program Files\Ren'IDE`
   - To change: Click "Browse" and select a different location
   - Click "Next"
4. **Start Menu Folder:**
   - Default: "Ren'IDE"
   - You can change this or leave it as-is
   - Check "Create a desktop shortcut" if you want a desktop icon
   - Click "Next"
5. **Installation:**
   - Click "Install" to begin copying files
   - Wait for the installation to complete (this takes 30-60 seconds)
6. **Completion:**
   - Check "Run Ren'IDE" if you want to launch immediately
   - Click "Finish"

Step 5: First Launch

If you checked "Run Ren'IDE" in the installer, the application will launch automatically. Otherwise:

1. Open the Start menu
2. Search for "Ren'IDE" or find it in the "Ren'IDE" folder
3. Click the Ren'IDE icon

On the first launch, you may see a Windows Firewall prompt. You can safely click "Allow access" or "Cancel" (Ren'IDE does not require network access for core functionality).

Uninstalling (Windows)

To uninstall Ren'IDE:

1. Open Settings → Apps → Installed apps (Windows 11) or Control Panel → Programs and Features (Windows 10)
2. Find "Ren'IDE" in the list
3. Click "Uninstall"
4. Follow the uninstaller prompts

5.1.2 2.1.2 macOS Installation

Ren'IDE for macOS is distributed as a DMG disk image.

Step 1: Download the DMG

1. Visit the releases page on GitHub
2. Scroll down to the "Assets" section
3. Download the file named `Vangard-RenPy-renide-[version].dmg` (e.g., `Vangard-RenPy-renide-0.7.1.dmg`
4. Wait for the download to complete (file size is approximately 150-200 MB)

Note: The DMG is a universal binary that works on both Intel and Apple Silicon (M1/M2/M3) Macs.

Step 2: Extract the ZIP (if downloaded as .zip)

If you downloaded a `.zip` file:

1. Locate the `.zip` file in your Downloads folder
2. Double-click to extract it
3. The `.dmg` file will appear in the same folder

Step 3: Open the DMG

1. Double-click the `.dmg` file
2. A Finder window will open showing the Ren'IDE application icon and an Applications folder shortcut
3. **Drag the Ren'IDE icon to the Applications folder**
4. Wait for the copy operation to complete

Step 4: Eject the DMG

1. In Finder, locate the mounted DMG volume (usually named "Ren'IDE" in the sidebar)
2. Right-click and select "Eject" (or click the eject button next to its name)

Step 5: First Launch

1. Open Finder and navigate to your Applications folder
2. Locate "Ren'IDE" (it may be named "Vangard-RenPy-renide")
3. **Right-click (or Control-click) the application**

4. Select "Open" from the context menu
5. **Gatekeeper warning:** You'll see a dialog saying "Ren'IDE is an app downloaded from the Internet. Are you sure you want to open it?"
   - This is normal for applications downloaded from outside the Mac App Store
   - Click "Open"
6. On subsequent launches, you can simply double-click the application; you only need to use the right-click method on the first launch

macOS Permissions

When you first open a project in Ren'IDE, macOS may prompt you to grant file system access:

1. A system dialog will appear asking for permission to access files in your project folder
2. Click "OK" or "Allow" to grant access
3. If you accidentally deny access, you can grant it later in System Preferences → Security & Privacy → Files and Folders → Ren'IDE

Adding to Dock

For quick access:

1. With Ren'IDE running, its icon will appear in the Dock
2. Right-click the icon
3. Select Options → Keep in Dock

Uninstalling (macOS)

To remove Ren'IDE:

1. Open Finder and navigate to Applications
2. Locate "Ren'IDE" or "Vangard-RenPy-renide"
3. Drag it to the Trash
4. Empty the Trash

5.1.3 2.1.3 Linux Installation

Ren'IDE for Linux is distributed as an AppImage, a self-contained executable that runs on most modern distributions without traditional installation.

Step 1: Download the AppImage

1. Visit the releases page on GitHub
2. Scroll down to the "Assets" section
3. Download the file named `Vangard-RenPy-renide-[version].AppImage` (e.g., `Vangard-RenPy-renide-0.7.`
4. Wait for the download to complete (file size is approximately 150-200 MB)

Step 2: Extract the ZIP (if downloaded as .zip)

If you downloaded a `.zip` file:

```
cd ~/Downloads
unzip Vangard-RenPy-renide-*.zip
```

The AppImage will be extracted to the current directory.

Step 3: Make the AppImage Executable

AppImages must have executable permissions to run. Open a terminal and run:

```
cd ~/Downloads  # or wherever you downloaded the file
chmod +x Vangard-RenPy-renide-*.AppImage
```

Step 4: Run the AppImage

There are two ways to run the AppImage:

Option A: Double-click in file manager

1. Open your file manager (Nautilus, Dolphin, Thunar, etc.)
2. Navigate to the directory containing the AppImage
3. Double-click the AppImage file
4. If prompted, select "Run" or "Execute"

Option B: Run from terminal

```
./Vangard-RenPy-renide-*.AppImage
```

FUSE Requirement

Some Linux distributions require FUSE (Filesystem in Userspace) to run AppImages. If you get an error about FUSE when trying to run the AppImage:

Ubuntu/Debian:

```
sudo apt install fuse libfuse2
```

Fedora/RHEL:

```
sudo dnf install fuse fuse-libs
```

Arch Linux:

```
sudo pacman -S fuse2
```

After installing FUSE, try running the AppImage again.

Step 5: Optional - Move to a Permanent Location

For convenience, you may want to move the AppImage to a more permanent location:

```
mkdir -p ~/Applications
mv Vangard-RenPy-renide-*.AppImage ~/Applications/
```

You can then run it with:

```
~/Applications/Vangard-RenPy-renide-*.AppImage
```

Desktop Integration (Optional)

To add Ren'IDE to your application menu:

Option A: Use AppImageLauncher (recommended)

AppImageLauncher is a utility that integrates AppImages into your desktop environment:

1. Install AppImageLauncher from its GitHub page or your distribution's package manager
2. Run the Ren'IDE AppImage
3. AppImageLauncher will prompt you to integrate it; click "Yes"
4. Ren'IDE will now appear in your application menu

Option B: Manual .desktop file

Create a file `~/.local/share/applications/renide.desktop` with this content:

```
[Desktop Entry]
Type=Application
Name=Ren'IDE
Comment=Visual IDE for Ren'Py Development
Exec=/path/to/Vangard-RenPy-renide-*.AppImage
Icon=/path/to/icon.png
Categories=Development;
Terminal=false
```

Replace `/path/to/` with the actual path to your AppImage and icon.

Uninstalling (Linux)

Since AppImages don't install in the traditional sense, simply delete the AppImage file:

```
rm ~/Applications/Vangard-RenPy-renide-*.AppImage
```

If you created a `.desktop` file:

```
rm ~/.local/share/applications/renide.desktop
```

5.1.4 2.1.4 Building from Source (Optional)

If you want to build Ren'IDE from source (for development, customization, or if pre-built binaries aren't available for your platform), follow these steps.

Prerequisites

- Node.js version 18 or newer
- npm (bundled with Node.js)
- Git (to clone the repository)

Step 1: Clone the Repository

```
git clone https://github.com/bluemoonfoundry/vangard-renpy-ide.git
cd vangard-renpy-ide
```

Step 2: Install Dependencies

```
npm install
```

This will download and install all required packages. It may take several minutes depending on your internet connection.

Step 3: Build and Run

Development mode (run without building):

```
npm run electron:start
```

This builds the React app and launches Electron. The app will open in a window.

Alternatively, run Vite dev server only:

```
npm run dev
```

This starts a Vite development server at `http://localhost:5173`. You can open this in a browser, but some features (like file system access via Electron APIs) won't work in browser mode.

Step 4: Build a Distributable (Optional)

To create a distributable package for your platform:

```
npm run dist
```

This uses electron-builder to create: - **Windows:** `.exe` installer in `release/` - **macOS:** `.dmg` disk image in `release/` - **Linux:** `.AppImage` in `release/`

The build process takes several minutes. The output will be in the `release/` directory.

Running Tests

To run the test suite:

```
npm test                # Run all tests once
npm run test:watch      # Run in watch mode
npm run test:coverage   # Run with coverage report
```

Linting

To check and fix code style issues:

```
npm run lint            # Check for issues
npm run lint:fix        # Automatically fix issues
```

5.2 2.2 First Launch

When you launch Ren'IDE for the first time, you'll see the **Welcome Screen**.

5.2.1 The Welcome Screen

The Welcome Screen has two main sections:

Open Existing Project

If you already have a Ren'Py project (a folder containing `.rpy` files):

1. Click the **"Open Existing Project"** button
2. A file picker dialog will appear
3. Navigate to your project's root directory (the folder that contains the `game/` subdirectory)
4. Select the folder and click "Open" or "Select Folder"

Ren'IDE will: - Scan the `game/` directory for `.rpy` files - Parse and analyze all Ren'Py code - Build the file tree, canvas, and story elements - Open the project in the main interface

This initial analysis may take a few seconds for small projects or up to a minute for very large projects (hundreds of files).

Create New Project

If you're starting a new visual novel from scratch:

1. Click the **"Create New Project"** button
2. The New Project Wizard will open (see Section 2.4 below for details)

Recent Projects

If you've opened projects before, the Welcome Screen will also show a list of recently opened projects. Click any project in the list to open it immediately without navigating the file picker.

Dismissing the Welcome Screen

After you open or create a project, the Welcome Screen will close and you'll see the main Ren'IDE interface. You can return to the Welcome Screen by closing your current project (File → Close Project) or by clicking the Ren'IDE logo in the top-left corner.

5.2.2 First Launch Permissions

Depending on your operating system, you may be prompted for permissions on first launch:

Windows: - Windows Defender Firewall may ask if you want to allow Ren'IDE to access the network. This is optional; Ren'IDE works fine without network access (except for the auto-updater and AI features).

macOS: - macOS will ask for permission to access files in your project directory. Click "OK" to grant access. - If you deny access, Ren'IDE won't be able to read or write project files. You can grant access later in System Preferences → Security & Privacy → Files and Folders → Ren'IDE.

Linux: - No special permissions are required. If you placed the AppImage in a location without execute permissions, you may need to `chmod +x` the file (see Section 2.1.3).

5.3 2.3 Opening an Existing Project

If you have an existing Ren'Py project—whether you created it in the Ren'Py launcher, in a text editor, or in Ren'IDE previously—you can open it in Ren'IDE.

5.3.1 What Ren'IDE Looks For

Ren'IDE expects a standard Ren'Py project structure:

```
your-project/
  game/
      script.rpy
      options.rpy
      screens.rpy
      (other .rpy files)
      images/
          (your image assets)
      audio/
          (your audio assets)
  .ide/  (created by Ren'IDE)
  project.ide.json  (created by Ren'IDE)
```

Specifically: - A `game/` subdirectory containing `.rpy` files - Optionally, `game/images/` and `game/audio/` directories for assets

Ren'IDE does not require any special setup. Any Ren'Py project will work.

5.3.2 Opening a Project

From the Welcome Screen:

1. Click "Open Existing Project"
2. Navigate to your project's root folder (the one that contains `game/`)
3. Click "Open" or "Select Folder"

From the File Menu (if you already have a project open):

1. Click File → Open Project (or press Ctrl+O / Cmd+O)
2. Navigate to the new project's root folder
3. Click "Open" or "Select Folder"

5.3.3 Initial Project Analysis

When you open a project, Ren'IDE performs a comprehensive analysis:

1. **File tree population:** Scans the `game/` directory recursively to build the file tree
2. **Code parsing:** Reads every `.rpy` file and parses:
   - Labels
   - Jumps and calls
   - Character definitions
   - Variables (define and default)
   - Screen definitions
   - Image definitions
   - Dialogue lines
3. **Canvas generation:** Creates blocks for each `.rpy` file and draws arrows for jumps and calls
4. **Route graph construction:** Builds the label-level graph for the Route Canvas
5. **Asset scanning:** Scans `game/images/` and `game/audio/` for asset files
6. **Diagnostic generation:** Analyzes the project for errors and warnings

For small projects (< 50 files, < 500 labels), this takes a few seconds. For large projects (hundreds of files, thousands of labels), it can take 30-60 seconds. A loading overlay with a spinner will appear while analysis is in progress.

5.3.4 Understanding the Canvas View

After analysis, you'll see:

- **Story Canvas (center):** Each `.rpy` file is represented as a draggable block. Arrows show jumps and calls between files. If this is the first time opening the project, blocks will be positioned automatically.
- **Project Explorer (left sidebar):** A tree view of your project's file structure. You can navigate, create, rename, and delete files and folders here.
- **Story Elements (right sidebar):** Tabs showing characters, variables, images, audio, screens, and more extracted from your code.
- **Editor Pane (bottom):** Initially empty. Double-click a block or file to open it in the code editor.

5.3.5 Persistence

Ren'IDE saves some project-specific data in two places:

1. **project.ide.json** (in your project root):
   - Canvas block positions
   - Saved compositions (Scene Composer, ImageMap Composer, Screen Composer)
   - Image and audio metadata (tags, Ren'Py names)
   - Scan directory handles
   - Project-specific settings (active canvas, open tabs, etc.)
2. **.ide/** directory (in your project root):
   - Cached analysis results (to speed up subsequent opens)
   - Temporary files

These files are automatically created the first time you save changes in Ren'IDE. You can safely commit `project.ide.json` to version control if you want to share canvas layouts and compositions with teammates. The `.ide/` directory can be added to `.gitignore`.

5.4 2.4 Creating a New Project

If you're starting a new visual novel from scratch, Ren'IDE includes a New Project Wizard that generates a complete Ren'Py project structure.

5.4.1 Starting the Wizard

From the Welcome Screen:

1. Click **"Create New Project"**

From the File Menu:

1. Click File → New Project (if you have a project already open)

The New Project Wizard will open. It has three steps:

5.4.2 Step 1: Project Name and Location

Project Name:

- Enter a name for your project (e.g., "My First Visual Novel")
- This will be used as:
  - The project folder name (with spaces replaced by hyphens or underscores)
  - The default game title in `options.rpy`

Project Location:

- Click "Browse" to select a parent directory where the project folder will be created
- Example: If you choose `C:\Users\YourName\RenpyProjects` and name your project "My Game", the full path will be `C:\Users\YourName\RenpyProjects\my-game\`

Click **"Next"** to continue.

5.4.3 Step 2: Resolution Presets

Choose the default display resolution for your visual novel. This affects the initial canvas size and is written to `options.rpy`.

Presets:

- **1920 × 1080 (Full HD)** — Recommended for modern PCs and consoles
- **1280 × 720 (HD)** — Good balance between quality and file size
- **1024 × 768 (XGA)** — Classic visual novel resolution
- **800 × 600 (SVGA)** — Retro or minimalist games
- **Custom** — Enter your own width and height

You can change this later by editing `options.rpy`.

Click **"Next"** to continue.

5.4.4 Step 3: Theme and Accent Color

Theme:

- Select a theme for Ren'IDE's interface from the dropdown
- Choose from 10 themes: System Light, System Dark, Colorful Light, Colorful Dark, Neon Dark, Ocean Dark, Candy Light, Forest Light, Solarized Light, Solarized Dark
- This is a personal preference and does not affect your generated project

Accent Color:

- Click the color picker to choose a default accent color for your game's UI
- This is written to `options.rpy` as `config.accent_color`
- You can change it later

Click **"Create Project"** to finish.

5.4.5 Project Generation

Ren'IDE will:

1. Create the project directory structure:

```
my-game/
   game/
      script.rpy       (initial story script with a start label)
      options.rpy      (generated with your resolution and accent color)
      gui.rpy          (default GUI settings)
      screens.rpy      (default Ren'Py screens)
      images/          (empty directory for your images)
      audio/           (empty directory for your audio)
   project.ide.json     (Ren'IDE settings)
```

2. Populate `script.rpy` with a minimal start label:

```
label start:
    "Welcome to my visual novel!"
    "This is the start of your story."
    return
```

3. Configure `options.rpy` with:
   - Your chosen resolution
   - Your chosen accent color
   - A default game title (your project name)
   - Version number (0.1)
4. Open the new project in Ren'IDE

You'll see: - A single block on the Story Canvas labeled `script.rpy` - An empty Project Explorer tree (just the `game/` folder and its subdirectories) - Empty Story Elements tabs (no characters, variables, or assets yet)

You're now ready to start writing your visual novel!

5.4.6 Next Steps After Creating a Project

1. **Write your story:** Double-click the `script.rpy` block to open it in the code editor. Start writing labels, dialogue, and choices.
2. **Add assets:** Drag images into the `game/images/` folder (via your OS file browser) and audio into `game/audio/`. They'll appear in the respective Story Elements tabs.
3. **Create new files:** Click the "Add Block" button (or press `N`) to create additional `.rpy` files for organizing your story.
4. **Configure Ren'Py SDK:** To run your game from Ren'IDE, configure the SDK path (see Section 2.5).

5.5 2.5 Configuring Ren'Py SDK Path

Ren'IDE is a development environment, not a game engine. To actually run and test your visual novel, you need the **Ren'Py SDK** installed separately.

5.5.1 Why You Need the Ren'Py SDK

The Ren'Py SDK: - Runs your game for testing - Builds distributable packages (Windows .exe, macOS .app, Linux, Android, iOS, web) - Provides the Ren'Py launcher for project management

Ren'IDE complements the SDK by providing: - Visual story structure and navigation - Advanced code editing with IntelliSense and snippets - Asset management - Diagnostics and analytics

You use both together: develop in Ren'IDE, run and build with the SDK.

5.5.2 Downloading and Installing the Ren'Py SDK

If you don't have the Ren'Py SDK yet:

1. Visit https://www.renpy.org/
2. Click "Download" in the top navigation
3. Download the SDK for your platform (Windows, macOS, or Linux)
4. Extract the downloaded archive to a permanent location (e.g., `C:\RenpySDK\` on Windows, `~/RenpySDK/` on macOS/Linux)
5. Do not move or rename the extracted folder after installing; Ren'IDE needs a stable path

The SDK does not require installation in the traditional sense. It's a portable application that runs from wherever you extract it.

5.5.3 Configuring the Path in Ren'IDE

Once you have the SDK installed:

Step 1: Open Settings

- Click the **Settings** button in the toolbar (gear icon), or
- Press `Ctrl+,` (Windows/Linux) or `Cmd+,` (macOS), or
- Click File → Settings in the menu

Step 2: Navigate to Ren'Py SDK Settings

- The Settings modal has multiple sections on the left sidebar
- Click **"Ren'Py SDK"**

Step 3: Enter the SDK Path

- In the "SDK Path" field, enter the full path to your Ren'Py SDK installation
- This should be the directory that contains `renpy.exe` (Windows), `renpy.app` (macOS), or `renpy.sh` (Linux)

Examples: - **Windows:** `C:\RenpySDK\renpy-8.1.0-sdk` - **macOS:** `/Users/yourname/RenpySDK/renpy-8.1.0-` - **Linux:** `/home/yourname/RenpySDK/renpy-8.1.0-sdk`

Alternatively, use the "Browse" button:

1. Click "Browse"
2. Navigate to your Ren'Py SDK directory
3. Select the folder and click "Open"

Step 4: Test the Configuration

- Click the **"Test SDK Path"** button
- Ren'IDE will check if the path is valid and if the Ren'Py executable can be found
- If successful, you'll see a green checkmark and message: "SDK path is valid"
- If unsuccessful, you'll see an error message explaining the problem (e.g., "Path does not exist" or "renpy executable not found")

Step 5: Save Settings

- Click "Save" or "Apply" to save your settings
- Close the Settings modal

5.5.4 Running Your Game from Ren'IDE

Once the SDK path is configured:

1. Make sure your project is open in Ren'IDE
2. Click the **"Run"** button in the toolbar (play icon)

3. Ren'IDE will:
    - Save all unsaved files
    - Launch the Ren'Py SDK with your project
    - Open the game window

The game runs as a child process. When you close the game window, control returns to Ren'IDE.

Console Output (if applicable):

Depending on your Ren'Py version and platform, you may see console output in a terminal window (Linux) or the Ren'Py log (Windows/macOS). Check this output for runtime errors or debug messages.

5.5.5 Troubleshooting SDK Configuration

"Path does not exist"

- Double-check the path you entered
- Make sure you're pointing to the extracted SDK directory, not a zip file
- Verify the directory exists by opening it in your file browser

"renpy executable not found"

- The path you entered exists, but it doesn't contain a Ren'Py executable
- Make sure you're pointing to the SDK root directory (the one that contains `renpy.exe`, `renpy.app`, or `renpy.sh`), not a subdirectory like `lib/` or `renpy/`

"Permission denied" (Linux/macOS)

- The Ren'Py executable may not have execute permissions
- Run: `chmod +x /path/to/renpy-sdk/renpy.sh` (Linux) or `chmod +x /path/to/renpy-sdk/renpy.app/Co` (macOS)

"SDK version not supported"

- Ren'IDE works best with Ren'Py 7.5 or later
- Very old versions of Ren'Py (pre-7.0) may not work correctly
- Download the latest SDK from renpy.org

Game doesn't launch when clicking "Run"

- Make sure the SDK path is configured and tested (green checkmark)
- Save all unsaved files before running
- Check the Ren'IDE status bar for error messages
- Try launching the game manually from the Ren'Py launcher to verify the project is valid

---

End of Section 2: Getting Started

Chapter 6

Section 3: Interface Overview

Now that you have Ren'IDE installed and your first project open, let's explore the interface. This section provides a comprehensive tour of every major UI element, explaining what each component does and how to use it effectively.

6.1 3.1 Application Layout

Ren'IDE's interface is organized into several distinct areas, each serving a specific purpose. Understanding this layout will help you navigate efficiently and find the tools you need.

6.1.1 Main Layout Areas

```
  Toolbar (Top)


Project                                          Story
Explorer          Canvas Area                  Elements
 (Left)                                         (Right)
                 (Story/Route/Choice)



              Split Pane Editor (Bottom)
            (Monaco code editor, optional)


  Status Bar (Bottom)
```

Top Toolbar: Contains action buttons for common operations (save, undo/redo, add block, run game, settings, etc.). Always visible regardless of which canvas or panel is active.

Canvas Area (Center): The main workspace. Displays one of three canvas views: - Story Canvas (file-level blocks with jump/call arrows) - Route Canvas (label-level control flow graph) - Choice Canvas (player-facing choice view)

Switch between canvases using the tabs in the toolbar or keyboard shortcuts. This area is where you'll spend most of your time visualizing and navigating your story structure.

Project Explorer (Left Sidebar): A collapsible file tree showing your project's directory structure. Create, rename, delete, and organize files and folders. Right-click files for context menu options like "Center on Canvas."

Story Elements Panel (Right Sidebar): A collapsible multi-tab panel showing project data extracted from your code: Characters, Variables, Images, Audio, Screens, Composers, Menus, and Snippets. This is your central hub for managing story elements and assets.

Split Pane Editor (Bottom): An optional code editing area that can be shown or hidden. Supports side-by-side editing of multiple files. Double-click a block on the canvas or a file in the Project Explorer to open it here.

Status Bar (Bottom): Displays contextual information: current file name, cursor position (line and column), project analysis status, and application version.

6.1.2 Collapsible Panels

Both the Project Explorer (left) and Story Elements panel (right) can be collapsed to maximize canvas space:

- Click the **arrow icon** at the edge of each panel to collapse/expand
- When collapsed, only a thin strip remains visible; click to expand
- Panel states are preserved when you close and reopen the project

6.1.3 Responsive Design

Ren'IDE's interface adapts to different screen sizes:

- **Minimum resolution (1280×720):** All panels are visible but may feel cramped. Consider collapsing one sidebar when working on the canvas.
- **Recommended resolution (1920×1080 or higher):** All panels comfortably visible. The split pane editor has enough height to show 20-30 lines of code.

- **Ultrawide displays:** Take advantage of side-by-side code editing in the split pane, or use the extra space to keep both sidebars open while working.

6.2 3.2 Toolbar Reference

The toolbar at the top of the window provides quick access to frequently used actions. Buttons are organized into logical groups.

6.2.1 Undo/Redo Group

Undo (Ctrl+Z / Cmd+Z) - Icon: Curved arrow pointing left - Reverts the most recent change - Works for: - Canvas block moves - Block creation/deletion - Composition edits (Scene/ImageMap/Screen Composer) - Code changes in the editor - Grayed out when there's nothing to undo

Redo (Ctrl+Y / Cmd+Y / Ctrl+Shift+Z) - Icon: Curved arrow pointing right - Re-applies a change that was undone - Only available after using Undo - Grayed out when there's nothing to redo

Tip: Ren'IDE maintains a history of your actions. You can undo multiple steps by pressing Ctrl+Z repeatedly.

6.2.2 Canvas Actions Group

Add Block (N) - Icon: Plus sign or document with a plus - Creates a new `.rpy` file on the canvas - Opens a dialog where you specify: - File name - Block type (story, screen, other) - Initial content (empty or boilerplate) - The new block appears at the canvas center or near your current viewport

Add Note - Icon: Sticky note or comment icon - Adds a sticky note annotation to the canvas - Use notes to: - Leave reminders for yourself or teammates - Mark sections of the story for revision - Document narrative structure directly on the canvas - Notes don't affect your code; they're saved in `project.ide.json`

Redraw - Icon: Circular arrows or layout grid - Triggers automatic layout of canvas blocks - Analyzes jump/call connections and arranges blocks hierarchically - Useful when: - Blocks are overlapping or cluttered - You've added many new blocks - You want to reset the layout to a clean state - **Note:** This replaces manual block positioning. Use with caution if you've carefully arranged blocks.

6.2.3 Panels Group

Diagnostics - Icon: Exclamation mark in a circle or alert icon - Opens the Diagnostics panel - Shows: - Errors (red): broken jumps, missing assets, syntax errors - Warnings (amber): unused

characters, unreachable labels - Info (blue): suggestions and best practices - Click any diagnostic to jump to the source file and line - Badge shows total issue count (e.g., "3" for 3 diagnostics)

Stats - Icon: Bar chart or analytics icon - Opens the Project Statistics panel - Displays metrics: - Word count and estimated play time - Dialogue counts per character (bar chart) - Scene and route counts - Branching complexity scores - Asset coverage - Metrics compute asynchronously (you'll see spinners while calculations run)

6.2.4 Canvas Tabs

Story / Route / Choice - Three tabs to switch between canvas views - **Story:** File-level view (blocks for .rpy files) - **Route:** Label-level view (nodes for labels, edges for jumps/calls) - **Choice:** Player-facing view (menu nodes with choice pills) - Active tab is highlighted - Clicking a tab switches the canvas immediately

6.2.5 Run Group

Drafting Mode (Toggle) - Icon: Construction cone or draft icon - Enables/disables placeholder assets - When ON (highlighted): - Ren'Py placeholders are added for missing images and audio - Your game can run even if some assets haven't been created yet - Visual indicator (badge or highlight) shows mode is active - When OFF (default): - Missing assets cause runtime errors (normal Ren'Py behavior) - Useful during early development or when collaborating with artists

Run (Play icon) - Launches your game in the Ren'Py SDK - Requires SDK path to be configured (see Section 2.5) - Process: 1. Saves all unsaved files 2. Invokes the Ren'Py executable with your project path 3. Game window opens; Ren'IDE waits in the background 4. When you close the game, control returns to Ren'IDE - If SDK path isn't configured, clicking this button opens Settings

6.2.6 File Actions Group

Save All (Ctrl+S / Cmd+S) - Icon: Floppy disk - Saves all unsaved changes across: - Open editor tabs (code files) - Canvas block positions - Compositions (Scene/ImageMap/Screen) - Project settings (project.ide.json) - Unsaved changes are indicated by: - Modified indicator (dot or asterisk) on editor tabs - "Unsaved changes" message in status bar - Ren'IDE does NOT auto-save by default; save frequently!

Settings (Gear icon) - Opens the Settings modal - Sections: - General: Theme, auto-save, undo history depth - Editor: Font size, tab size, word wrap - Canvas: Pan mode, zoom sensitivity - Ren'Py SDK: SDK path configuration - AI: API provider and key management - Mouse: Preferences for canvas interaction - Changes take effect immediately or on "Apply" - Some settings are project-specific, others are global

6.2.7 Additional Toolbar Features

Application Logo (Top-Left) - Clicking the Ren'IDE logo returns you to the Welcome Screen - Use this to switch projects or create a new project

Window Controls (Top-Right, platform-dependent) - Minimize, Maximize/Restore, Close (standard window controls) - On macOS, these are in the native window title bar

---

6.3 3.3 Project Explorer (Left Sidebar)

The Project Explorer displays your project's file tree and provides tools for file and folder management.

6.3.1 File Tree Structure

The tree shows: - **Root node:** Your project name (folder name) - **game/ subdirectory:** Contains all `.rpy` files and asset folders - **images/ subdirectory (if exists):** Image assets - **audio/ subdirectory (if exists):** Audio assets - **Other files and folders:** Any other directories or files in your project

File Type Icons: - `.rpy` files: Script icon - `.py` files: Python icon - Folders: Folder icon (open or closed state) - Images: Thumbnail or image icon - Audio: Audio waveform icon - Other: Generic file icon

Expand/Collapse: - Click the arrow next to a folder to expand or collapse it - Double-click a folder to expand/collapse

6.3.2 Opening Files

To open a file in the code editor: - Double-click a `.rpy` or `.py` file in the tree - The file opens in the split pane editor at the bottom - If the editor pane is hidden, it appears automatically

To open an image or audio file: - Double-click to open in the respective editor view (Image Editor or Audio Editor) - For images: shows thumbnail, metadata, and Ren'Py name - For audio: shows playback controls and metadata

6.3.3 Creating Files and Folders

Right-click anywhere in the tree (or right-click a folder to create inside it):

New File: 1. Select "New File" from context menu 2. Enter the file name (e.g., `chapter1.rpy`) 3. Press Enter 4. The file is created and opens in the editor

New Folder: 1. Select "New Folder" from context menu 2. Enter the folder name (e.g., `chapters`) 3. Press Enter 4. The folder is created and appears in the tree

Tip: Ren'Py expects `.rpy` files to be in the `game/` directory or its subdirectories. Creating files outside `game/` is allowed but they won't be parsed by Ren'Py.

6.3.4 Renaming Files and Folders

To rename: 1. Right-click the file or folder 2. Select "Rename" from context menu 3. Edit the name in place 4. Press Enter to confirm or Escape to cancel

Important: Renaming a `.rpy` file does not automatically update jumps or calls that reference labels in that file. Use the Diagnostics panel to find broken jumps after renaming.

6.3.5 Deleting Files and Folders

To delete: 1. Right-click the file or folder 2. Select "Delete" from context menu 3. Confirm the deletion in the dialog that appears

Note: Deletion is permanent (files are removed from disk, not sent to trash/recycle bin). Always commit your project to version control before large deletions.

6.3.6 Cut, Copy, and Paste

Ren'IDE supports clipboard operations for files and folders:

Cut (Ctrl+X / Cmd+X): 1. Right-click a file or folder 2. Select "Cut" 3. The item is marked for moving (visually dimmed)

Copy (Ctrl+C / Cmd+C): 1. Right-click a file or folder 2. Select "Copy" 3. The item is marked for copying

Paste (Ctrl+V / Cmd+V): 1. Right-click the destination folder 2. Select "Paste" 3. The cut item is moved, or the copied item is duplicated

Tip: Cut and paste is useful for reorganizing your project structure. For example, move all character definition files into a `characters/` subfolder.

6.3.7 Drag-and-Drop Organization

You can also reorganize files by dragging:

1. Click and hold on a file or folder
2. Drag it to a new location in the tree
3. Drop it on a folder to move it inside that folder

Visual feedback: - A highlight or line indicates where the item will be dropped - If the drop location is invalid (e.g., trying to move a folder inside itself), the cursor changes to "not allowed"

6.3.8 Context Menu Actions

Right-click any file or folder to access these actions (depending on file type):

For .rpy files: - Open in Editor - Rename - Delete - Cut / Copy / Paste - **Center on Canvas:** Locates the corresponding block on the Story Canvas and zooms to it

For image files: - Open in Image Editor - Copy `scene` statement - Copy `show` statement - Rename / Delete / Cut / Copy / Paste

For audio files: - Open in Audio Editor - Copy `play music` statement - Copy `play sound` statement - Copy `queue audio` statement - Rename / Delete / Cut / Copy / Paste

For folders: - New File / New Folder - Rename / Delete - Cut / Copy / Paste - Expand All / Collapse All (for nested folders)

6.3.9 "Center on Canvas" Feature

One of the most useful features for navigation:

1. Right-click any `.rpy` file in the Project Explorer
2. Select **"Center on Canvas"**
3. The Story Canvas activates (if you're on a different canvas)
4. The canvas pans and zooms to center on the block representing that file
5. The block is highlighted briefly

This is invaluable for locating a specific file when you have many blocks on the canvas.

6.3.10 File Tree Search (if applicable)

Some versions of Ren'IDE include a search box at the top of the Project Explorer:

- Type a file name or partial name
- The tree filters to show only matching files
- Clear the search box to show all files again

If your version doesn't have this, use the main search panel (Ctrl+F) to search file contents.

6.4 3.4 Story Elements Panel (Right Sidebar)

The Story Elements panel is a multi-tab interface that displays project data extracted from your Ren'Py code. It uses a **two-level tab navigation** design: a primary row of section tabs at the top, with scrollable content below.

6.4.1 Tab Navigation

Primary tabs (always visible): - Characters - Variables - Images (Img) - Audio (Snd) - Screens - Composers - Menus - Snippets

Click a tab to switch to that section. The active tab is highlighted. Content scrolls vertically within the panel area.

6.4.2 Characters Tab

Displays all character definitions found in your project (defined with `define [tag] = Character(...)`).

Information shown: - **Character tag** (e.g., `e` for Eileen) - **Display name** (e.g., "Eileen") - **Color** (color swatch showing the character's dialogue color) - **Dialogue count** (number of lines spoken by this character)

Actions: - **Add New Character:** Click the "+ New Character" button to define a new character. A dialog opens where you specify the tag, name, color, and other Ren'Py parameters. - **Edit Character:** Click a character entry to open the Character Editor, where you can modify all properties. - **Find Usages:** Right-click a character and select "Find Usages" to see every location where this character speaks or is referenced. - **Jump to Definition:** Right-click and select "Jump to Definition" to open the file and line where the character is defined.

Unused Characters: Characters that are defined but never speak in your project are marked with a warning icon. These may be leftover from cut content or placeholders.

6.4.3 Variables Tab

Displays all global variables defined with `define` or `default`.

Information shown: - **Variable name** (e.g., `player_name`) - **Type** (`define` or `default`) - **Initial value** (e.g., `"Player"` or `0`)

Actions: - **Find Usages:** Right-click a variable to see everywhere it's read or written. - **Jump to Definition:** Navigate to the file and line where the variable is defined.

Note: Ren'IDE tracks globally defined variables only. Local variables (defined within labels) are not shown here.

6.4.4 Images Tab (Image Asset Manager)

Browses and manages all image assets in your project.

Organization: - Images are grouped by folder (e.g., `game/images/characters/`, `game/images/backgrounds/`) - Each image shows a thumbnail preview - Click a folder to expand/collapse its contents

Actions: - **Scan External Directory:** Click to add an external directory to the image list without copying files into your project. Useful for referencing shared asset libraries. - **Import Images:** Drag images from your OS file browser into the Images tab, or click "Import" to open a file picker. Images are copied to `game/images/` (or a subfolder you specify). - **Edit Metadata:** Double-click an image to open the Image Editor, where you can set: - **Ren'Py name** (e.g., `"eileen happy"` for `images/eileen_happy.png`) - **Tags** (custom tags for organization and filtering) - **Subfolder** (move the image to a subfolder) - **Copy Code:** Right-click an image and select: - **Copy `scene` statement** → `scene bg_office` is copied to clipboard - **Copy `show` statement** → `show eileen happy` is copied to clipboard - **Drag to Composer:** Drag an image thumbnail onto the Scene Composer stage or Screen Layout Composer to add it to a composition.

Thumbnails: Thumbnails are generated asynchronously. If you see a placeholder, wait a moment for the thumbnail to load.

Filtering: Some versions include a search/filter box at the top of the Images tab to quickly find images by name or tag.

6.4.5 Audio Tab (Audio Asset Manager)

Browses and manages all audio assets (music and sound effects).

Organization: - Audio files grouped by folder (e.g., `game/audio/music/`, `game/audio/sfx/`) - Each audio file shows: - File name - Format (MP3, OGG, WAV, etc.) - Duration (if available)

Actions: - **Scan External Directory:** Add external audio directories without copying files. - **Import Audio:** Drag audio files into the tab or click "Import." - **Built-in Player:** Click an audio file to load it in the built-in player. Controls include: - Play/pause - Volume slider - Seek bar - Loop toggle - **Edit Metadata:** Double-click to set Ren'Py name, tags, and subfolder. - **Copy Code:** Right-click and select: - **Copy `play music` statement** → `play music "audio/music/theme.mp3"` - **Copy `play sound` statement** → `play sound "audio/sfx/doorbell.wav"` - **Copy `queue audio` statement** → `queue audio "audio/music/theme.mp3"`

6.4.6 Screens Tab

Lists all `screen` definitions found in your project (typically in `screens.rpy` or custom screen files).

Information shown: - **Screen name** (e.g., `say`, `choice`, `main_menu`) - **File location** (which `.rpy` file defines it) - **Parameters** (if the screen takes arguments)

Actions: - **Jump to Definition:** Click a screen to open the file and scroll to the screen definition. - **Add Screen Boilerplate:** Click "+ New Screen" to create a new screen with basic boilerplate code. A dialog prompts for the screen name.

Locked/Editable Indicator: Screens can be viewed in the Screen Layout Composer in read-only mode. To edit a screen composition, duplicate it first.

6.4.7 Composers Tab

Manages saved compositions from the three visual composers: Scene Composer, ImageMap Composer, and Screen Layout Composer.

Saved Compositions: Each composition entry shows: - **Name** (e.g., "Office Scene", "Main Menu") - **Type** (Scene, ImageMap, or Screen) - **Thumbnail** (preview image)

Actions: - **Load Composition:** Click a composition to open it in the respective composer. - **Delete Composition:** Right-click and select "Delete" to remove it from the library. - **Create New Composition:** Click "+ New Scene", "+ New ImageMap", or "+ New Screen" to start a fresh composition.

Persistence: Compositions are saved in `project.ide.json`. Commit this file to version control if you want teammates to have access to your compositions.

6.4.8 Menus Tab (Visual Menu Builder)

The Menus tab provides a visual interface for designing Ren'Py menu structures.

Features: - **See all menus in your project:** Extracted from your code - **Create new menus:** Click "+ New Menu" to design a menu structure - **Configure choices:** - Choice text (what the player sees) - Destination (jump or call to a label) - Condition (optional `if` guard) - Custom code blocks (arbitrary code to run when choice is selected)

Workflow: 1. Click "+ New Menu" or select an existing menu 2. Use the visual designer to add choices 3. Configure each choice's properties in the side panel 4. Click "Generate Code" to copy the Ren'Py menu code to clipboard 5. Paste into your script

Integration with Choice Canvas: Menus designed here are also visualized on the Choice Canvas as menu nodes with choice pills.

6.4.9 Snippets Tab

The Snippets tab is your code snippet library.

Built-in Snippets: Ren'IDE includes 28+ built-in Ren'Py snippets for common patterns: - `label` — create a new label - `jump` — jump to a label - `call` — call a label - `menu` — create a menu with choices - `show` — show an image - `scene` — display a scene - `play music`, `play sound` — audio playback - And many more

User-Defined Snippets: Create custom snippets for project-specific boilerplate:

1. Click "+ New Snippet"
2. Enter a **trigger prefix** (e.g., `mytemplate`)
3. Enter the snippet **content** (use `$1`, `$2`, etc. for tab-stop placeholders)
4. Save

When you type the trigger prefix in the code editor and press Tab, the snippet expands.

Example custom snippet:

Trigger: `mychar`

Content:

```
define $1 = Character("$2", color="$3")
$0
```

Typing `mychar` + Tab in the editor inserts the template with placeholders for the character tag, name, and color.

Snippet Management: - Edit snippets by clicking them - Delete snippets with the trash icon - Export/import snippet libraries (if supported)

6.5 3.5 Split Pane Editor (Bottom)

The split pane editor is a flexible code editing area powered by the **Monaco Editor** (the same engine as Visual Studio Code).

6.5.1 Opening Files

Files open in the editor when you: - Double-click a block on the canvas - Double-click a file in the Project Explorer - Click a diagnostic in the Diagnostics panel (opens at the specific line) - Use "Jump to Definition" from Story Elements

6.5.2 Editor Panes

The editor supports **split pane editing**:

Single Pane (default): - One file visible at a time - Tabs at the top show open files - Click a tab to switch files

Side-by-Side Panes: - Two files visible horizontally - Drag a tab from one pane to the other to open it side by side - Useful for comparing files or editing two related files simultaneously

Bottom Split (not yet implemented in all versions): - Two files visible vertically (one above the other) - Drag a tab to the bottom edge to create a bottom split

Resizing Panes: - Drag the divider (sash) between panes to resize - Double-click the sash to reset to 50/50 split

Closing Panes: - Close all tabs in a pane to hide that pane - Or click the X icon to close the pane (if present)

6.5.3 Tab Management

Tab Bar: - Shows all open files as tabs - Active tab is highlighted - Modified files show an indicator (dot or asterisk)

Closing Tabs: - Click the X icon on a tab - Middle-click a tab (if supported) - Right-click and select "Close"

Tab Context Menu: Right-click a tab for options: - Close - Close Others - Close All - Close Saved (close tabs without unsaved changes) - Reveal in File Explorer (highlight in Project Explorer)

Dragging Tabs: - Drag a tab within the same pane to reorder - Drag a tab to another pane to move it - Drag a tab outside the editor area to undock (if supported)

6.5.4 Monaco Editor Features

Syntax Highlighting: - Ren'Py code is highlighted with a custom TextMate grammar - Labels, variables, character tags, and screen references are semantically colored

Line Numbers: - Displayed on the left - Click a line number to set the cursor to that line

IntelliSense (Autocomplete): - Press `Ctrl+Space` to manually trigger suggestions - Autocomplete appears automatically as you type: - After `jump` or `call`: suggests label names - After `show` or `scene`: suggests image names - For character tags in dialogue - For variable names

Snippets: - Type a snippet trigger prefix (e.g., `label`, `menu`) and press Tab - Tab-stop placeholders let you jump between fields with Tab

Code Folding: - Fold/unfold blocks of code (labels, menus, etc.) - Click the arrow icon in the gutter next to line numbers

Find and Replace: - **Find:** Press `Ctrl+F` to open the find bar - **Replace:** Press `Ctrl+H` to open the find/replace bar - Supports regex and case-sensitive matching

Multi-Cursor Editing: - Hold `Ctrl` (Windows/Linux) or `Cmd` (macOS) and click multiple locations to create multiple cursors - Hold `Alt` and drag to create a column selection

Cursor Position Display: - Current line and column are shown in the status bar (e.g., `Ln 42, Col 15`)

6.5.5 Saving Changes

- Press `Ctrl+S` (or use the toolbar "Save All" button) to save all open files
- Modified files show an indicator on their tab
- Unsaved changes are also indicated in the status bar

6.6 3.6 Status Bar

The status bar at the bottom of the window displays contextual information and application status.

6.6.1 Left Side

Current File Name: - Shows the name of the file open in the active editor pane - Example: `script.rpy`

Cursor Position: - Format: `Ln [line], Col [column]` - Example: `Ln 42, Col 15` means the cursor is on line 42, column 15

6.6.2 Center

Project Analysis Status: - Shows messages like: - "Analyzing project…" (with spinner) during initial load - "Project analysis complete" after loading - "Saving…" during save operations - "Unsaved changes" if there are modified files

6.6.3 Right Side

Application Version: - Example: `v0.7.1` - If a build number is present: `v0.7.1 (Build 1234)` - Always visible so you know which version you're running

Additional Indicators (if present): - SDK connection status (green = connected, red = not configured) - Drafting mode indicator (icon if enabled) - Network status (for auto-updater)

6.7 3.7 Keyboard Shortcuts

Keyboard shortcuts make navigation and editing faster. Here are the most important shortcuts to know.

6.7.1 Global Shortcuts

Shortcut	Action
`Ctrl+S` / `Cmd+S`	Save All
`Ctrl+Z` / `Cmd+Z`	Undo
`Ctrl+Y` / `Cmd+Y` / `Ctrl+Shift+Z`	Redo
`Ctrl+G` / `Cmd+G`	Go to Label (command palette)
`Escape`	Deselect all / Close modal

6.7.2 Canvas Shortcuts

Shortcut	Action
`N`	New Block
`G`	Group selected blocks
`Delete`	Delete selected blocks
`Shift+Drag`	Pan canvas (configurable in Settings)
`Mouse Scroll`	Zoom canvas
`Ctrl+Click` / `Cmd+Click`	Multi-select blocks
`Drag`	Rubber-band selection (when not on a block)

6.7.3 Editor Shortcuts

Shortcut	Action
`Ctrl+F` / `Cmd+F`	Find in file
`Ctrl+H` / `Cmd+H`	Replace in file
`Ctrl+Space`	Trigger IntelliSense manually
`Ctrl+/` / `Cmd+/`	Toggle line comment
`Ctrl+D` / `Cmd+D`	Add selection to next find match (multi-cursor)
`Alt+Up/Down`	Move line up/down
`Ctrl+Shift+K` / `Cmd+Shift+K`	Delete line

6.7.4 File Explorer Shortcuts

Shortcut	Action
`Ctrl+C` / `Cmd+C`	Copy file
`Ctrl+X` / `Cmd+X`	Cut file
`Ctrl+V` / `Cmd+V`	Paste file
`Delete`	Delete file/folder
`F2`	Rename file/folder

6.7.5 Customizing Shortcuts

Some versions of Ren'IDE allow shortcut customization:

1. Open Settings → Keyboard Shortcuts
2. Click the shortcut you want to change
3. Press the new key combination
4. Save settings

6.8 3.8 Themes

Ren'IDE offers 10 themes to personalize your workspace.

6.8.1 Available Themes

Light Themes: - **System Light:** Follows your OS theme (light variant) - **Colorful Light:** Vibrant colors on a light background - **Candy Light:** Soft pastels on light background - **Forest Light:** Nature-inspired greens and browns on light - **Solarized Light:** Classic Solarized light palette

Dark Themes: - **System Dark:** Follows your OS theme (dark variant) - **Colorful Dark:** Vibrant colors on a dark background - **Neon Dark:** Bright neon colors on dark background - **Ocean Dark:** Blue and teal tones on dark background - **Solarized Dark:** Classic Solarized dark palette

6.8.2 Changing Themes

1. Click the Settings button in the toolbar (gear icon)
2. In the General section, find **"Theme"**
3. Open the dropdown and select a theme
4. The interface updates immediately
5. Close Settings (theme is saved automatically)

6.8.3 Theme Persistence

Themes are saved per project in `project.ide.json`. If you switch projects, each project remembers its theme.

To set a global default theme: - Some versions allow setting a default in global settings (affects new projects) - Check Settings → General → Default Theme

6.8.4 Dark Mode vs. Light Mode

Why choose dark mode: - Reduces eye strain in low-light environments - Saves battery on OLED/AMOLED displays - Preferred by many developers

Why choose light mode: - Better readability in bright environments - Preferred for printouts or screenshots for documentation - Less contrast can be easier on the eyes for some users

Tip: Try different themes to find the one that's most comfortable for long editing sessions.

End of Section 3: Interface Overview

Chapter 7

Section 4: Core Features Tour

This section provides an in-depth tour of Ren'IDE's most powerful features: the three canvas views, the integrated code editor, project-wide search, diagnostics, statistics, and the undo/redo system.

7.1 4.1 The Three Canvas System

Ren'IDE's signature feature is its three-canvas visualization system. Each canvas shows your project from a different perspective, helping you understand narrative structure at multiple levels of abstraction.

7.1.1 4.1.1 Story Canvas - File-Level View

The Story Canvas is the default view when you open a project. It represents your project at the **file level**: each `.rpy` file is a draggable block on the canvas, and arrows show jumps and calls between files.

7.1.1.1 What the Story Canvas Shows

Blocks: - Each block represents one `.rpy` file (e.g., `script.rpy`, `chapter1.rpy`, `screens.rpy`) - Block size reflects the number of lines in the file (larger blocks = more code) - Block color can indicate: - **Role tinting:** Blocks are tinted based on which characters appear in them - **File type:** Story blocks vs. screen blocks vs. config blocks

Arrows: - **Solid arrows:** Represent `jump` statements (transfer control to another file) - **Dashed arrows:** Represent `call` statements (call a label and return) - Arrows are color-coded (consistent with the legend overlay) - Arrow thickness may reflect the number of connections

Diagnostic Glow: - Blocks with errors show a **red outer glow** - Blocks with warnings show an **amber outer glow** - This makes problem areas visible even when zoomed out

Block Labels: - Each block displays its file name - Hover to see additional info (number of labels, characters, etc.)

7.1.1.2 Interacting with Blocks

Selecting Blocks: - Click a block to select it (highlights with a border) - Multi-select: Hold `Ctrl` (Windows/Linux) or `Cmd` (macOS) and click multiple blocks - Rubber-band selection: Click and drag on empty canvas to draw a selection box around multiple blocks

Moving Blocks: - Click and drag a block to reposition it - Selected blocks move together - Block positions are saved in `project.ide.json`

Opening Files: - Double-click a block to open its file in the code editor - The editor pane appears (if hidden) and the file loads with syntax highlighting

Block Context Menu: Right-click a block for options: - Open in Editor - Find Label in Block (if block contains multiple labels) - Delete Block (moves the file to trash/recycle bin) - Group with Selected (create a visual group container) - Set Color (override default block color)

7.1.1.3 Canvas Navigation

Panning: - Hold `Shift` and drag to pan the canvas (configurable in Settings) - Alternatively, drag with the middle mouse button (if supported) - Touch devices: Two-finger drag

Zooming: - Scroll the mouse wheel to zoom in/out - Pinch to zoom on touch devices - Zoom focuses on the cursor position

Minimap: - A small overview of the entire canvas appears in the corner (if enabled) - Shows all blocks as small rectangles - Current viewport is indicated by a rectangle - Click anywhere on the minimap to jump to that area

Fit-to-Screen: - Click the "Fit" button in the canvas controls (bottom-right) - Automatically zooms and pans so all blocks are visible

Go-to-Start: - Click the "Start" button in the canvas controls - Centers on the block containing the `start` label and zooms in

7.1.1.4 Go-to-Label Features

Ctrl+G Command Palette: 1. Press `Ctrl+G` (or `Cmd+G` on Mac) from any canvas 2. A command palette appears at the top of the screen 3. Type a label name (supports fuzzy search): - **Exact matches** rank first - **Prefix matches** (e.g., typing "sta" matches "start") rank second - **Substring matches** (e.g., "art" matches "start") rank third - **Fuzzy matches** (e.g., "srt" matches "start") rank fourth 4. Press `Enter` or click a result to navigate 5. The canvas zooms in to at least scale 1.0 and centers on the target label

Toolbox Label Search: - A "Go to Label" search box appears in the Story Canvas toolbox (top-left) - Type to filter labels - Click a result to navigate - Same zoom-on-navigate behavior as Ctrl+G

7.1.1.5 Filtering and Grouping

Character Filter: - In the toolbox, select a character from the dropdown - Blocks that don't involve that character are dimmed or hidden - Useful for tracing a single character's arc through the story

Role Tinting: - When enabled, blocks are tinted based on which characters appear in them - Example: Blocks with Eileen are tinted blue, blocks with Lucy are tinted pink - Helps visualize character distribution across the project

Grouping Modes: - **None:** No grouping (default) - **Connected Components:** Groups blocks that are connected by jumps/calls - **File Prefix:** Groups blocks by common file name prefix (e.g., `chapter1_*.rpy`)

To apply grouping: 1. Open the toolbox dropdown for "Grouping" 2. Select a grouping mode 3. The layout mode auto-switches to **Clustered Flow** (the only mode that renders grouping) 4. Blocks are visually grouped in colored containers

Layout Modes: - **Flow:** Hierarchical layout based on jumps/calls (default) - **Hierarchical:** Tree-like layout with root blocks at the top - **Circular:** Blocks arranged in a circle - **Clustered Flow:** Flow layout with grouping containers

Change layout in the toolbox dropdown.

7.1.1.6 Auto-Layout (Redraw)

Click the **Redraw** button in the toolbar to trigger automatic layout: - Analyzes jump/call connections - Arranges blocks hierarchically (root blocks at the top, leaf blocks at the bottom) - Minimizes arrow crossings - Spreads blocks evenly

Warning: Redraw overwrites manual block positioning. If you've carefully arranged blocks, this will reset them. Use with caution or save your layout first.

7.1.1.7 Canvas Legend

Click the **Legend** button (if available) in the toolbox to show an overlay explaining: - Arrow types (solid = jump, dashed = call) - Block colors (role tinting, file type) - Diagnostic indicators (red glow = error, amber = warning)

The legend is a floating panel that can be moved or dismissed.

7.1.1.8 Tips for Using Story Canvas

- **Use grouping for large projects:** If you have 50+ files, grouping by connected components helps identify independent storylines.
- **Zoom out to see the big picture:** The diagnostic glow means you can identify problem areas even when blocks are too small to read.
- **Use the minimap for navigation:** Faster than panning manually when blocks are far apart.
- **Combine with Route Canvas:** Story Canvas shows file-level structure; Route Canvas shows label-level detail. Use both for complete understanding.

7.1.2 4.1.2 Route Canvas - Label-Level View

The Route Canvas visualizes your project at the **label level**: every `label` becomes a node, and every `jump`, `call`, or implicit fall-through becomes an edge. This is a control flow graph that shows how the player moves through your story.

7.1.2.1 What the Route Canvas Shows

Nodes (Labels): - Each node represents a label (e.g., `start`, `chapter1`, `ending_happy`) - Node size may reflect the label's importance (e.g., `start` is larger) - Node color indicates: - **Role:** Start node (green), end node (red), choice node (blue), etc. - **Route membership:** Nodes on the same route share a color

Edges (Jumps/Calls): - **Solid arrows:** `jump` statements - **Dashed arrows:** `call` statements - **Dotted lines:** Implicit fall-through (control passes from one label to the next without an explicit jump) - Arrow colors match the source node's route color

Unreachable Labels: - Labels that no path from `start` leads to are highlighted in red or marked with a warning icon - These are "dead code"—players will never see them

Menu Nodes: - Labels that contain `menu` statements are specially marked (e.g., with a branching icon) - Hover over a menu node to see a popover with: - All choice texts - Destinations (where each choice leads) - Conditional guards (`if` statements)

7.1.2.2 Graph Layout

The Route Canvas uses a graph layout algorithm to position nodes:

BFS Layered Layout (default): - Nodes are arranged in horizontal layers - `start` is at the leftmost layer - Each layer contains nodes at a certain "distance" from `start` - Edges generally flow left-to-right

Handling Cycles: - If your story has loops (e.g., a repeating scene), the algorithm detects cycles - Back-edges (edges that point "backward" to earlier layers) are drawn with a distinct style - The layout tries to minimize back-edge crossings

Force-Directed Layout (if available): - Nodes repel each other; edges act like springs - Produces organic-looking layouts - Good for projects with lots of cyclic structures

7.1.2.3 Route Highlighting

The Route Canvas supports **route highlighting**: tracing specific narrative paths through the story.

What is a Route? A route is a complete path from the `start` label to an ending label (a label with no outgoing edges or a `return` statement).

Highlighting a Route: 1. In the Route List panel (left sidebar, collapsible), you'll see a list of discovered routes 2. Each route has: - A descriptive name (e.g., "Start → Chapter 1 → Good Ending") - A color swatch 3. Click a route to highlight it on the canvas: - Nodes on the route are highlighted in the route's color - Nodes NOT on the route are dimmed (lower opacity) - Edges on the route are highlighted

Multiple Route Highlighting: - Select multiple routes (hold `Ctrl` and click) to highlight them simultaneously - Each route gets a distinct color - Overlapping nodes (nodes that appear in multiple routes) show multiple colors (striped or blended)

Route List Panel: - Collapsible sidebar on the left (separate from Project Explorer) - Shows all discovered routes - Hover over a route entry to expand it and see the full list of labels in that route - Right-click a route for options: - Rename Route - Change Color - Delete Route (removes from list, doesn't affect code)

7.1.2.4 Go-to-Label on Route Canvas

The Route Canvas also supports: - **Ctrl+G command palette** (same as Story Canvas) - **Toolbox label search** (filter and jump to labels) - Both zoom-on-navigate to at least scale 1.0

7.1.2.5 Menu Decision Inspector

When you hover over a menu node: 1. A popover appears 2. Shows the menu prompt (if any) and all choices: - Choice text (what the player sees) - Destination label (where the choice leads) - Condition (if the choice is conditional) 3. Click a choice in the popover to navigate to its destination label

This is invaluable for understanding complex branching without opening the code.

7.1.2.6 Node Roles

Each node is classified by role: - **Start:** The `start` label (usually green) - **End:** Labels with no outgoing edges or `return` statements (usually red) - **Choice:** Labels that contain `menu` statements (usually blue) - **Decision:** Labels with conditional jumps (e.g., `if/else` branches) - **Story:** Regular labels (neutral color)

Roles are indicated by color and/or icon.

7.1.2.7 Tips for Using Route Canvas

- **Use route highlighting to verify story paths:** Highlight the "good ending" route to make sure all choices leading to it are correct.
- **Identify unreachable labels early:** Red/highlighted unreachable labels indicate dead code that needs attention.
- **Hover over menu nodes instead of opening files:** Save time by inspecting choices directly on the canvas.
- **Combine with Choice Canvas:** Route Canvas shows code structure; Choice Canvas shows player experience.

7.1.3 4.1.3 Choice Canvas - Player Experience View

The Choice Canvas shows your project from the **player's perspective**: what choices will they see, what text appears, where do choices lead, and what conditions guard them?

7.1.3.1 What the Choice Canvas Shows

Menu Nodes: - Each node represents a menu the player will encounter - The node displays the menu prompt (if any)

Choice Pills: - Branching from each menu node are **choice pills**: colored, labeled rectangles representing each choice - Each pill shows: - **Choice text** (what the player reads) - **Conditional badge** (if the choice is guarded by an `if` statement) - Pills are color-coded to indicate their destination

Destination Labels: - Each pill connects to a destination label (where the choice leads) - Following a choice shows you where the player ends up

Color Coding: - Choices leading to the same destination share a color - This makes it easy to see when multiple paths converge

Conditional Badges: - Choices with conditions display a badge like `if player_name == "Alice"` - Helps you understand when choices are available vs. hidden

7.1.3.2 Tracing Player Paths

The Choice Canvas is designed for **player experience testing**:

1. Start at the first menu node
2. Read the choice texts
3. Follow a choice pill to its destination
4. Continue following subsequent choices
5. Trace the path all the way to an ending

This lets you "play" your game visually without launching Ren'Py.

7.1.3.3 Understanding Branching Complexity

The Choice Canvas makes branching structure obvious: - **Linear sections:** Few or no menu nodes (straight line through the story) - **Dense branching:** Many menu nodes with many choices (complex decision trees) - **Convergent paths:** Multiple choices leading back to a common label (paths merging)

Visualizing this helps you balance player agency with manageable development complexity.

7.1.3.4 Go-to-Label on Choice Canvas

The Choice Canvas supports: - **Ctrl+G command palette** for instant label navigation - **Toolbox label search** to filter and jump - Zoom-on-navigate to ensure the target is visible

7.1.3.5 Comparing Canvases

Feature	Story Canvas	Route Canvas	Choice Canvas
Granularity	File-level	Label-level	Menu-level
Nodes	.rpy files	Labels	Menus
Arrows	Jumps/calls between files	Jumps/calls between labels	Choice → destination
Best For	Project structure	Control flow	Player experience
Use When	Organizing files, seeing big picture	Debugging logic, tracing paths	Testing choices, narrative design

Tip: Use all three canvases together. Switch between them with the canvas tabs in the toolbar.

7.1.4 4.1.4 Canvas Navigation (All Canvases)

These navigation features work on **all three canvases**:

Go-to-Label Command Palette (Ctrl+G / Cmd+G): - Press the shortcut from any canvas - Type a label name - Press Enter to navigate - Canvas zooms in to at least scale 1.0

Toolbox Label Search: - Each canvas has a "Go to Label" search box in the toolbox - Type to filter labels - Click a result to navigate

Fit-to-Screen: - Button in canvas controls (bottom-right) - Zooms and pans to fit all nodes/blocks within the viewport

Go-to-Start: - Button in canvas controls - Centers on the `start` label and zooms in

Auto-Center on First Open: - When you first open a project, each canvas auto-centers on `start` - This behavior is persisted (won't repeat when you reopen the project)

Zoom-on-Navigate: - All "go to" actions snap zoom to at least 1.0× if currently zoomed out - Prevents navigating to a label that's too small to see

---

7.2 4.2 Code Editor

Ren'IDE includes a full-featured code editor powered by **Monaco** (the same engine as Visual Studio Code). This section covers editor features beyond what was mentioned in Section 3.5.

7.2.1 4.2.1 Monaco Editor Basics

Opening Files: - Double-click a block on the canvas - Double-click a file in the Project Explorer - Click a diagnostic to jump to the line with the issue

Multi-File Editing: - Open multiple files as tabs - Drag tabs to split panes for side-by-side editing

Saving: - Press `Ctrl+S` to save all open files - Modified files show an indicator on their tab

7.2.2 4.2.2 Syntax Highlighting

Ren'IDE uses a **TextMate grammar** specifically designed for Ren'Py:

Highlighted Elements: - **Keywords:** `label`, `jump`, `call`, `menu`, `if`, `else`, `define`, `default`, `return`, `scene`, `show`, `play`, etc. - **Strings:** Dialogue text in quotes - **Comments:** Lines starting with `#` - **Variables:** Recognized variable names (from analysis) - **Labels:** Label names at definition sites - **Character tags:** In dialogue lines - **Screen names:** In `screen` definitions and `call screen` statements

Semantic Token Provider: In addition to syntax highlighting, Ren'IDE adds **semantic coloring**: - Labels in scope are colored distinctly - Variables in scope are highlighted - Screen references use a special color - This makes it easier to distinguish different types of identifiers at a glance

7.2.3 4.2.3 IntelliSense & Autocomplete

IntelliSense provides context-aware suggestions as you type:

Jump/Call Targets: - Type `jump` and a space - IntelliSense shows a list of all labels in your project - Select one with arrow keys and press Enter (or click)

Show/Scene Images: - Type `show` or `scene` and a space - IntelliSense suggests image names from your assets

Character Tags: - When writing dialogue, start typing a character tag - IntelliSense suggests defined characters

Screen Names: - Type `call screen` or `show screen` and a space - IntelliSense suggests screen names

Variables: - Type a variable name - IntelliSense suggests globally defined variables

Manual Triggering: - Press `Ctrl+Space` at any time to manually trigger IntelliSense

Accepting Suggestions: - Press `Enter` or `Tab` to accept the selected suggestion - Press `Escape` to dismiss the suggestion list

7.2.4 4.2.4 Snippets

Snippets are templates for common Ren'Py patterns:

Built-in Snippets (28+): - `label` → Creates a new label with placeholder - `jump` → Jump statement - `call` → Call statement - `menu` → Menu with two choices - `choice` → Single menu choice - `show` → Show statement - `scene` → Scene statement - `play music` → Play music statement - `if` → If/else block - `define` → Define statement - `default` → Default statement - And many more…

Using Snippets: 1. Type the snippet trigger prefix (e.g., `label`) 2. Press `Tab` to expand 3. The snippet inserts with placeholders highlighted 4. Type to replace the first placeholder 5. Press `Tab` to jump to the next placeholder 6. Press `Tab` until all placeholders are filled

Example: Typing `label` + Tab inserts:

```
label ${1:label_name}:
    ${2:# your content here}
    return
```

The cursor is on `label_name`. Type the actual label name, press Tab, and the cursor moves to the content line.

User-Defined Snippets: Create custom snippets in the Snippets tab (see Section 3.4.8).

7.2.5 4.2.5 Advanced Editor Features

Code Folding: - Click the arrow icons in the gutter (left of line numbers) to fold/unfold blocks - Useful for collapsing large labels or menus to focus on specific sections

Find and Replace: - **Find:** Press `Ctrl+F` to open the find bar at the top of the editor - Enter search text - Use regex by clicking the `.*` icon - Toggle case-sensitive with the `Aa` icon - Navigate matches with up/down arrows - **Replace:** Press `Ctrl+H` to open find/replace - Enter replacement text - Click "Replace" to replace current match - Click "Replace All" to replace all matches in the file

Multi-Cursor Editing: - Hold `Ctrl` (Windows/Linux) or `Cmd` (macOS) and click multiple locations to create multiple cursors - Type to edit all locations simultaneously - Useful for renaming multiple instances of a variable in one go

Column Selection: - Hold `Alt` and drag to create a column selection (rectangular selection) - Useful for editing tabular data or aligning code

Move Line Up/Down: - Press `Alt+Up` to move the current line (or selected lines) up - Press `Alt+Down` to move them down

Delete Line: - Press `Ctrl+Shift+K` to delete the current line

Toggle Line Comment: - Press `Ctrl+/` to comment or uncomment the current line (adds or removes `#`)

Bracket Matching: - Place the cursor next to a bracket, parenthesis, or brace - The matching pair is highlighted

Auto-Indentation: - Press `Enter` after a line ending with `:` (e.g., after a label or if statement) - The next line is automatically indented

---

7.3 4.3 Project-Wide Search & Replace

Ren'IDE includes a powerful full-text search across your entire project.

7.3.1 Opening the Search Panel

- Press `Ctrl+Shift+F` (Windows/Linux) or `Cmd+Shift+F` (macOS), or
- Click the Search icon in the left sidebar (magnifying glass)

The Search Panel replaces the Project Explorer in the left sidebar.

7.3.2 Performing a Search

Search Box: - Enter your search query (text or regex)

Options: - **Match Case:** Toggle to make the search case-sensitive - **Whole Word:** Toggle to match whole words only - **Regex:** Toggle to use regular expressions

Click "Search" or press Enter.

7.3.3 Search Results

Results are displayed as a list: - **File name** (grouped) - **Line number** and **matched text** (highlighted)

Example:

```
script.rpy
  12: label start:
  34: jump chapter1
chapter1.rpy
  5: label chapter1:
```

Clicking a result: - Opens the file in the editor - Scrolls to the line with the match - Highlights the matched text

7.3.4 Replace

Replace Box: - Enter replacement text

Replace Options: - **Replace:** Replaces the currently selected match (in the editor) - **Replace All in File:** Replaces all matches in the current file - **Replace All:** Replaces all matches in all files

Bulk Replace with Confirmation: If you click "Replace All," Ren'IDE may prompt: - Show a confirmation dialog listing all files that will be modified - You can review the list and confirm or cancel

Regex Replace: If regex mode is enabled, you can use capture groups in the replacement: - Search: `label (\w+):` - Replace: `label new_$1:` - This renames all labels by prepending "new_"

7.3.5 Search History

The search panel remembers recent queries. Click the dropdown in the search box to see and reuse previous searches.

7.3.6 Tips for Search & Replace

- **Use regex for powerful searches:** Find all labels: `label \w+:`

- **Preview before replacing all:** Always review the search results before clicking "Replace All"
- **Use version control:** Commit your project before large replace operations so you can revert if needed

---

7.4 4.4 Diagnostics Panel

The Diagnostics panel is your quality assurance tool, continuously analyzing your project for issues.

7.4.1 Opening Diagnostics

- Click the **Diagnostics** button in the toolbar (exclamation mark icon), or
- Press `Ctrl+D` (if shortcut is configured)

The Diagnostics panel opens as a modal or bottom panel.

7.4.2 Diagnostic Categories

Errors (Red): - **Invalid Jumps:** `jump` or `call` to a label that doesn't exist - **Syntax Errors:** Code that Ren'Py can't parse - **Missing Assets:** References to images or audio files not found in the project

Warnings (Amber): - **Undefined Characters:** Character tags used in dialogue but never defined with `define` - **Undefined Screens:** Screens called or shown but never defined - **Unused Characters:** Characters defined but never used in dialogue - **Unreachable Labels:** Labels no path from `start` leads to

Info (Blue): - **Suggestions:** Best practices, code style recommendations - **Usage Notes:** Informational messages about your project

7.4.3 Diagnostic Details

Each diagnostic entry shows: - **Severity icon** (red X, amber warning, blue info) - **Description** (e.g., "Jump to undefined label 'chapter2' ") - **File path** (e.g., `script.rpy`) - **Line number** (e.g., `Line 42`)

7.4.4 Jumping to Source

Click any diagnostic to: - Open the file in the editor - Scroll to the line with the issue - Highlight the problematic code

This makes fixing errors fast and easy.

7.4.5 Filtering Diagnostics

Severity Filter: - Toggle checkboxes at the top of the panel: - **Errors** (show/hide) - **Warnings** (show/hide) - **Info** (show/hide)

File Filter: - If available, filter diagnostics by file name

7.4.6 Converting Diagnostics to Tasks

Some diagnostics can be converted to tasks: - Right-click a diagnostic - Select "Convert to Task" - The issue is added to a task list (tracked with your project) - Mark tasks as complete as you fix them

Task List Integration: Tasks appear in the Diagnostics panel under a "Tasks" tab. Track your progress as you fix issues.

7.4.7 Diagnostic Badges

Toolbar Badge: The Diagnostics button in the toolbar shows a badge with the total count of errors and warnings (e.g., "3" for 3 total issues). This gives you an at-a-glance indication of project health.

Canvas Block Glow: Blocks on the Story Canvas that contain diagnostics display a colored glow (red for errors, amber for warnings). This makes it easy to spot problem areas even when zoomed out.

7.4.8 Tips for Using Diagnostics

- **Check diagnostics regularly:** Make it a habit to open the Diagnostics panel after making changes
- **Fix errors first, then warnings:** Errors will cause runtime failures; warnings are less critical
- **Use "Jump to Source" instead of manually finding issues:** Saves time and reduces frustration
- **Convert persistent issues to tasks:** If an issue can't be fixed immediately, add it to the task list as a reminder

---

7.5 4.5 Project Statistics

The Project Statistics panel provides analytical insights into your visual novel project.

7.5.1 Opening Stats

- Click the **Stats** button in the toolbar (bar chart icon)

The Stats panel opens as a modal or side panel.

7.5.2 Deferred Loading

For large projects, statistics computation can be expensive. Ren'IDE uses **deferred loading**: - The Stats panel opens immediately - Each metric shows an inline spinner while it calculates - Metrics become visible as they complete (usually within a few seconds)

This prevents the app from freezing when you open Stats.

7.5.3 Metrics Displayed

Word Count: - Total word count across all dialogue and narration - Excludes code (labels, jumps, etc.)

Estimated Play Time: - Based on average reading speed (e.g., 200 words per minute) - Formula: `total_words / reading_speed` - Shown in minutes or hours

Lines of Dialogue: - Total number of dialogue lines spoken by characters - Excludes narration

Per-Character Dialogue Breakdown: - Bar chart showing dialogue count for each character - Sorted by count (character with most lines at the top) - Hover over a bar to see exact count - Useful for identifying characters who need more/less screen time

Scene Count: - Number of distinct scenes (files or labels, depending on how scenes are organized)

Label Count: - Total number of labels in the project

Route Count: - Number of discovered narrative routes (paths from `start` to an ending)

Branching Complexity Scores: - Metrics like: - Average choices per menu - Maximum path depth (longest route) - Cyclomatic complexity (measure of decision points) - Helps you understand how complex your branching structure is

Asset Coverage: - **Images:** Number of images defined vs. number used in code - **Audio:** Number of audio files defined vs. number used - Identifies unused assets (clutter) and missing assets (errors)

7.5.4 Interpreting Stats

High word count but short play time estimate: - May indicate fast-paced dialogue or minimal narration

Uneven character dialogue breakdown: - Some characters dominate; others are underutilized - Consider whether this is intentional

High branching complexity: - Your story has many choices and paths - Be aware of testing burden (you need to test all paths)

Low asset coverage: - Many defined assets aren't used (consider cleaning up) - Or many used assets aren't defined (errors—check Diagnostics)

7.5.5 Exporting Stats

Some versions of Ren'IDE allow exporting stats as a report: - Click "Export" or "Save Report" - Choose format (CSV, JSON, PDF) - Save to disk

Use this for tracking progress over time or sharing with teammates.

7.6 4.6 Undo/Redo System

Ren'IDE maintains a history of your actions so you can undo mistakes and redo changes.

7.6.1 What Can Be Undone

The undo/redo system tracks: - **Canvas block moves:** Dragging blocks to new positions - **Block creation/deletion:** Adding or removing blocks - **Composition edits:** Changes in Scene Composer, ImageMap Composer, Screen Layout Composer - **Code changes:** Typing in the editor (grouped by typing session)

7.6.2 Using Undo

- Press `Ctrl+Z` (Windows/Linux) or `Cmd+Z` (macOS), or
- Click the **Undo** button in the toolbar

The most recent action is reverted.

Press Undo multiple times to step backward through history.

7.6.3 Using Redo

- Press `Ctrl+Y` (Windows/Linux) or `Cmd+Y` / `Cmd+Shift+Z` (macOS), or
- Click the **Redo** button in the toolbar

Re-applies an action that was undone.

Redo is only available after Undo has been used. If you make a new change after Undo, the redo history is cleared.

7.6.4 History Depth

Ren'IDE maintains a history of the last 50 actions (configurable in Settings).

When the history limit is reached, the oldest action is removed.

7.6.5 Visual Feedback

- **Undo/Redo buttons:** Grayed out when unavailable (nothing to undo/redo)
- **Status bar:** May show "Undo applied" or "Redo applied" briefly after use

7.6.6 Limitations

Undo/Redo does NOT track: - File system operations (rename, delete files outside the canvas) - Settings changes - Asset imports

These actions are permanent and cannot be undone through the undo system. Always commit to version control before large changes!

7.6.7 Tips for Using Undo/Redo

- **Undo is your safety net:** Don't be afraid to experiment—you can always undo
- **Use version control for file operations:** Undo won't save you from accidental file deletions
- **Check the toolbar buttons:** If Undo is grayed out, there's nothing to undo

---

End of Section 4: Core Features Tour

Chapter 8

Section 5: For Writers

This section is for visual novel writers—the people crafting narratives, developing characters, and designing branching storylines. Ren'IDE offers powerful tools specifically for narrative design, and this section shows you how to use them effectively.

8.1 5.1 Visualizing Your Narrative

As a writer, your primary challenge is managing the complexity of branching narratives. Ren'IDE's three canvas views transform your story from abstract text into a visual structure you can navigate and understand at a glance.

8.1.1 5.1.1 Using the Story Canvas for Narrative Structure

The Story Canvas shows your project at the file level, making it ideal for understanding **high-level narrative organization**.

Use Cases for Writers:

Organizing Story Arcs: - Create one `.rpy` file per chapter or arc (e.g., `chapter1.rpy`, `chapter2.rpy`, `epilogue.rpy`) - Arrange blocks spatially on the canvas to reflect narrative flow: - Main story arc in the center - Side quests or optional content off to the sides - Endings clustered together at the bottom/right - This spatial organization helps you understand story structure without opening files

Tracking Character Arcs: - Enable **role tinting** in the canvas toolbox - Blocks are colored based on which characters appear in them - Blocks with Eileen might be blue, blocks with Lucy might be pink - Visual inspection reveals: - Which sections focus on specific characters - Where character arcs begin and end - Imbalances (e.g., one character dominates the early chapters but disappears later)

Character Filter for Focus: - Select a character from the Character Filter dropdown - Blocks that don't involve that character are dimmed - Trace that character's journey through your story - Ensure every major character has a satisfying arc

Identifying Isolated Content: - Use **grouping by connected components** - Files that aren't connected to the main story graph appear in separate groups - These might be: - Leftover content from cut scenes - Side content that needs integration - Bonus content intentionally separate from the main story

Creating a Visual Story Outline: - Before writing, create placeholder `.rpy` files for each major scene or chapter - Arrange them on the canvas in narrative order - Draw arrows (by adding temporary `jump` statements) to indicate flow - Use sticky notes to annotate sections with reminders or ideas - This becomes your visual outline—a roadmap for development

Example Workflow: 1. Create blocks: `intro.rpy`, `chapter1.rpy`, `chapter2.rpy`, `chapter3.rpy`, `ending_good.rpy`, `ending_bad.rpy` 2. Arrange them left-to-right on the canvas 3. Add placeholder labels and jumps so arrows appear 4. Enable role tinting to see character distribution 5. Begin writing, filling in each block's content

8.1.2 5.1.2 Using the Route Canvas for Plot Structure

The Route Canvas shows **label-level control flow**, perfect for understanding how players move through your story.

Use Cases for Writers:

Tracing Player Paths: - Open the Route Canvas - Visually follow the path from the `start` label - See every decision point (menu nodes) and where choices lead - Ensure every path reaches a satisfying conclusion

Verifying Branching Logic: - Menu nodes show all choices - Hover over a menu node to inspect choice text and destinations - Verify that: - Choice text is clear and player-facing - Destinations are correct - Conditional choices (if guards) are logically sound

Identifying Dead Ends: - Labels marked as **unreachable** are red or highlighted - These are scenes or endings players can never reach - Either: - Add jumps to make them reachable, or - Delete them if they're leftover from cut content

Route Highlighting for Testing: - In the Route List panel, select a route (e.g., "Start → Chapter 1 → Good Ending") - The route highlights on the canvas - Follow it step by step to verify the narrative makes sense - Repeat for every ending to ensure all paths are coherent

Understanding Convergence: - Where do different branches converge back to a common path? - The Route Canvas shows when multiple choices lead to the same label - This helps you understand how much unique content each path requires

Example Workflow: 1. Write a branching scene with multiple choices 2. Open the Route Canvas 3. Verify that every choice leads somewhere sensible 4. Use route highlighting to "play" through each branch visually 5. Identify and fix any broken paths

8.1.3 5.1.3 Using the Choice Canvas for Player Experience

The Choice Canvas shows what the **player actually sees**: choice text, menu prompts, and conditional options.

Use Cases for Writers:

Testing the Player Experience: - Read choice text as if you were the player - Does the choice make sense in context? - Is the language clear and consistent? - Are conditional choices properly guarded?

Balancing Choices: - Count how many choices each menu offers - Too few choices (just 1 or 2) may feel linear - Too many choices (6+) may overwhelm the player - Aim for 2-4 meaningful choices per menu for good pacing

Ensuring Choice Consequences: - Follow a choice pill to its destination - Continue following subsequent choices - Verify that early choices have lasting consequences (or intentionally converge if that's your design)

Identifying Conditional Choice Issues: - Choices with conditional badges show `if` statements - Ask: Is this condition clear to the player? - If a choice is hidden by a condition, does the player understand why? - Consider adding hints or feedback when conditions aren't met

Example Workflow: 1. Write a menu with choices 2. Open the Choice Canvas 3. Read the choice text aloud 4. Follow each choice to its destination 5. Verify that the player experience is coherent and satisfying

8.1.4 5.1.4 Combining All Three Canvases

Use all three canvases together for a complete picture:

- **Story Canvas:** "How is my project organized?"
- **Route Canvas:** "How does the plot flow?"
- **Choice Canvas:** "What does the player experience?"

Example Combined Workflow: 1. Use the Story Canvas to organize chapters 2. Switch to the Route Canvas to verify plot logic 3. Switch to the Choice Canvas to test player-facing text 4. Iterate: make changes in the code editor, save, and refresh the canvas views

---

8.2 5.2 Character Management

Characters are the heart of any visual novel. Ren'IDE helps you track, manage, and analyze your characters.

8.2.1 5.2.1 Character Database (Characters Tab)

The Characters tab in the Story Elements panel shows all defined characters.

Information Displayed: - **Character tag** (e.g., `e`, `lucy`) - **Display name** (e.g., "Eileen", "Lucy") - **Color** (color swatch for dialogue text) - **Dialogue count** (number of lines spoken)

Why This Matters for Writers: - See all characters in one place - Identify characters with too few lines (underdeveloped) - Identify characters with too many lines (potential over-representation)

Adding a New Character: 1. Click "+ New Character" in the Characters tab 2. Fill in: - **Tag:** Short identifier (e.g., `e` for Eileen) - **Display Name:** Full name shown in dialogue - **Color:** Hex code or color picker (e.g., `#c8ffc8` for light green) - **Other parameters:** Ren'Py allows many parameters (voice, image, dynamic name, etc.) 3. Click "Save" 4. Ren'IDE generates the `define` statement and adds it to an appropriate file (e.g., `characters.rpy`)

Editing a Character: 1. Click a character in the list 2. The Character Editor opens (side panel or modal) 3. Modify properties (name, color, parameters) 4. Click "Save" 5. The code is updated automatically

Finding Character Usages: 1. Right-click a character 2. Select "Find Usages" 3. A search results panel lists every line where the character speaks 4. Click any result to jump to that line in the editor

Jump to Definition: 1. Right-click a character 2. Select "Jump to Definition" 3. The file with the `define` statement opens at the correct line

8.2.2 5.2.2 Character Editor

The Character Editor is a detailed interface for configuring character properties.

Editable Properties:

Basic: - **Tag:** Character identifier used in code - **Display Name:** Name shown to the player - **Color:** Dialogue text color

Advanced (Ren'Py Parameters): - **Voice Tag:** For voice acting or auto-voice systems - **Image:** Default character sprite - **Dynamic Name:** If the character's name changes during the story (e.g., "???" becomes "Eileen") - **What Prefix/Suffix:** Text before/after the character name (e.g., "[eileen]" or "(thinking)") - **Callback:** Custom function called when the character speaks - **And more** (see Ren'Py documentation for full list)

Profile Notes: A free-text field for writer notes: - Character backstory - Personality traits - Voice/tone reminders - Relationships with other characters - Arc summary

These notes are saved in `project.ide.json` and are for internal use only (not shown to players).

Visual Color Picker: Click the color swatch to open a color picker. Choose a color visually instead of typing hex codes.

Example Use Case: You're developing a mystery visual novel. The player doesn't know the protagonist's name until chapter 3. Use the Character Editor to: - Set the initial display name to "???" - Add a profile note: "Reveal true name (Alex) in chapter 3" - Later, update the display name to "Alex" when appropriate

8.2.3 5.2.3 Dialogue Tracking

Ren'IDE tracks how many lines each character speaks.

Dialogue Count: Displayed in the Characters tab as a number (e.g., "47 lines").

Per-Character Word Count (in Stats): The Stats panel shows a bar chart of dialogue counts. This visual makes it easy to see: - Which characters dominate the narrative - Which characters are underutilized

Why This Matters: - **Balance:** Ensure all main characters get adequate screen time - **Pacing:** If one character has 500 lines and another has 5, is that intentional? - **Cut Content Detection:** If a character was planned to be major but only has a handful of lines, they may need more development

Unused Character Detection: Characters defined but never used in dialogue are flagged with a warning in the Characters tab and the Diagnostics panel.

Possible Causes: - Leftover from cut content - Placeholder for future development - Typo in the character tag (defined as `lucy` but used as `lcuy`)

How to Fix: - Delete the unused character definition, or - Add dialogue for that character, or - Correct the typo

Example Workflow: 1. Open the Stats panel 2. Review the per-character dialogue bar chart 3. Identify imbalances 4. Decide: Is this intentional? If not, revise to give underutilized characters more lines 5. Reopen Stats to verify the balance improved

---

8.3 5.3 Menu Builder & Choices

Menus are the core of branching narratives. Ren'IDE's visual Menu Builder helps you design choices without writing code.

8.3.1 5.3.1 Visual Menu Designer

Open the **Menus tab** in the Story Elements panel to access the Menu Builder.

Features: - See all menus in your project (extracted from code) - Create new menus with a visual interface - Add, edit, and remove choices - Configure choice properties (text, destination, conditions, custom code)

Creating a New Menu: 1. Click "+ New Menu" 2. The Menu Builder opens (modal or side panel) 3. Enter: - **Menu Prompt (Optional):** Text displayed before choices (e.g., "What do you do?") - **Menu Name:** Internal identifier (optional, for your reference) 4. Add choices (see below)

8.3.2 5.3.2 Adding and Configuring Choices

Add a Choice: 1. Click "+ Add Choice" in the Menu Builder 2. A new choice entry appears

Configure Each Choice:

Choice Text: - What the player sees (e.g., "Go left", "Stay silent", "Tell the truth") - Keep it concise (1-5 words) for readability

Destination: - Where the choice leads - Options: - **Jump to label:** Transfer control permanently - **Call label:** Call a subscene and return to the menu - Select a label from the dropdown (populated from your project's labels)

Condition (Optional): - An `if` statement that determines if the choice is available - Example: `player_intelligence >= 5` - If the condition is false, the choice is hidden from the player - Leave blank if the choice is always available

Custom Code Block (Optional): - Arbitrary Ren'Py code to execute when the choice is selected - Example: `renpy  $ player_karma += 10  "You feel good about your decision."` - This code runs before jumping/calling to the destination

Example Configuration:

Choice 1: - Text: "Help the stranger" - Destination: `jump help_stranger` - Condition: `player_karma >= 0` - Custom Code: `renpy  $ player_karma += 5`

Choice 2: - Text: "Ignore and walk away" - Destination: `jump ignore_stranger` - Condition: (none) - Custom Code: `renpy  $ player_karma -= 5`

8.3.3 5.3.3 Generating Menu Code

Once you've configured all choices: 1. Click **"Generate Code"** 2. Ren'IDE generates the Ren'Py `menu` statement 3. The code is copied to your clipboard 4. Paste it into your script at the appropriate location

Generated Code Example:

```
menu:
    "What do you do?"

    "Help the stranger" if player_karma >= 0:
        $ player_karma += 5
        jump help_stranger

    "Ignore and walk away":
        $ player_karma -= 5
        jump ignore_stranger
```

8.3.4 5.3.4 Editing Existing Menus

The Menus tab lists all menus found in your project (extracted during analysis).

To edit an existing menu: 1. Click the menu entry in the list 2. The Menu Builder opens with the menu's current configuration 3. Make changes 4. Click "Update Code" 5. Ren'IDE updates the code in place (or copies the updated code to clipboard)

Note: Editing may require manual adjustment if the original menu has complex inline code not supported by the visual builder.

8.3.5 5.3.5 Integration with Choice Canvas

Menus designed in the Menu Builder are visualized on the **Choice Canvas**: - Each menu becomes a menu node - Choice pills show the choice text and destination - Conditional badges appear if choices have `if` guards

Use the Choice Canvas to visually verify your menu design after generating code.

8.4 5.4 Go-to-Label Navigation

Large projects can have hundreds of labels. Ren'IDE's Go-to-Label features make navigation instant.

8.4.1 5.4.1 Command Palette (Ctrl+G)

The fastest way to navigate to any label:

How to Use: 1. Press **Ctrl+G** (Windows/Linux) or **Cmd+G** (macOS) from any canvas 2. A command palette appears at the top center of the screen 3. Start typing a label name (e.g., "chapter") 4. A list of matching labels appears, ranked by relevance: - **Exact matches** first

(typing “start” matches “start” exactly) - **Prefix matches** second (typing “sta” matches “start”) - **Substring matches** third (typing “art” matches “start”) - **Fuzzy matches** fourth (typing “srt” matches “start” because all letters appear in order) 5. Navigate the list with arrow keys or mouse 6. Press **Enter** or click to jump to the selected label

Zoom-on-Navigate: After navigating, the canvas: - Centers on the label - Zooms in to at least scale 1.0 (so the label is clearly visible) - Highlights the label briefly

Why Use This: - Faster than manually scrolling or searching - Works across all three canvases (Story, Route, Choice) - Fuzzy search tolerates typos and incomplete names

8.4.2 5.4.2 Toolbox Label Search

Each canvas (Story, Route, Choice) has a **“Go to Label”** search box in the toolbox (top-left corner).

How to Use: 1. Locate the search box in the canvas toolbox 2. Type a label name (or part of it) 3. A dropdown list of matching labels appears 4. Click a label to navigate

This is persistent (always visible) unlike the command palette (which appears on Ctrl+G and then dismisses).

When to Use: - When you prefer a persistent search box - When you’re browsing labels (not just jumping to one specific label)

8.4.3 5.4.3 Comparison: Palette vs. Toolbox

Feature	Command Palette (Ctrl+G)	Toolbox Search
Trigger	Keyboard shortcut	Always visible
Interface	Modal (covers content)	Persistent (part of toolbox)
Ranking	Intelligent fuzzy ranking	Simple substring match
Use Case	Fast single jump	Browsing/exploring labels

Both zoom on navigate and work on all three canvases.

8.5 5.5 Story Flow Visualization

Understanding how your story flows—how players move from scene to scene—is critical for writers. Ren’IDE provides several tools for visualizing flow.

8.5.1 5.5.1 Understanding Arrows

Arrows on the Story Canvas and Route Canvas represent narrative connections.

Jump Arrows (Solid): - Indicate a `jump` statement - Transfer control permanently to another label - Example: `jump chapter2` from the end of chapter 1

Call Arrows (Dashed/Distinct): - Indicate a `call` statement - Call a subscene and return to the caller after it ends - Example: `call flashback` to show a flashback scene, then return to present

Fall-Through (Dotted, Route Canvas Only): - When one label ends without a `jump`, `call`, or `return`, control falls through to the next label - Represented by dotted lines on the Route Canvas

Arrow Colors: - Colors may match the source node's route color (if route highlighting is active) - Or use a default color (configurable in Settings)

Arrow Thickness: - May reflect the number of connections (thicker = more jumps to this destination) - Or uniform thickness (depending on canvas settings)

8.5.2 5.5.2 Block Classification

Blocks on the Story Canvas are classified by role:

Root Blocks: - Blocks with no incoming jumps - Entry points to your story (e.g., `script.rpy` with the `start` label) - Usually at the "top" or "left" in auto-layout

Leaf Blocks: - Blocks with no outgoing jumps - Endings or dead ends - Usually at the "bottom" or "right" in auto-layout

Branching Blocks: - Blocks that contain choices (menus) - Multiple outgoing arrows

Story Blocks: - Blocks that contain labels (most `.rpy` files)

Config/Screen Blocks: - Blocks that don't contain labels (e.g., `options.rpy`, `screens.rpy`) - May be visually distinct (different color or icon)

Understanding these classifications helps you see the structure of your project at a glance.

8.5.3 5.5.3 Layout Strategies

Manual Positioning: - Drag blocks to arrange them in a way that makes sense to you - Example: Linear story left-to-right, side quests above/below - Positions are saved and persist across sessions

Auto-Layout with Redraw: - Click the **Redraw** button in the toolbar - Ren'IDE analyzes jump/call connections and arranges blocks automatically - Useful for: - Initial layout of a new project - Cleaning up a cluttered canvas - Seeing the algorithmic "ideal" layout

Layout Modes: - **Flow (default):** Hierarchical flow from root to leaf - **Hierarchical:** Strict tree layout - **Circular:** Blocks in a circle (useful for cyclic stories) - **Clustered Flow:** Flow with grouping containers

Grouping: - **By Connected Components:** Groups blocks that are connected by jumps/calls - **By File Prefix:** Groups blocks with common file name prefixes (e.g., `chapter1_*.rpy`)

Enable grouping in the toolbox, then switch to Clustered Flow layout to see grouping containers.

8.5.4 5.5.4 Using Sticky Notes

Add **sticky notes** to the canvas for annotations:

How to Add: 1. Click the **Add Note** button in the toolbar 2. A sticky note appears on the canvas 3. Click it to edit the text 4. Drag it to position it near relevant blocks

Use Cases: - Mark sections for revision ("TODO: Add more dialogue here") - Document narrative structure ("This is the climax") - Leave notes for collaborators ("Need input from artist on this scene") - Track development status ("Done", "In Progress", "Needs Review")

Persistence: Notes are saved in `project.ide.json` and persist across sessions.

8.6 5.6 Variable Tracking

Variables drive branching logic and state in visual novels. Ren'IDE helps you track them.

8.6.1 5.6.1 Variables Tab

The **Variables tab** in the Story Elements panel shows all globally defined variables.

Information Displayed: - **Variable name** (e.g., `player_name`, `relationship_score`) - **Type:** `define` (unchanging constant) or `default` (initial value that can change) - **Initial value** (e.g., `"Player"`, `0`, `True`)

Finding Variable Usages: 1. Right-click a variable 2. Select "Find Usages" 3. A search results panel lists every location where the variable is read or written 4. Click any result to jump to that line

Jump to Definition: 1. Right-click a variable 2. Select "Jump to Definition" 3. The file with the `define` or `default` statement opens

8.6.2 5.6.2 Understanding Variable Scope

Global Variables (shown in Variables tab): - Defined with `define` or `default` at the top level (outside labels) - Accessible throughout the entire project

Local Variables (not tracked by Ren'IDE): - Defined with `$` inside labels (e.g., `$ temp = 5`) - Only exist within that label - Not shown in the Variables tab

As a writer, focus on global variables for tracking story state (player choices, relationship scores, flags, etc.).

8.6.3 5.6.3 Using Variables in Choices

Variables are often used in conditional choices:

```
menu:
    "What do you say?"

    "I love you" if relationship_score >= 10:
        jump confession

    "Let's just be friends":
        jump friend_zone
```

Use the Choice Canvas to see which choices are conditional. The conditional badge shows the `if` statement.

Tips for Writers: - Give variables descriptive names (e.g., `eileen_affection` instead of `var1`) - Document what each variable tracks in comments - Use Find Usages to ensure variables are set before being checked in conditions

---

8.7 5.7 Writing Tips & Workflow

Here are some best practices for writers using Ren'IDE.

8.7.1 5.7.1 Starting a New Story

Step 1: Outline on the Canvas - Create placeholder `.rpy` files for each major scene or chapter - Arrange them on the Story Canvas in narrative order - Add placeholder labels and jumps so arrows appear - Use sticky notes to annotate sections with ideas or reminders

Step 2: Define Characters - Use the Characters tab to add all characters - Fill in display names and colors - Write profile notes with character backstories and arcs

Step 3: Define Key Variables - In a file like `variables.rpy`, define variables for tracking state (e.g., `default player_karma = 0`) - Document what each variable means in comments

Step 4: Write the First Draft - Start with the `start` label and write linearly - Don't worry about perfection; focus on getting the story down - Use the Menu Builder for choices as you go

Step 5: Visualize and Revise - Switch to the Route Canvas to verify flow - Use the Choice Canvas to test player experience - Iterate: make changes in the editor, save, and refresh the canvases

8.7.2 5.7.2 Organizing Files and Labels

File Organization: - One file per chapter, scene, or narrative unit - Use descriptive names (e.g., `chapter1_intro.rpy`, `ending_happy.rpy`) - Group related files with common prefixes for easy grouping on the canvas

Label Naming: - Use clear, descriptive label names (e.g., `chapter1_start`, `flashback_childhood`, `ending_true`) - Avoid generic names like `scene1`, `scene2` (hard to distinguish) - Use underscores or hyphens for readability

Comments: - Add comments to document narrative intent `renpy # Player discovers the truth about their past label revelation: "..."`

8.7.3 5.7.3 Using Sticky Notes for Annotations

Mark Sections for Revision:

`Note: "TODO: Expand this dialogue"`

Document Structure:

`Note: "Act 2 begins here"`

Track Development Status:

`Note: " Done" or " In Progress"`

8.7.4 5.7.4 Collaborative Workflow Considerations

If you're working with a team:

Version Control: - Commit your project to Git (or another VCS) - Commit `project.ide.json` so teammates see your canvas layout and compositions

Communicate with Canvas: - During meetings, open the Story Canvas or Route Canvas to discuss narrative structure - Non-technical team members can understand the visual representation

Asset Coordination: - Writers specify which images/audio are needed in the script - Artists see references in the Diagnostics panel (missing asset warnings) - Artists add assets to the project; writers verify in the Assets tab

Use Diagnostics: - Before committing, check the Diagnostics panel - Fix any errors (invalid jumps, undefined characters) - Fix or document warnings (unreachable labels may be intentional WIP content)

8.7.5 5.7.5 Best Practices for Large Projects

Modularize: - Break your story into logical modules (chapters, acts, routes) - One file per module - Use `call` for reusable subscenes (e.g., flashbacks, recurring dreams)

Test Regularly: - Use the Route Canvas to verify every path reaches an ending - Use the Choice Canvas to test player-facing text - Run the game frequently to catch issues early

Track Progress: - Use the Stats panel to monitor word count and dialogue distribution - Set milestones (e.g., "Reach 50,000 words," "Complete all character arcs")

Refactor as You Go: - If files get too large (500+ lines), split them - If labels become unwieldy, reorganize - Use Ren'IDE's visualization to guide refactoring decisions

8.7.6 5.7.6 Writer-Specific Keyboard Shortcuts

Shortcut	Action
`Ctrl+G`	Go to Label (fastest navigation)
`Ctrl+S`	Save All (save frequently!)
`Ctrl+Z`	Undo (experiment freely)
`N`	New Block (add a scene quickly)

End of Section 5: For Writers

Chapter 9

Section 6: For Artists

This section is for visual novel artists—the people creating backgrounds, character sprites, CGs, UI elements, music, and sound effects. Ren'IDE offers powerful asset management tools and visual composers designed to streamline your workflow.

9.1 6.1 Image Asset Management

Managing hundreds of image files is a common challenge in visual novel development. Ren'IDE's Image Asset Manager makes browsing, organizing, and integrating images effortless.

9.1.1 6.1.1 Image Assets Tab Overview

Open the **Images tab** in the Story Elements panel (right sidebar) to access the Image Asset Manager.

What You See: - **Folder tree:** Images organized by directory structure (e.g., `game/images/characters/`, `game/images/backgrounds/`) - **Visual thumbnails:** Each image shows a small preview - **Metadata:** Ren'Py name, tags, file size, dimensions

Folder Navigation: - Click a folder to expand/collapse it - Browse through your entire image library without opening a file browser

9.1.2 6.1.2 Scanning External Directories

If you keep assets in an external library (e.g., a shared Dropbox folder or a separate asset repository), you can **scan external directories** without copying files into your project.

How to Scan: 1. Click **"Scan External Directory"** in the Images tab 2. A file picker appears 3. Navigate to the external directory (e.g., `/Users/yourname/Dropbox/Assets/`) 4. Click "Select Folder" 5. Ren'IDE scans the directory and adds all images to the asset list

Benefits: - **No duplication:** Images remain in their original location - **Shared libraries:** Multiple projects can reference the same asset library - **Version control friendly:** External assets don't bloat your project repo

Scan Persistence: The scan directory handle is saved in `project.ide.json`. When you reopen the project, Ren'IDE automatically rescans the external directory to reflect any changes.

Refreshing: If assets in the external directory change: 1. Right-click the scan directory entry 2. Select "Refresh" 3. Ren'IDE rescans and updates the asset list

9.1.3 6.1.3 Importing Images

To copy images into your project's `game/images/` directory:

Drag-and-Drop: 1. Open the Images tab 2. Drag image files from your OS file browser (Finder, Explorer, Nautilus, etc.) 3. Drop them onto the Images tab 4. A dialog appears: - **Destination subfolder:** Choose where to copy them (e.g., `characters/`, `backgrounds/`, or create a new subfolder) - **Tags:** Add custom tags for organization (optional) 5. Click "Import" 6. Images are copied to the project and appear in the asset list with thumbnails

Import Button: 1. Click **"Import Images"** in the Images tab 2. A file picker appears 3. Select one or more images 4. Choose destination subfolder and tags (as above) 5. Click "Import"

Batch Import: Select multiple images in the file picker to import them all at once.

9.1.4 6.1.4 Image Metadata

Every image can have metadata for organization and easy reference.

Metadata Fields:

Ren'Py Name: - The name Ren'Py uses to reference this image in code - Example: For `game/images/eileen_happy.png`, the Ren'Py name is `eileen happy` (spaces, not underscores) - Ren'IDE auto-generates this from the file name, but you can override it

Tags: - Custom tags for categorization (e.g., "character", "CG", "UI", "sprite") - Multiple tags allowed (comma-separated) - Use tags to filter assets (see below)

Subfolder: - The subfolder within `game/images/` where the file is stored - Example: `characters/` or `backgrounds/`

Editing Metadata: 1. Double-click an image in the asset list 2. The **Image Editor** opens (modal or side panel) 3. Edit the Ren'Py name, tags, or subfolder 4. Click "Save"

9.1.5 6.1.5 Quick Code Generation

One of the most useful features for artists: instantly copy Ren'Py code to your clipboard.

Right-Click Context Menu: Right-click any image in the asset list: - **Copy `scene` statement:** Copies `scene bg_office` (or the appropriate name) to clipboard - **Copy `show` statement:** Copies `show eileen happy` to clipboard

Workflow: 1. Find the image you need 2. Right-click and copy the appropriate statement 3. Switch to the code editor 4. Paste the statement into your script

Why This Matters: - **No typos:** The Ren'Py name is always correct - **Fast:** No need to manually type `show eileen happy at left with dissolve` - **Consistent:** Everyone on the team uses the same naming conventions

9.1.6 6.1.6 Filtering and Searching Images

For large projects with hundreds of images:

Search Box (if available): - Type an image name or tag - The asset list filters to show only matching images

Tag Filter: - Select a tag from the dropdown - Only images with that tag are shown

Folder Filter: - Collapse folders to hide their contents - Focus on specific subfolders

Example Use Case: You need to find all character sprites. Filter by the "sprite" tag. The list shows only sprites, ignoring backgrounds and UI elements.

9.1.7 6.1.7 Drag to Composers

Images can be dragged from the asset list directly onto: - **Scene Composer stage** (to add them to a scene composition) - **Screen Layout Composer** (to add them to a screen layout)

This makes composition workflows seamless—no need to manually browse for files in a file picker.

9.2 6.2 Audio Asset Management

Audio asset management works similarly to image management, with features tailored for music and sound effects.

9.2.1 6.2.1 Audio Assets Tab Overview

Open the **Snd (Audio) tab** in the Story Elements panel to access the Audio Asset Manager.

What You See: - **Folder tree:** Audio files organized by directory (e.g., `game/audio/music/`, `game/audio/sfx/`) - **File list:** Each audio file shows: - File name - Format (MP3, OGG, WAV, FLAC, etc.) - Duration (if available) - File size

9.2.2 6.2.2 Scanning External Audio Directories

Like images, you can scan external directories for audio: 1. Click **"Scan External Directory"** in the Audio tab 2. Select a folder with audio files 3. Ren'IDE adds them to the asset list without copying

Use Cases: - Reference a shared music library - Keep large audio files outside the project repo

9.2.3 6.2.3 Importing Audio

Drag-and-Drop: - Drag audio files from your OS file browser - Drop onto the Audio tab - Choose destination subfolder (e.g., `music/`, `sfx/`) - Click "Import"

Import Button: - Click "Import Audio" - Select files in the file picker - Choose destination and import

9.2.4 6.2.4 Built-in Audio Player

Ren'IDE includes a built-in audio player so you can preview sounds without leaving the app.

To Play an Audio File: 1. Click an audio file in the asset list 2. The audio player appears at the bottom of the Audio tab (or in a side panel) 3. Controls: - **Play/Pause button** - **Seek bar:** Drag to jump to a specific time - **Volume slider:** Adjust playback volume - **Loop toggle:** Enable looping for background music testing

Why This Matters: - Quickly audition sound effects or music tracks - Verify audio quality before integrating into the game - No need to open a separate audio player app

9.2.5 6.2.5 Audio Metadata

Each audio file can have metadata:

Ren'Py Name: - The identifier used in code (e.g., `audio/music/theme.mp3`) - Auto-generated from file path

Tags: - Custom tags (e.g., "battle", "sad", "ambience") - Use for categorization and filtering

Subfolder: - Location within `game/audio/`

Editing Metadata: Double-click an audio file to open the Audio Editor and modify metadata.

9.2.6 6.2.6 Quick Code Generation

Right-click any audio file for quick code copying: - **Copy `play music` statement:** `play music "audio/music/theme.mp3"` - **Copy `play sound` statement:** `play sound "audio/sfx/doorbell.wav"` - **Copy `queue audio` statement:** `queue music "audio/music/theme.mp3"`

Paste into your script for instant audio integration.

9.2.7 6.2.7 Audio Workflow Tips for Artists

Organize by Type: - `music/` for background music - `sfx/` for sound effects - `voice/` for voice acting (if applicable) - `ambience/` for ambient loops

Use Tags: - Tag tracks by mood ("happy", "tense", "sad") - Tag sound effects by category ("ui", "footsteps", "doors")

Test with Loop Toggle: - Enable loop in the player to hear how music loops - Check for jarring transitions or pops

Coordinate with Writers: - Writers specify which audio is needed in the script - Diagnostics panel flags missing audio - Artists add audio, writers verify in the Audio tab

---

9.3 6.3 Scene Composer

The Scene Composer is a visual tool for composing scenes: laying out backgrounds and character sprites, applying transforms, and generating Ren'Py code.

9.3.1 6.3.1 Overview

What is the Scene Composer? A canvas where you can visually arrange images (backgrounds, sprites, overlays) to create a scene mockup. When you're satisfied, copy the generated Ren'Py code (`scene` and `show` statements) into your script.

Use Cases: - **Mockups:** Show the writer or director what a scene will look like - **Reference compositions:** Save complex scenes for later reuse - **Quick layouts:** Experiment with sprite positions without writing code - **Social media:** Export scenes as flat PNGs for promotion

Opening the Scene Composer: 1. Go to the **Composers tab** in the Story Elements panel 2. Click **"+ New Scene"** 3. The Scene Composer opens (full-screen modal or dedicated panel)

9.3.2 6.3.2 Creating a Composition

Stage: The central area where you place images. The stage represents the game window.

Adding Images: - **Drag from Image Assets tab:** Drag an image from the asset list onto the stage - **Or use the "Add Image" button:** Click it, select an image from a file picker

Types of Images: - **Backgrounds:** Full-screen images (e.g., `bg office`, `bg park`) - **Sprites:** Character images, usually with transparency (e.g., `eileen happy`, `lucy sad`) - **Overlays:** Additional elements (UI elements, effects)

Initial Placement: Images appear at the center of the stage. Drag them to position.

9.3.3 6.3.3 Stage Controls

Configurable Resolution: Set the stage resolution to match your game's resolution: - **Presets:** 1920×1080, 1280×720, 1024×768, 800×600 - **Custom:** Enter any width and height

Why This Matters: Ensures your composition accurately reflects how the scene will appear in-game.

Zoom Controls: - Zoom in/out to see details or get an overview - Fit stage to viewport

Grid/Guides (if available): - Toggle grid overlay for alignment - Snap to grid for precise positioning

9.3.4 6.3.4 Sprite Transforms

Select a sprite to access per-sprite transform controls (usually in a side panel or toolbar):

Position (X, Y): - Drag the sprite on the stage, or - Enter exact X/Y coordinates in the controls

Scale/Zoom: - Slider or input field to scale the sprite - Example: 0.5 = half size, 2.0 = double size - Preserves aspect ratio

Flip Horizontal: - Checkbox or button to flip the sprite left/right - Useful for reusing sprites in different orientations

Flip Vertical: - Checkbox or button to flip top/bottom (less common)

Rotate: - Slider or input to rotate the sprite (degrees) - Example: 90° rotates the sprite 90 degrees clockwise

Alpha/Opacity: - Slider from 0.0 (fully transparent) to 1.0 (fully opaque) - Example: 0.5 = 50% transparent (ghostly effect)

Blur: - Slider to apply a blur effect - Use for background/out-of-focus elements

Transform Preview: Changes apply in real-time on the stage.

9.3.5 6.3.5 Layer Ordering

Images on the stage are layered (like layers in Photoshop): - **Bottom layer:** Usually the background - **Middle layers:** Character sprites - **Top layers:** Overlays or effects

Reordering Layers: - In the layer list (side panel), drag a layer up or down - Or use **Bring Forward** / **Send Backward** buttons

Why This Matters: Ensure characters appear in front of the background and overlays appear in front of everything.

Example Layer Stack: 1. Background (`bg office`) 2. Character sprite 1 (`eileen happy`) 3. Character sprite 2 (`lucy sad`) 4. Overlay effect (`vignette`)

9.3.6 6.3.6 Code Generation

Once your composition is complete:

Generate Code: 1. Click **"Generate Code"** button 2. Ren'IDE generates Ren'Py `scene` and `show` statements: `renpy` `scene bg office` `show eileen happy at left with dissolve:` `zoom 0.9` `alpha 1.0` `show lucy sad at right with dissolve:` `zoom 1.0` `xflip True` 3. The code is copied to your clipboard

Paste into Script: Open your `.rpy` file in the editor and paste the generated code. The scene will render in-game exactly as you composed it.

Transform Attributes: The generated code includes all transforms you applied (zoom, flip, rotate, alpha, blur).

9.3.7 6.3.7 Exporting as PNG

You can export the composition as a flat PNG image:

How to Export: 1. Click **"Export as PNG"** 2. Choose a file name and location 3. Ren'IDE renders the stage to a PNG file

Use Cases: - Social media promotion (share a scene preview) - Development logs or dev blogs - Reference images for team discussions - Marketing materials

9.3.8 6.3.8 Managing Compositions

Saving Compositions: 1. Click **"Save Composition"** 2. Enter a name (e.g., "Office Scene - Day 1") 3. The composition is saved in `project.ide.json`

Loading Compositions: 1. Go to the Composers tab 2. Click a saved composition 3. It opens in the Scene Composer with all layers and transforms intact

Deleting Compositions: 1. Right-click a composition in the library 2. Select "Delete" 3. Confirm

Sharing Compositions: Commit `project.ide.json` to version control so teammates can load your compositions.

9.4 6.4 ImageMap Composer

The ImageMap Composer is a tool for creating **clickable imagemaps**: images with rectangular hotspots that trigger actions (jumps or calls) when clicked.

9.4.1 6.4.1 Overview

What is an Imagemap? A Ren'Py construct where a single image has multiple clickable regions. Common uses: - Point-and-click navigation (e.g., a map where clicking regions goes to different locations) - Interactive UIs (e.g., a phone screen with app icons) - Puzzle interfaces

Opening the ImageMap Composer: 1. Go to the Composers tab 2. Click **"+ New ImageMap"** 3. The ImageMap Composer opens

9.4.2 6.4.2 Setting Ground and Hover Images

Ground Image: - The base image displayed on screen - Example: A map, a UI mockup, a room background

Hover Overlay (Optional): - An alternate image displayed when the mouse hovers over a hotspot - Example: Highlighted regions, glowing buttons

Setting Images: - **Drag from Image Assets tab:** Drag an image onto the "Ground Image" or "Hover Image" placeholder - **Or click "Set Ground Image":** Select from a file picker

9.4.3 6.4.3 Drawing Hotspots

Creating a Hotspot: 1. Click **"Add Hotspot"** or click and drag on the ground image 2. A rectangular hotspot appears 3. Drag the edges or corners to resize 4. Drag the hotspot to reposition

Multiple Hotspots: Create as many hotspots as needed. Each hotspot can have a different action.

Deleting a Hotspot: - Select the hotspot - Press `Delete` or click the trash icon

9.4.4 6.4.4 Configuring Hotspot Actions

Select a hotspot to configure its properties (in a side panel):

Hotspot Name: - Internal identifier (e.g., "office_button", "map_region_north")

Action Type: - **Jump:** Transfer control permanently to a label - **Call:** Call a label and return

Target Label: - Select from a dropdown (populated with all labels in your project) - Example: `jump office` or `call flashback`

Condition (Optional): - An `if` statement that determines if the hotspot is active - Example: `if map_unlocked` - If the condition is false, the hotspot is non-clickable

9.4.5 6.4.5 Code Generation

When your imagemap is complete:

Generate Code: 1. Click **"Generate Code"** 2. Ren'IDE generates a `screen` with `imagebutton` or `imagemap` code: "'renpy screen map_screen: imagemap: ground "map.png" hover "map_hover.png"

```
    hotspot (100, 50, 200, 150) action Jump("office")
    hotspot (350, 50, 200, 150) action Jump("park")
```

```
3. The code is copied to clipboard

**Paste into Script:**
Add the generated screen to your project (e.g., in `screens.rpy`). Call it with:
```renpy
call screen map_screen
```
```

The player can now click the hotspots to navigate.

9.4.6 6.4.6 Managing ImageMap Compositions

Save Composition: Click "Save Composition" and name it (e.g., "World Map").

Load Composition: Go to the Composers tab, click the saved composition, and it opens in the ImageMap Composer.

Delete Composition: Right-click and select "Delete".

9.5 6.5 Asset Preview Features

Ren'IDE includes several features for previewing assets without leaving the app.

9.5.1 6.5.1 Image Thumbnails

In the Project Explorer: - Image files show a small thumbnail icon - Hover to see a larger preview (if supported)

In the Image Assets Tab: - Each image has a thumbnail - Thumbnails are generated asynchronously (you may see a placeholder briefly while it loads)

9.5.2 6.5.2 Double-Click to Open in Editor

Images: - Double-click an image in the asset list - The **Image Editor** opens, showing: - Full-size preview - Metadata (Ren'Py name, tags, dimensions, file size) - Metadata edit controls

Audio: - Double-click an audio file - The **Audio Editor** opens with the built-in player

9.5.3 6.5.3 Full-Size Image Preview

In the Image Editor: - The image displays at full size (or scaled to fit the panel) - Zoom in/out if the image is large - Rotate or flip for review (does not modify the file)

9.5.4 6.5.4 Dimensions and File Size Display

Image Editor: - Dimensions: `1920×1080` - File size: `2.4 MB`

Audio Editor: - Duration: `3:45` - Format: `MP3, 320 kbps` - File size: `8.5 MB`

Why This Matters: - Verify image dimensions match your game's resolution - Check file sizes to optimize for web distribution (if applicable)

9.6 6.6 Drafting Mode for Artists

Drafting Mode adds placeholder assets for missing images and audio, allowing the game to run during development.

9.6.1 6.6.1 What is Drafting Mode?

When enabled, Ren'Py placeholders replace missing assets: - **Missing images:** A gray rectangle with the asset name overlaid - **Missing audio:** Silence (or a placeholder tone, depending on configuration)

9.6.2 6.6.2 Enabling Drafting Mode

Click the **Drafting Mode** button in the toolbar (toggle on/off).

Visual Indicator: When active, the button highlights, or a badge appears in the status bar.

9.6.3 6.6.3 Why Artists Should Use This

Scenario: The writer has referenced 50 images in the script, but you've only created 20 so far.

Without Drafting Mode: Running the game crashes or shows errors for the 30 missing images.

With Drafting Mode: - Missing images display as placeholders - The game runs normally - You and the writer can test gameplay and dialogue flow - You continue creating assets without blocking development

9.6.4 6.6.4 Workflow

1. Writer writes the script, referencing images by name
2. Artist enables Drafting Mode

3. Artist runs the game to see how scenes will look (with placeholders for missing assets)
4. Artist creates assets one by one
5. As assets are added to `game/images/`, they replace placeholders
6. Eventually, all placeholders are gone; Drafting Mode can be disabled

9.6.5 6.6.5 Placeholder Customization (If Available)

Some versions of Ren'Py or Ren'IDE allow customizing placeholders: - Change placeholder color - Add custom placeholder images (e.g., a "WIP" graphic)

Check Ren'Py documentation or Ren'IDE settings for options.

9.7 6.7 Artist Workflow Tips

Here are best practices for artists using Ren'IDE.

9.7.1 6.7.1 Organizing Assets by Scene/Character

Folder Structure:

```
game/images/
  characters/
      eileen/
          eileen_happy.png
          eileen_sad.png
          eileen_angry.png
      lucy/
          lucy_happy.png
          lucy_sad.png
  backgrounds/
      bg_office.jpg
      bg_park.jpg
      bg_home.jpg
  ui/
      button_idle.png
      button_hover.png
```

Benefits: - Easy to find specific assets - Clear organization for teammates - Subfolder-based filtering in the asset manager

9.7.2 6.7.2 Naming Conventions

Descriptive Names: - Use names that describe the content: `eileen_happy.png`, not `char1_01.png` - Include character name, emotion, and variant (if multiple versions exist)

Consistent Format: - Stick to one naming scheme across the project - Example: `[character]_[emotion]_[varia` - `eileen_happy_talking.png`, `eileen_happy_standing.png`

Match Ren'Py Conventions: - Ren'Py converts underscores to spaces for image names - `eileen_happy.png` becomes `eileen happy` in code - Avoid special characters in file names

9.7.3 6.7.3 Using Tags for Filtering

Assign tags to assets: - **Type tags:** "sprite", "background", "CG", "UI" - **Status tags:** "final", "WIP", "placeholder" - **Mood tags:** "happy", "sad", "tense" (for backgrounds)

Filter by tags in the asset manager to quickly find what you need.

9.7.4 6.7.4 Coordinating with Writers

Writers Reference Assets: Writers use the image/audio names in the script:

```
show eileen happy at left
scene bg office
play music "theme.mp3"
```

Artists Add Assets: Create the assets and add them to the project (via import or scan).

Verification: - Writers check the Diagnostics panel for missing asset warnings - Artists check the asset list to see what's needed - Iterate until all assets are in place

Communication: - Use sticky notes on the canvas to annotate asset needs - Use task lists in the Diagnostics panel to track asset TODOs - Comment in the code to specify asset requirements

9.7.5 6.7.5 Version Control for Assets

Git LFS (Large File Storage): If using Git, consider Git LFS for large image and audio files:

```
git lfs track "*.png"
git lfs track "*.jpg"
git lfs track "*.mp3"
git lfs track "*.ogg"
```

This prevents the repo from bloating.

Asset Directories: - Commit finalized assets to the repo - Keep WIP assets in a separate directory (excluded from version control)

9.7.6 6.7.6 Optimizing Assets

Image Optimization: - Compress images to reduce file size (use tools like TinyPNG, ImageOptim) - Choose appropriate formats: - **PNG:** For sprites with transparency - **JPEG:** For backgrounds (no transparency needed) - **WebP:** For smaller file sizes (check Ren'Py support)

Audio Optimization: - Use OGG Vorbis for music (good compression, wide support) - Use MP3 or OGG for sound effects - Avoid WAV (uncompressed, large file sizes) unless necessary

9.7.7 6.7.7 Testing with Scene Composer

Mockup Workflow: 1. Create a scene composition in the Scene Composer 2. Export as PNG and share with the team for feedback 3. Iterate on the composition 4. Generate code and integrate into the script

Benefits: - Visual approval before writing code - Experiment with layouts quickly - Reuse compositions across scenes

9.7.8 6.7.8 Leveraging Quick Code Copy

Speed Up Integration: Instead of manually typing:

```
show eileen happy at left
```

Right-click the image in the asset list, copy the `show` statement, and paste.

Avoid Typos: Using copy-paste ensures the asset name is correct.

End of Section 6: For Artists

Chapter 10

Section 7: For Developers

This section is for programmers and technical contributors—the people building systems, writing complex code, managing project structure, and integrating with the Ren'Py SDK. Ren'IDE offers powerful development tools designed for technical workflows.

10.1 7.1 Code Editor for Developers

As a developer, you'll spend significant time in the code editor. Ren'IDE's Monaco editor (the VS Code engine) provides professional-grade editing features.

10.1.1 7.1.1 Monaco Editor Features

Why Monaco? - **Industry standard:** The same engine powering Visual Studio Code - **Feature-rich:** IntelliSense, multi-cursor, regex find/replace, code folding, and more - **Extensible:** Custom grammars and semantic tokens for Ren'Py

Core Capabilities: - Syntax highlighting with TextMate grammars - Context-aware autocomplete (IntelliSense) - Code snippets with tab stops - Multi-file editing with split panes - Find and replace (with regex support) - Multi-cursor editing - Code folding - Bracket matching - Auto-indentation

10.1.2 7.1.2 Multi-File Editing Workflows

Split Panes: - Open two files side by side for: - Comparing implementations - Copying code between files - Referencing documentation while coding - Drag tabs between panes to rearrange

Tab Management: - Open as many tabs as needed - Drag tabs to reorder - Close inactive tabs to reduce clutter - Use "Close Others" to focus on one file

Persistent State: - Open tabs and pane layout are saved in `project.ide.json` - When you reopen the project, your editor state is restored

10.1.3 7.1.3 Advanced Editing Techniques

Multi-Cursor Editing: 1. Hold `Ctrl` (Windows/Linux) or `Cmd` (macOS) and click multiple locations 2. Multiple cursors appear 3. Type to edit all locations simultaneously 4. Press `Escape` to return to single cursor

Use Cases: - Rename a variable in multiple places - Add the same prefix/suffix to multiple lines - Align code vertically

Column Selection: 1. Hold `Alt` and drag vertically 2. A rectangular selection appears 3. Type to edit all selected lines at once

Use Cases: - Editing tabular data - Adding indentation to multiple lines - Commenting out blocks of code

Move Line Up/Down: - `Alt+Up`: Move the current line (or selected lines) up - `Alt+Down`: Move them down - Preserves indentation

Delete Line: - `Ctrl+Shift+K`: Delete the current line - No need to select the entire line first

Toggle Line Comment: - `Ctrl+/`: Comment or uncomment the current line (adds or removes `#`) - Works on multiple selected lines

Bracket Matching: - Place the cursor next to a bracket, parenthesis, or brace - The matching pair is highlighted - Press `Ctrl+Shift+\` to jump to the matching bracket (if supported)

Auto-Indentation: - Press `Enter` after a line ending with `:` (e.g., `label start:`, `if condition:`) - The next line is automatically indented - Monaco detects Ren'Py's indentation-based syntax

10.1.4 7.1.4 Find and Replace with Regex

Find: - Press `Ctrl+F` to open the find bar - Enter search text - Options: - **Match Case** (Aa icon) - **Whole Word** (ab| icon) - **Regex** (.*) icon) - Navigate matches with up/down arrows

Replace: - Press `Ctrl+H` to open find/replace - Enter replacement text - Options: - **Replace:** Replace current match - **Replace All:** Replace all matches in the file

Regex Examples:

Find all labels: - Pattern: `label \w+:` - Matches: `label start:`, `label chapter1:`

Find all jumps: - Pattern: `jump \w+` - Matches: `jump start`, `jump ending`

Replace with capture groups: - Find: `label (\w+):` - Replace: `label new_$1:` - Result: `label start:` becomes `label new_start:`

Find all character definitions: - Pattern: `define \w+ = Character\(.*\)` - Matches: `define e = Character("Eileen", color="#c8ffc8")`

10.2 7.2 Syntax Highlighting

Ren'IDE uses a custom **TextMate grammar** and **semantic token provider** for accurate Ren'Py syntax highlighting.

10.2.1 7.2.1 TextMate Grammar

What is a TextMate Grammar? A set of regex patterns that define how to color different language constructs. TextMate grammars are used by VS Code, Sublime Text, and other editors.

Ren'Py Grammar (`renpy.tmLanguage.json`): Ren'IDE includes a grammar specifically designed for Ren'Py, supporting: - **Keywords:** `label`, `jump`, `call`, `menu`, `if`, `else`, `elif`, `while`, `for`, `define`, `default`, `return`, `pass`, `scene`, `show`, `hide`, `play`, `stop`, `queue`, `with`, etc. - **Strings:** Dialogue text in quotes (single or double) - **Comments:** Lines starting with `#` - **Operators:** `=`, `+`, `-`, `*`, `/`, `==`, `!=`, `<`, `>`, `and`, `or`, `not`, etc. - **Numbers:** Integers and floats - **Built-in functions:** `renpy.call`, `renpy.jump`, `renpy.pause`, etc.

10.2.2 7.2.2 Semantic Token Provider

Beyond syntax, Ren'IDE adds **semantic highlighting**:

What are Semantic Tokens? Tokens based on the meaning of code, not just its syntax. For example, a label name is semantically different from a variable name, even if both are identifiers.

Semantic Coloring in Ren'IDE: - **Labels:** Highlighted distinctly when defined (`label start:`) and when referenced (`jump start`) - **Variables:** Global variables from `define`/`default` are colored differently from local variables - **Screen names:** In `screen` definitions and `call screen` statements - **Character tags:** In dialogue lines - **Image names:** In `show`/`scene` statements

Benefits: - **Easier code comprehension:** Different concepts have different colors - **Faster debugging:** Spot typos or incorrect references visually - **Consistent styling:** Semantic tokens update dynamically as you modify code

10.2.3 7.2.3 Customizing Colors (If Supported)

Some versions of Ren'IDE allow customizing syntax colors: 1. Go to Settings → Editor → Syntax Colors 2. Choose a color scheme or customize individual token colors 3. Changes apply immediately

Check your version's documentation for customization options.

10.3 7.3 Snippets Library

Snippets are templates for common code patterns. Ren'IDE includes 28+ built-in snippets and supports user-defined snippets.

10.3.1 7.3.1 Built-in Snippets Overview

Structural Snippets: - `label`: Create a new label with placeholder `renpy label ${1:label_name}: ${2:# your content} return` - `menu`: Create a menu with two choices "'renpy menu: "${1:What do you do?}"

```
  "${2:Choice 1}":
      ${3:jump somewhere}

  "${4:Choice 2}":
      ${5:jump elsewhere}
```

```
- `if`: If/else block
```renpy
if ${1:condition}:
    ${2:# code}
else:
    ${3:# code}
```renpy
```

Display Snippets: - `show`: Show an image `renpy show ${1:image_name} at ${2:left} with ${3:dissolve}` - `scene`: Display a scene `renpy scene ${1:bg_name} with ${2:fade}` - `hide`: Hide an image `renpy hide ${1:image_name} with ${2:dissolve}`

Audio Snippets: - `play music`: Play music `renpy play music "${1:audio/music/theme.mp3}" fadein ${2:1.0}` - `play sound`: Play sound effect `renpy play sound "${1:audio/sfx/doorbell.wav}"` - `stop music`: Stop music `renpy stop music fadeout ${1:1.0}`

Control Flow Snippets: - `jump`: Jump to a label `renpy jump ${1:label_name}` - `call`: Call a label `renpy call ${1:label_name}` - `return`: Return from a call `renpy return`

Variable Snippets: - `define`: Define a constant `renpy define ${1:variable_name} = ${2:value}` - `default`: Define a variable with default value `renpy default ${1:variable_name} = ${2:value}` - `$`: Python statement `renpy $ ${1:variable_name} = ${2:value}`

And many more...

10.3.2 7.3.2 Using Snippets

Trigger a Snippet: 1. Type the snippet prefix (e.g., `label`, `menu`, `show`) 2. Press `Tab` to expand 3. The snippet inserts with placeholders highlighted 4. Type to replace the first placeholder 5. Press `Tab` to jump to the next placeholder 6. Repeat until all placeholders are filled

Example: 1. Type `label` and press `Tab` 2. The snippet inserts: `renpy label |: # your content return` (where `|` is the cursor) 3. Type `chapter1` (replaces `label_name`) 4. Press `Tab` 5. The cursor moves to `# your content` 6. Type your label content

IntelliSense Integration: Snippets appear in IntelliSense suggestions. Start typing a snippet prefix, and it appears in the autocomplete list.

10.3.3 7.3.3 Creating Custom Snippets

Create project-specific or personal snippets for boilerplate code.

To Create a Snippet: 1. Go to the **Snippets tab** in the Story Elements panel 2. Click **"+ New Snippet"** 3. Fill in: - **Trigger Prefix:** The text you type to trigger the snippet (e.g., `mychar`, `mytemplate`) - **Snippet Content:** The template code with placeholders

Placeholder Syntax: - `${1:placeholder_text}`: First placeholder (cursor starts here after expansion) - `${2:placeholder_text}`: Second placeholder (cursor moves here after Tab) - `$0`: Final cursor position (after all placeholders are filled)

Example Custom Snippet:

Trigger: `mychar`

Content:

```
define ${1:tag} = Character("${2:Display Name}", color="${3:#ffffff}", image="${4:sprite_name}"
$0
```

Usage: 1. Type `mychar` and press `Tab` 2. Type the character tag (e.g., `e`) 3. Press `Tab`, type the display name (e.g., `Eileen`) 4. Press `Tab`, enter the color (e.g., `#c8ffc8`) 5. Press `Tab`, enter the sprite name (e.g., `eileen`) 6. Press `Tab`, cursor moves to the end

Result:

```
define e = Character("Eileen", color="#c8ffc8", image="eileen")
|
```

10.3.4 7.3.4 Snippet Management

Edit a Snippet: 1. Click the snippet in the Snippets tab 2. Modify the trigger prefix or content 3. Click "Save"

Delete a Snippet: 1. Right-click the snippet 2. Select "Delete" 3. Confirm

Export/Import Snippets (If Supported): - Export your snippet library to share with teammates - Import snippet libraries from others

---

10.4 7.4 Screen Layout Composer

The Screen Layout Composer is a visual tool for building Ren'Py `screen` definitions—perfect for creating custom UIs, menus, HUDs, and more.

10.4.1 7.4.1 Overview

What is a Ren'Py Screen? A `screen` is Ren'Py's way of defining user interfaces. Screens are built from widgets (containers, text, images, buttons, input fields) arranged in a hierarchy.

What is the Screen Layout Composer? A drag-and-drop interface for creating screens visually. Instead of writing screen code manually, you arrange widgets on a stage and configure their properties. The composer generates the Ren'Py `screen` code for you.

Opening the Screen Layout Composer: 1. Go to the **Composers tab** 2. Click **"+ New Screen"** 3. The composer opens

10.4.2 7.4.2 Available Widgets

Containers (for layout): - **vbox:** Vertical box (stacks children vertically) - **hbox:** Horizontal box (arranges children horizontally) - **frame:** A bordered container with optional background - **null:** Invisible spacer (for layout control)

Content: - **text:** Display text - **image:** Display an image

Buttons: - **textbutton:** Button with text label - **button:** Generic button (can contain other widgets) - **imagebutton:** Button with image (idle and hover states)

Input: - **input:** Text input field - **bar:** Progress bar or slider

Others: - **viewport:** Scrollable container - **label:** Text with specific styling (less common) - **timer:** For timed events (advanced)

10.4.3 7.4.3 Building a Screen

Adding Widgets: 1. In the widget palette (side panel), select a widget type (e.g., `vbox`) 2. Click or drag onto the stage 3. The widget appears on the stage

Nesting Widgets: - Drag a widget onto a container (vbox, hbox, frame) to nest it - The hierarchy is shown in the widget tree (side panel) - Example: `vbox  text "Title"  hbox  textbutton "Start Game"  textbutton "Quit"  image "logo.png"`

Configuring Widget Properties: Select a widget to see its properties in the property panel: - **Common properties (all widgets):** - **id:** Unique identifier (optional) - **xalign, yalign:** Alignment (0.0 = left/top, 0.5 = center, 1.0 = right/bottom) - **xpos, ypos:** Absolute position (pixels or percentage) - **xsize, ysize:** Size (pixels, percentage, or None for auto) - **padding:** Padding around the widget

- **Text-specific properties:**
  - **text:** The text content
  - **size:** Font size
  - **color:** Text color (hex code or name)
  - **bold, italic:** Style toggles
- **Image-specific properties:**
  - **image:** File path or image name
  - **xsize, ysize:** Scale the image
- **Button-specific properties:**
  - **action:** What happens when clicked (e.g., `Start()`; `Jump("label")`, `Return()`)
  - **idle_image, hover_image:** For imagebutton
- **Container-specific properties:**
  - **spacing:** Space between children (for vbox/hbox)
  - **xfill, yfill:** Whether container fills available space

10.4.4 7.4.4 Asset Integration

Drag Images from Assets Panel: 1. Open the Images tab 2. Drag an image onto the stage 3. An `image` widget is created automatically

Or Drag onto an Existing Widget: - Drag an image onto an `imagebutton` widget to set its idle or hover image

Drag onto Image Widget: - Replaces the widget's image

10.4.5 7.4.5 Code Generation

Once your screen layout is complete:

Generate Code: 1. Click **"Generate Code"** 2. Ren'IDE generates the `screen` definition: "'renpy screen main_menu: vbox: xalign 0.5 yalign 0.5 spacing 20

```
    text "My Visual Novel" size 48 color "#ffffff"

    hbox:
        spacing 10

        textbutton "Start Game":
            action Start()

        textbutton "Quit":
            action Quit(confirm=True)
```

```
       image "logo.png" xsize 200
3. The code is copied to clipboard

**Paste into Script:**
- Open `screens.rpy` (or create a new screen file)
- Paste the generated code
- Call the screen with `call screen main_menu` or show it with `show screen main_menu`

### 7.4.6 Locked-Screen Workflow

Ren'IDE can parse existing screens from your project and display them in the composer in **read

**Viewing an Existing Screen:**
1. Go to the Screens tab in Story Elements
2. Click a screen in the list
3. The Screen Layout Composer opens with the screen loaded
4. The interface is locked (you can't edit it)

**Why Locked?**
Existing screens may have complex logic or custom code that the composer doesn't fully support

**To Edit:**
1. Click **"Duplicate to Edit"**
2. A copy of the screen is created in the composer
3. Edit the copy as desired
4. Generate code for the new version
5. Manually replace or integrate the code into your project

### 7.4.7 Managing Screen Compositions

**Save Composition:**
- Click "Save Composition"
- Enter a name (e.g., "Main Menu Screen")
- The composition is saved in `project.ide.json`

**Load Composition:**
- Go to the Composers tab
- Click a saved screen composition
- It opens in the Screen Layout Composer
```

Delete Composition:
- Right-click and select "Delete"

7.5 Diagnostics Panel for Developers

The Diagnostics panel is a quality assurance tool. For developers, it's invaluable for catchin

7.5.1 Error Detection

Parse Errors:
- Code that doesn't follow Ren'Py syntax
- Example: Missing colon after `label`, incorrect indentation, unclosed quotes
- Severity: **Error** (red)

Invalid Jumps:
- `jump` or `call` to a label that doesn't exist
- Example: `jump chapter2` when `chapter2` is never defined
- Severity: **Error** (red)

Missing Assets:
- References to images or audio files not found in the project
- Example: `show eileen happy` when `eileen_happy.png` doesn't exist in `game/images/`
- Severity: **Error** (red)

Undefined Characters:
- Using a character tag in dialogue before defining it
- Example: `e "Hello"` when `define e = Character(...)` doesn't exist
- Severity: **Error** (red)

Undefined Screens:
- Calling or showing a screen that hasn't been defined
- Example: `call screen settings` when `screen settings:` doesn't exist
- Severity: **Error** (red)

7.5.2 Warning Categories

Unused Characters:

- Characters defined but never used in dialogue
- Possible causes: Leftover from cut content, placeholder, typo in tag
- Severity: **Warning** (amber)

Unreachable Labels:
- Labels that no path from `start` leads to
- Possible causes: Dead code, orphaned content, intentional (e.g., bonus content)
- Severity: **Warning** (amber)

Missing Variables (Conditional Checks):
- Variables used in `if` statements but never defined
- Example: `if player_intelligence >= 5:` when `player_intelligence` is never defined
- Severity: **Warning** (amber)

7.5.3 Code Quality Checks

Info-Level Diagnostics:
- Suggestions for best practices
- Example: "Consider using `call` instead of `jump` for subscenes"
- Severity: **Info** (blue)

Complexity Metrics:
- Labels with very high line counts (might need refactoring)
- Menus with too many choices (player overwhelm risk)
- Deep nesting (code readability issue)

7.5.4 Jumping to Source

Click any diagnostic to jump to the source:
1. The file opens in the code editor
2. The cursor positions on the line with the issue
3. The problematic code is highlighted

Why This Matters:
- No manual searching through files
- Immediate context for fixing the issue

7.5.5 Task Tracking

Convert Diagnostics to Tasks:

1. Right-click a diagnostic
2. Select "Convert to Task"
3. The issue is added to a task list

Task List:
- Shown in the Diagnostics panel under a "Tasks" tab
- Track progress as you fix issues
- Mark tasks as complete when resolved

Use Cases:
- Large backlog of warnings (fix incrementally)
- Team workflow (assign tasks to team members)
- Prioritization (tackle errors first, then warnings)

7.5.6 Continuous Analysis

Diagnostics update in real-time as you code:
- Add a label → diagnostics for "invalid jump" are resolved
- Reference an image → diagnostics for "missing image" trigger
- Fix a syntax error → diagnostic disappears

Why This Matters:
- Immediate feedback loop
- Catch mistakes as you make them, not during testing

7.6 Project Statistics for Developers

The Stats panel provides metrics useful for developers beyond word counts.

7.6.1 Code Metrics

Lines of Code:
- Total lines across all `.rpy` files
- Excludes comments and blank lines (if configured)

Lines of Dialogue:
- Total dialogue lines (character speech)
- Useful for estimating localization effort

```
**Label Count:**
- Total number of labels in the project

**Scene/Route Count:**
- Number of distinct scenes (depending on how you define scenes)
- Number of narrative routes (paths from `start` to an ending)

**Jump/Call Count:**
- Total `jump` and `call` statements
- Indicates branching complexity

### 7.6.2 Complexity Analysis

**Branching Complexity:**
- Metrics like:
- Average choices per menu
- Maximum path depth (number of decisions from `start` to an ending)
- Cyclomatic complexity (number of linearly independent paths)
- **Why It Matters:** High complexity = more testing required, higher QA burden

**Path Depth Statistics:**
- Minimum, maximum, and average path depth
- Identifies very long paths (potential pacing issues) or very short paths (incomplete content

**Unreachable Code Ratio:**
- Percentage of labels that are unreachable
- Ideal: 0% (all code is reachable)

### 7.6.3 Asset Coverage

**Images:**
- **Defined:** Number of images in `game/images/`
- **Used:** Number of images referenced in code
- **Unused:** Defined but not used
- **Missing:** Used but not defined (errors)

**Audio:**
- Same breakdown as images
```

Why It Matters:
- **Unused assets:** Clutter, increases distribution size
- **Missing assets:** Errors that need fixing

7.6.4 Exporting Stats

Export as Report:
1. Click "Export" or "Save Report" in the Stats panel
2. Choose format:
- **CSV:** For spreadsheet analysis
- **JSON:** For programmatic processing
- **PDF:** For documentation or sharing with non-technical stakeholders
3. Save to disk

Use Cases:
- Tracking project growth over time
- Reporting to stakeholders
- Identifying trends (e.g., complexity increasing faster than content)

7.7 Ren'Py SDK Integration

Ren'IDE works alongside the Ren'Py SDK. This section covers integration details beyond basic se

7.7.1 Configuring SDK Path (Review)

(See Section 2.5 for initial setup.)

Developer-Specific Notes:
- SDK path is stored in global settings (affects all projects)
- You can override the SDK path per project (if supported) for testing multiple Ren'Py versions

7.7.2 Running the Game

Run Button:
- Click the **Run** button in the toolbar
- Ren'IDE:
1. Saves all unsaved files
2. Invokes the Ren'Py SDK with your project path

3. Opens the game window
4. Waits for the game to close

Console Output:
- If the SDK outputs console messages, they may appear in:
- A terminal window (Linux)
- The Ren'Py log file (Windows/macOS)
- An output panel in Ren'IDE (if supported)

Error Handling:
- If the SDK fails to launch, Ren'IDE shows an error message
- Common causes: SDK path incorrect, SDK version incompatible, project structure invalid

7.7.3 Build and Distribution

Using Ren'Py SDK Tools:
While Ren'IDE is for development, the Ren'Py SDK handles building distributable packages:
1. Open the Ren'Py launcher (separately from Ren'IDE)
2. Select your project
3. Click "Build Distributions"
4. Choose target platforms (Windows, macOS, Linux, Android, iOS, Web)
5. SDK builds the packages

Workflow:
- Develop in Ren'IDE
- Test with the Run button
- Build distributions with the SDK
- Distribute to players

7.7.4 SDK Version Compatibility

Recommended Version:
Ren'Py 7.5 or later

Older Versions:
- Ren'Py 7.0-7.4: May work but some features might not be fully supported
- Ren'Py 6.x: Not recommended (missing modern features)

Newer Versions:
- Ren'Py 8.x: Fully supported

```
**Checking Compatibility:**
- Test your project with the SDK version you'll use for distribution
- Run the game from Ren'IDE to ensure compatibility

---

## 7.8 File System Integration

Ren'IDE uses modern file system APIs for direct access to your project files.

### 7.8.1 File System Access API

**What is It?**
A web standard API for reading and writing local files from a web app (or Electron app). Ren'I

**Permissions:**
- On first project open, the OS may prompt you to grant access
- Once granted, Ren'IDE can read and write files in that directory
- Permissions persist (you don't need to grant access every time)

### 7.8.2 Project Structure

**Expected Structure:**
```

your-project/ game/ *.rpy files images/ audio/ .ide/ (created by Ren'IDE)
project.ide.json (created by Ren'IDE)

```
**game/ Directory:**
- Contains all `.rpy` files and assets
- Required by Ren'Py

**.ide/ Directory:**
- Created by Ren'IDE
- Contains cached analysis results and temporary files
- Can be excluded from version control (.gitignore)

**project.ide.json:**
- Stores Ren'IDE-specific settings:
  - Canvas block positions
```

```
  - Saved compositions (Scene, ImageMap, Screen)
  - Image and audio metadata
  - Project-specific settings (active canvas, open tabs, etc.)
- Can be committed to version control if you want teammates to share layouts

### 7.8.3 File Operations

**Creating Files:**
- Use the Project Explorer to create `.rpy` files
- Or use the "Add Block" button to create files on the canvas
- Files are written to disk immediately

**Renaming Files:**
- Right-click a file in the Project Explorer, select "Rename"
- The file is renamed on disk
- **Note:** Renaming a file doesn't update jumps/calls that reference labels in that file (use

**Deleting Files:**
- Right-click a file, select "Delete"
- The file is moved to the OS trash/recycle bin (not permanently deleted)
- **Warning:** Deletion is permanent from Ren'IDE's perspective (no undo via Ren'IDE's undo sys

**Moving Files:**
- Drag-and-drop in the Project Explorer
- Or cut/paste to move files between folders

### 7.8.4 File System Watches (If Supported)

Some versions of Ren'IDE watch the file system for external changes:
- If you edit a file in another editor, Ren'IDE detects the change and prompts you to reload
- If files are added/deleted externally, the Project Explorer updates

**Developer Workflow:**
- Edit code in Ren'IDE
- Use external tools (Git, image editors, etc.) as needed
- Ren'IDE stays in sync

---

## 7.9 Developer Workflow Tips
```

```
Here are best practices for developers using Ren'IDE.

### 7.9.1 Organizing Code by Module

**Modular Structure:**
- Break your project into logical modules (files or folders)
- Example:
```

game/ characters.rpy # Character definitions variables.rpy # Global variables script.rpy # Main story chapters/ chapter1.rpy chapter2.rpy chapter3.rpy endings/ ending_good.rpy ending_bad.rpy screens/ main_menu.rpy preferences.rpy “‘

Benefits: - Easy to find specific code - Reduces merge conflicts (team members work in different files) - Clear separation of concerns

10.4.6 7.9.2 Using Labels Effectively

Label Naming Conventions: - Use descriptive names: `chapter1_intro`, `flashback_childhood`, `ending_true` - Avoid generic names: `scene1`, `label2` - Use prefixes for grouping: `ch1_`, `ch2_`, `ending_`

Label Organization: - One label per major scene or subscene - Use `call` for reusable subscenes (flashbacks, recurring events) - Use `jump` for permanent transitions (chapter endings, epilogues)

10.4.7 7.9.3 Leveraging Diagnostics for QA

Pre-Commit Checklist: 1. Open the Diagnostics panel 2. Fix all errors (red) 3. Review warnings (amber): - Unreachable labels: Intentional or bug? - Unused characters: Remove or add dialogue? 4. Optional: Address info-level suggestions

Continuous Integration (CI): If you have a CI pipeline: - Export diagnostics as JSON - Parse in CI to fail builds on errors - Automate QA checks

10.4.8 7.9.4 Version Control Integration (Git)

Committing Changes: - Commit `.rpy` files to track code changes - Commit `project.ide.json` if you want to share canvas layouts and compositions - Exclude `.ide/` directory (add to `.gitignore`)

Branching: - Use branches for features or experiments - Ren’IDE works with any branch (file system state reflects the current branch)

Merge Conflicts: - If `.rpy` files conflict, resolve manually - If `project.ide.json` conflicts, prefer one version or merge carefully (JSON structure must remain valid)

10.4.9 7.9.5 Collaborative Development

Team Roles: - **Writers:** Focus on dialogue, narrative flow - **Artists:** Create and manage assets - **Developers:** Write systems, screens, technical code

Communication: - Use the Story Canvas in meetings to discuss structure - Use sticky notes on the canvas to leave messages - Use the Diagnostics panel to track issues

File Ownership: - Assign files to team members to reduce conflicts - Example: Writer owns `script.rpy`, Developer owns `screens.rpy`

10.4.10 7.9.6 Testing Strategies

Manual Testing: - Run the game frequently from Ren'IDE - Test all branches and endings - Use the Route Canvas to identify paths to test

Automated Testing (Advanced): - Write Ren'Py unit tests (if supported by your SDK version) - Integrate with a testing framework

Playtesting: - Use the Choice Canvas to trace the player experience - Ensure all choices make sense from the player's perspective

10.4.11 7.9.7 Performance Optimization

Large Projects: - If canvas rendering is slow (100+ blocks): - Use grouping and filtering to reduce visible blocks - Close unnecessary editor tabs - Collapse unused folders in the Project Explorer

Asset Optimization: - Compress images and audio before importing - Use appropriate formats (PNG for sprites, JPEG for backgrounds, OGG for audio)

Code Optimization: - Avoid deeply nested labels (refactor into separate files) - Use `call` for subscenes to improve code organization

---

End of Section 7: For Developers

Chapter 11

Section 8: Complete Feature Reference

This section provides a comprehensive quick-reference guide to all Ren'IDE features. Use this as a lookup reference when you need specific information about a feature.

11.1 8.1 Canvas Features

11.1.1 Story Canvas

Feature	Description	Key Actions
File-Level Blocks	Each `.rpy` file is a draggable block	Click to select, drag to move, double-click to open in editor
Jump/Call Arrows	Solid arrows for `jump`, dashed for `call`	Auto-drawn based on code analysis
Diagnostic Glow	Red glow for errors, amber for warnings	Visible even when zoomed out
Role Tinting	Blocks colored by character presence	Enable in toolbox, select character
Character Filter	Show only blocks involving specific character	Dropdown in toolbox
Grouping Modes	Group by connected components or file prefix	Dropdown in toolbox, requires Clustered Flow layout
Layout Modes	Flow, Hierarchical, Circular, Clustered Flow	Dropdown in toolbox
Auto-Layout (Redraw)	Automatically arrange blocks	Toolbar button, overwrites manual positioning

Feature	Description	Key Actions
Minimap	Overview of entire canvas	Click to navigate
Go-to-Label Search	Filter and jump to labels	Toolbox search box
Ctrl+G Palette	Quick label navigation with fuzzy search	`Ctrl+G` / `Cmd+G` from any canvas
Fit-to-Screen	Zoom to show all blocks	Button in canvas controls (bottom-right)
Go-to-Start	Center on `start` label	Button in canvas controls
Zoom/Pan	Mouse scroll to zoom, Shift+Drag to pan	Configurable in Settings
Multi-Select	Select multiple blocks	`Ctrl+Click` or rubber-band drag
Sticky Notes	Canvas annotations	Toolbar "Add Note" button
Canvas Legend	Explains arrow types and colors	Button in toolbox (if available)

11.1.2 Route Canvas

Feature	Description	Key Actions
Label-Level Nodes	Each label is a node	Click to select
Jump/Call/Fall-Through Edges	Solid (jump), dashed (call), dotted (fall-through)	Auto-drawn
Unreachable Label Detection	Labels with no path from `start` highlighted	Red/warning color
Menu Decision Inspector	Hover over menu nodes to see choices	Hover popover shows all choices and destinations
Route Highlighting	Highlight specific narrative paths	Route List panel, click routes to highlight
Route List Panel	List of all discovered routes	Collapsible left sidebar, hover to expand
Node Roles	Start (green), End (red), Choice (blue), etc.	Color/icon indicates role
Graph Layout Algorithm	BFS layered or force-directed	Auto-computed
Go-to-Label Features	Same as Story Canvas	Ctrl+G, toolbox search
Fit/Go-to-Start	Same as Story Canvas	Canvas controls

11.1.3 Choice Canvas

Feature	Description	Key Actions
Menu Nodes	Each menu is a node	Shows menu prompt
Choice Pills	Colored rectangles for each choice	Shows choice text
Conditional Badges	Display `if` guards on choices	Badge shows condition
Destination Connections	Pills connect to destination labels	Color-coded
Go-to-Label Features	Same as Story/Route Canvas	Ctrl+G, toolbox search

---

11.2 8.2 Code Editor Features

11.2.1 Monaco Editor Core

Feature	Description	Shortcut
Syntax Highlighting	TextMate grammar + semantic tokens	Auto
IntelliSense	Context-aware autocomplete	`Ctrl+Space` or auto-trigger
Code Folding	Collapse/expand code blocks	Click gutter arrows
Line Numbers	Display line numbers	Always on
Multi-Cursor	Edit multiple locations simultaneously	`Ctrl+Click` or `Ctrl+D`
Column Selection	Rectangular selection	`Alt+Drag`
Find	Search within file	`Ctrl+F`
Replace	Find and replace	`Ctrl+H`
Regex Find/Replace	Use regular expressions	Enable `.*` icon in find bar
Move Line Up/Down	Reorder lines	`Alt+Up` / `Alt+Down`
Delete Line	Remove current line	`Ctrl+Shift+K`
Toggle Comment	Comment/uncomment line	`Ctrl+/`
Bracket Matching	Highlight matching brackets	Auto
Auto-Indentation	Indent after `:`	Auto on Enter

11.2.2 IntelliSense Suggestions

Context	Suggestions	Example
After `jump`	All label names	`jump chapter1`
After `call`	All label names	`call flashback`

Context	Suggestions	Example
After `show`	Image names	`show eileen happy`
After `scene`	Image names	`scene bg office`
Character tags	Defined characters	`e "Hello"`
Screen names	Defined screens	`call screen main_menu`
Variables	Global variables	`if player_karma > 0:`

11.2.3 Snippets (28+ Built-in)

Trigger	Expands To	Use Case
`label`	`label name:\n  # content\n return`	Create new label
`menu`	`menu:\n  "prompt"\n "choice1":\n  ...\n "choice2":\n  ...`	Create menu
`jump`	`jump label_name`	Jump statement
`call`	`call label_name`	Call statement
`show`	`show image at left with dissolve`	Show image
`scene`	`scene bg with fade`	Display scene
`play music`	`play music "path" fadein 1.0`	Play music
`play sound`	`play sound "path"`	Play sound
`if`	`if condition:\n  # code\nelse:\n  # code`	If/else block
`define`	`define var = value`	Define constant
`default`	`default var = value`	Define variable
`$`	`$ var = value`	Python statement

11.3 8.3 Story Elements Panel Tabs

11.3.1 Characters Tab

- **Displays:** All `define Character(...)` definitions
- **Info Shown:** Tag, display name, color, dialogue count
- **Actions:**
  - Add new character
  - Edit character (opens Character Editor)

  - Find usages
  - Jump to definition
- **Character Editor Fields:** Tag, name, color, Ren'Py parameters, profile notes

11.3.2 Variables Tab

- **Displays:** All `define`/`default` global variables
- **Info Shown:** Variable name, type (define/default), initial value
- **Actions:**
  - Find usages
  - Jump to definition

11.3.3 Images Tab (Image Asset Manager)

- **Displays:** All images in `game/images/` and scanned directories
- **Organization:** Folder tree with thumbnails
- **Actions:**
  - Scan external directory
  - Import images (drag-and-drop or button)
  - Edit metadata (double-click)
  - Copy `scene` statement (right-click)
  - Copy `show` statement (right-click)
  - Drag to Scene Composer or Screen Composer
- **Metadata Fields:** Ren'Py name, tags, subfolder

11.3.4 Snd Tab (Audio Asset Manager)

- **Displays:** All audio in `game/audio/` and scanned directories
- **Organization:** Folder tree
- **Actions:**
  - Scan external directory
  - Import audio
  - Play in built-in player (play/pause, seek, volume, loop)
  - Edit metadata
  - Copy `play music` statement
  - Copy `play sound` statement
  - Copy `queue audio` statement
- **Metadata Fields:** Ren'Py name, tags, subfolder

11.3.5 Screens Tab

- **Displays:** All `screen` definitions

- **Info Shown:** Screen name, file location, parameters
- **Actions:**
  - Jump to definition
  - Add screen with boilerplate
  - View in Screen Layout Composer (read-only)

11.3.6 Composers Tab

- **Displays:** Saved compositions (Scene, ImageMap, Screen)
- **Actions:**
  - Load composition (click to open in respective composer)
  - Delete composition
  - Create new composition (+ New Scene/ImageMap/Screen buttons)

11.3.7 Menus Tab (Visual Menu Builder)

- **Displays:** All menus in project
- **Actions:**
  - Create new menu
  - Edit existing menu
  - Add/configure choices (text, destination, condition, custom code)
  - Generate menu code

11.3.8 Snippets Tab

- **Displays:** Built-in and user-defined snippets
- **Actions:**
  - Add new snippet (trigger prefix, content)
  - Edit snippet
  - Delete snippet
  - Export/import snippet library (if supported)

11.4 8.4 Visual Composers

11.4.1 Scene Composer

Purpose: Compose scenes visually (backgrounds, sprites, overlays)

Features: - Drag images from asset panel onto stage - Configurable stage resolution (presets: 1920×1080, 1280×720, 1024×768, 800×600, custom) - Per-sprite transforms: - Position (drag or X/Y input) - Scale/zoom (slider) - Flip horizontal/vertical (checkbox) - Rotate (slider, degrees)

- Alpha/opacity (slider, 0.0-1.0) - Blur (slider) - Layer reordering (drag in layer list, Bring Forward/Send Backward) - Code generation (generates `scene` and `show` statements with transforms) - Export as PNG (flat image for promotion/reference) - Save/load compositions

11.4.2 ImageMap Composer

Purpose: Create clickable imagemaps (hotspots on images)

Features: - Set ground image and hover overlay (drag from asset panel) - Draw rectangular hotspots (click and drag on image) - Resize and move hotspots - Configure per-hotspot: - Hotspot name - Action type (jump or call) - Target label (dropdown) - Condition (optional `if` statement) - Code generation (generates `imagemap` or `imagebutton` screen code) - Save/load imagemap compositions

11.4.3 Screen Layout Composer

Purpose: Build Ren'Py screens visually

Available Widgets: - **Containers:** vbox, hbox, frame, null - **Content:** text, image - **Buttons:** textbutton, button, imagebutton - **Input:** input, bar

Features: - Drag widgets onto stage - Nest widgets (drag onto containers) - Configure widget properties (side panel): - Common: id, xalign, yalign, xpos, ypos, xsize, ysize, padding - Text-specific: text content, size, color, bold, italic - Image-specific: image path, xsize, ysize - Button-specific: action (Start, Jump, Return, etc.), idle_image, hover_image - Container-specific: spacing, xfill, yfill - Drag images from asset panel to set image widgets or imagebutton images - Code generation (generates `screen` definition) - Locked-screen workflow (view existing screens read-only, duplicate to edit) - Save/load screen compositions

---

11.5 8.5 Diagnostics

11.5.1 Error Types (Red)

Type	Description	Example
Invalid Jump	`jump`/`call` to nonexistent label	`jump chapter2` when `chapter2` label doesn't exist
Syntax Error	Code that can't be parsed	Missing colon, unclosed quote, incorrect indentation
Missing Image	Image referenced but not found	`show eileen happy` when `eileen_happy.png` missing

Type	Description	Example
Missing Audio	Audio referenced but not found	`play music "theme.mp3"` when file missing
Undefined Character	Character tag used but not defined	`e "Hello"` when `define e = Character(...)` missing
Undefined Screen	Screen called but not defined	`call screen settings` when `screen settings:` missing

11.5.2 Warning Types (Amber)

Type	Description	Possible Causes
Unused Character	Character defined but never speaks	Leftover from cut content, placeholder, typo
Unreachable Label	Label no path from `start` leads to	Dead code, orphaned content, intentional bonus
Missing Variable	Variable used but never defined	Typo, forgot to define, conditional logic error

11.5.3 Info Types (Blue)

- Best practice suggestions
- Code style recommendations
- Complexity metrics

11.5.4 Actions

Action	Description	Shortcut
Click to Jump	Opens file at line with issue	Click diagnostic entry
Filter by Severity	Show/hide errors, warnings, info	Checkboxes at top
Convert to Task	Add diagnostic to task list	Right-click → Convert to Task
View Tasks	See task list	Tasks tab in Diagnostics panel

11.6 8.6 Project Statistics

11.6.1 Metrics

Metric	Description	Use Case
Word Count	Total words in dialogue/narration	Estimate project size
Estimated Play Time	Based on reading speed (e.g., 200 wpm)	Marketing, scope estimation
Lines of Dialogue	Total dialogue lines by characters	QA, localization estimate
Per-Character Dialogue	Bar chart of dialogue counts	Balance character screen time
Scene Count	Number of distinct scenes	Project organization
Label Count	Total labels in project	Complexity indicator
Route Count	Number of narrative paths	Branching complexity
Branching Complexity	Average choices/menu, max path depth, cyclomatic complexity	QA burden, testing effort
Asset Coverage (Images)	Defined vs. used vs. missing	Identify unused/missing assets
Asset Coverage (Audio)	Defined vs. used vs. missing	Identify unused/missing assets

11.6.2 Export Formats

- **CSV:** For spreadsheet analysis
- **JSON:** For programmatic processing
- **PDF:** For documentation/reporting (if supported)

11.7 8.7 Project Explorer

11.7.1 File Operations

Operation	Action	Shortcut
Open File	Double-click file	N/A
Create File	Right-click folder → New File	N/A
Create Folder	Right-click folder → New Folder	N/A
Rename	Right-click → Rename	`F2`
Delete	Right-click → Delete	`Delete`
Cut	Right-click → Cut	`Ctrl+X` / `Cmd+X`
Copy	Right-click → Copy	`Ctrl+C` / `Cmd+C`

Operation	Action	Shortcut
Paste	Right-click destination → Paste	`Ctrl+V` / `Cmd+V`
Drag-and-Drop	Drag file/folder to new location	N/A
Center on Canvas	Right-click `.rpy` file → Center on Canvas	N/A

11.7.2 Context Menu Actions by File Type

File Type	Additional Actions
.rpy	Center on Canvas
Images	Copy `scene` statement, Copy `show` statement
Audio	Copy `play music`, Copy `play sound`, Copy `queue audio`

11.8 8.8 Search & Replace

11.8.1 Features

Feature	Description	Shortcut
Full-Text Search	Search across all project files	`Ctrl+Shift+F` / `Cmd+Shift+F`
Regex Support	Use regular expressions	Toggle `.*` icon
Match Case	Case-sensitive search	Toggle `Aa` icon
Whole Word	Match whole words only	Toggle `ab|` icon
Replace	Replace individual matches	Click "Replace"
Replace All in File	Replace all matches in one file	Click "Replace All in File"
Replace All	Replace all matches in all files	Click "Replace All" (with confirmation)
Search History	Access previous searches	Dropdown in search box

---

11.9 8.9 Settings

11.9.1 General Settings

Setting	Description	Options
Theme	UI theme	10 themes (see Section 3.8)
Auto-Save Interval	How often to auto-save (if supported)	Minutes or disabled
Undo History Depth	Number of actions to remember	50 (default), customizable

11.9.2 Editor Settings

Setting	Description	Options
Font Size	Code editor font size	pt (e.g., 14pt)
Tab Size	Spaces per tab	2, 4 (default for Ren'Py), 8
Word Wrap	Wrap long lines	On/Off
Minimap Visibility	Show minimap in editor	On/Off

11.9.3 Canvas Settings

Setting	Description	Options
Pan Mode	How to pan canvas	Shift+Drag (default), Middle-Click, etc.
Zoom Sensitivity	Mouse scroll zoom speed	Slider
Auto-Layout Algorithm	Preferred layout algorithm	Flow, Hierarchical, Circular

11.9.4 Ren'Py SDK Settings

Setting	Description	Actions
SDK Path	Path to Ren'Py SDK installation	Browse button, Test SDK Path button

11.9.5 AI Settings

Setting	Description	Options
API Provider	AI service for content generation	Google Gemini, OpenAI, Anthropic
API Key	Encrypted API key	Input field (stored securely)

11.9.6 Mouse Preferences

Setting	Description	Options
Pan Mode	Canvas panning shortcut	Shift+Drag, Alt+Drag, Middle-Click
Zoom Behavior	How zoom focuses	Cursor position, center

11.10 8.10 Undo/Redo System

11.10.1 What Can Be Undone

- Canvas block moves
- Block creation/deletion
- Composition edits (Scene/ImageMap/Screen Composer)
- Code changes (editor)

11.10.2 Shortcuts

Action	Shortcut
Undo	`Ctrl+Z` / `Cmd+Z`
Redo	`Ctrl+Y` / `Cmd+Y` / `Ctrl+Shift+Z`

11.10.3 Limitations

- Does NOT track file system operations (rename, delete via Project Explorer)
- Does NOT track settings changes
- Does NOT track asset imports
- History depth: Last 50 actions (configurable)

11.11 8.11 Drafting Mode

Feature	Description	Use Case
Placeholder Images	Missing images show as gray rectangles with name	Continue development with incomplete assets
Placeholder Audio	Missing audio plays as silence or tone	Test game with incomplete audio

Feature	Description	Use Case
Toggle	Enable/disable in toolbar	Button toggles on/off
Visual Indicator	Toolbar button highlights when active	Know when mode is on

---

11.12 8.12 Keyboard Shortcuts (Complete List)

11.12.1 Global

Shortcut	Action
`Ctrl+S` / `Cmd+S`	Save All
`Ctrl+Z` / `Cmd+Z`	Undo
`Ctrl+Y` / `Cmd+Y` / `Ctrl+Shift+Z`	Redo
`Ctrl+G` / `Cmd+G`	Go to Label (command palette)
`Escape`	Deselect all / Close modal

11.12.2 Canvas

Shortcut	Action
`N`	New Block
`G`	Group selected blocks
`Delete`	Delete selected blocks
`Shift+Drag`	Pan canvas (configurable)
`Mouse Scroll`	Zoom canvas
`Ctrl+Click` / `Cmd+Click`	Multi-select blocks
`Drag`	Rubber-band selection (on empty canvas)

11.12.3 Editor

Shortcut	Action
`Ctrl+F` / `Cmd+F`	Find in file
`Ctrl+H` / `Cmd+H`	Replace in file
`Ctrl+Space`	Trigger IntelliSense
`Ctrl+/` / `Cmd+/`	Toggle line comment
`Ctrl+D` / `Cmd+D`	Add selection to next find match (multi-cursor)

Shortcut	Action
`Alt+Up` / `Alt+Down`	Move line up/down
`Ctrl+Shift+K` / `Cmd+Shift+K`	Delete line
`Alt+Drag`	Column selection

11.12.4 File Explorer

Shortcut	Action
`Ctrl+C` / `Cmd+C`	Copy file
`Ctrl+X` / `Cmd+X`	Cut file
`Ctrl+V` / `Cmd+V`	Paste file
`Delete`	Delete file/folder
`F2`	Rename file/folder

11.12.5 Search Panel

Shortcut	Action
`Ctrl+Shift+F` / `Cmd+Shift+F`	Open search panel

11.13 8.13 Themes (10 Available)

11.13.1 Light Themes

- **System Light:** OS theme (light)
- **Colorful Light:** Vibrant colors on light background
- **Candy Light:** Soft pastels on light
- **Forest Light:** Nature greens/browns on light
- **Solarized Light:** Classic Solarized light

11.13.2 Dark Themes

- **System Dark:** OS theme (dark)
- **Colorful Dark:** Vibrant colors on dark background
- **Neon Dark:** Bright neon on dark
- **Ocean Dark:** Blue/teal tones on dark
- **Solarized Dark:** Classic Solarized dark

How to Change: Settings → General → Theme dropdown

11.14 8.14 Auto-Updater

Feature	Description
Check on Launch	Checks for updates when app starts
Prompt to Install	Asks if you want to download and install new version
Release Notes	Shows changelog for new version
Manual Check	Help → Check for Updates (if available)

11.15 8.15 New Project Wizard

Steps:

1. **Project Name & Location**
   - Enter project name
   - Choose parent directory
   - Project folder is created at `parent/project-name/`
2. **Resolution Presets**
   - Choose display resolution (1920×1080, 1280×720, 1024×768, 800×600, custom)
   - Written to `options.rpy`
3. **Theme & Accent Color**
   - Choose Ren'IDE theme (personal preference)
   - Choose accent color for game UI (written to `options.rpy`)

Generated Structure:

```
project-name/
  game/
      script.rpy
      options.rpy
      gui.rpy
      screens.rpy
      images/
      audio/
  project.ide.json
```

11.16 8.16 Run Game

Feature	Description
Run Button	Toolbar button (play icon)
Process	Saves all files, invokes Ren'Py SDK, opens game window
Console Output	May appear in terminal (Linux) or log file (Windows/macOS)
Error Handling	Shows error if SDK not configured or fails to launch

---

11.17 8.17 Markdown Preview

Feature	Description
Double-Click .md	Opens rendered preview
GitHub-Style Rendering	Supports standard Markdown syntax
Toggle Edit Mode	Switch to Monaco editor for editing
Syntax Highlighting	Code blocks in preview are highlighted

11.18 8.18 AI Story Generator (If Configured)

Feature	Description
Supported Providers	Google Gemini, OpenAI, Anthropic
API Key Management	Encrypted storage via Electron `safeStorage`
Prompt Input	Enter generation prompt
Generated Content	Preview before inserting
Insert into Project	Adds generated text to current file

Setup: Settings → AI → Choose provider, enter API key

End of Section 8: Complete Feature Reference

Chapter 12

Section 9: Appendices

12.1 Appendix A: Keyboard Shortcuts Reference

12.1.1 Complete Shortcut Table

Category	Action	Windows/Linux	macOS
Global			
	Save All	`Ctrl+S`	`Cmd+S`
	Undo	`Ctrl+Z`	`Cmd+Z`
	Redo	`Ctrl+Y` or `Ctrl+Shift+Z`	`Cmd+Y` or `Cmd+Shift+Z`
	Go to Label	`Ctrl+G`	`Cmd+G`
	Deselect All / Close Modal	`Escape`	`Escape`
Canvas			
	New Block	`N`	`N`
	Group Selected Blocks	`G`	`G`
	Delete Selected	`Delete`	`Delete`
	Pan Canvas	`Shift+Drag`	`Shift+Drag`
	Zoom Canvas	`Mouse Scroll`	`Mouse Scroll`
	Multi-Select Blocks	`Ctrl+Click`	`Cmd+Click`
	Rubber-Band Selection	`Drag on empty canvas`	`Drag on empty canvas`
Code Editor			
	Find in File	`Ctrl+F`	`Cmd+F`

Category	Action	Windows/Linux	macOS
	Replace in File	`Ctrl+H`	`Cmd+H`
	Trigger IntelliSense	`Ctrl+Space`	`Ctrl+Space`
	Toggle Line Comment	`Ctrl+/`	`Cmd+/`
	Multi-Cursor (Next Match)	`Ctrl+D`	`Cmd+D`
	Multi-Cursor (Click)	`Ctrl+Click`	`Cmd+Click`
	Column Selection	`Alt+Drag`	`Alt+Drag`
	Move Line Up	`Alt+Up`	`Alt+Up`
	Move Line Down	`Alt+Down`	`Alt+Down`
	Delete Line	`Ctrl+Shift+K`	`Cmd+Shift+K`
	Jump to Matching Bracket	`Ctrl+Shift+\` (if supported)	`Cmd+Shift+\`
Project Explorer			
	Copy File	`Ctrl+C`	`Cmd+C`
	Cut File	`Ctrl+X`	`Cmd+X`
	Paste File	`Ctrl+V`	`Cmd+V`
	Delete File/Folder	`Delete`	`Delete`
	Rename File/Folder	`F2`	`F2`
Search Panel			
	Open Search Panel	`Ctrl+Shift+F`	`Cmd+Shift+F`

12.2 Appendix B: Troubleshooting

12.2.1 Installation Issues

12.2.1.1 Windows: "App cannot be opened" (SmartScreen Warning)

Problem: Windows SmartScreen blocks the installer.

Solution: 1. Click "More info" in the SmartScreen dialog 2. Click "Run anyway"

Why This Happens: Ren'IDE doesn't have an expensive code-signing certificate, so Windows

treats it as an "unrecognized app."

12.2.1.2 macOS: "App is damaged" or "Cannot be opened" (Gatekeeper)

Problem: macOS Gatekeeper blocks the application.

Solution: 1. Right-click (or Control-click) the application in Finder 2. Select "Open" from the context menu 3. Click "Open" in the confirmation dialog 4. Subsequent launches can use double-click

Why This Happens: Apps downloaded from outside the Mac App Store require this first-launch override.

12.2.1.3 Linux: AppImage Not Launching

Problem: AppImage won't run when double-clicked.

Solution: 1. Make it executable: `chmod +x Vangard-RenPy-renide-*.AppImage` 2. Install FUSE if needed: - Ubuntu/Debian: `sudo apt install fuse libfuse2` - Fedora: `sudo dnf install fuse fuse-libs` - Arch: `sudo pacman -S fuse2` 3. Run from terminal to see error messages: `./Vangard-RenPy-renide-*.AppImage`

12.2.2 Performance Issues

12.2.2.1 Large Projects Loading Slowly

Problem: Opening a project with 100+ files takes a long time.

Solutions: - Wait for initial analysis to complete (one-time process) - Close unnecessary editor tabs - Use filtering and grouping on the canvas to reduce rendered blocks - Upgrade RAM (8GB recommended for large projects)

12.2.2.2 Canvas Rendering Lag

Problem: Dragging blocks or zooming is sluggish.

Solutions: - Collapse unused folders in Project Explorer - Use Fit-to-Screen instead of manual panning - Filter canvas by character to reduce visible blocks - Update graphics drivers

12.2.2.3 Memory Usage

Problem: Ren'IDE uses a lot of RAM.

Causes: Electron apps (including Ren'IDE) use more memory than native apps.

Solutions: - Close other applications - Close unused editor tabs and panes - Restart Ren'IDE periodically - Upgrade to 8GB+ RAM

12.2.3 File System Issues

12.2.3.1 "Permission Denied" Errors

Problem: Can't read or write project files.

Solutions: - **macOS:** Grant file system access in System Preferences → Security & Privacy → Files and Folders → Ren'IDE - **Windows:** Run Ren'IDE as administrator (right-click → Run as administrator) - **Linux:** Check file permissions: `chmod -R u+rw /path/to/project`

12.2.3.2 File System Access API Not Supported

Problem: Browser version of Ren'IDE can't access local files.

Solution: Use the Electron desktop app (File System Access API is fully supported in Electron).

12.2.3.3 Lost Project Handle

Problem: Ren'IDE asks you to reopen a project you've opened before.

Causes: Browser cleared site data, or macOS revoked file access.

Solutions: - Reopen the project (file system access will be requested again) - On macOS, grant permanent access in System Preferences

---

12.2.4 Ren'Py SDK Integration

12.2.4.1 "SDK Path Not Found"

Problem: Ren'IDE can't find the Ren'Py SDK.

Solutions: 1. Verify the SDK is installed and extracted 2. Go to Settings → Ren'Py SDK 3. Enter the correct path (the folder containing `renpy.exe`, `renpy.app`, or `renpy.sh`) 4. Click "Test SDK Path" to verify 5. Restart Ren'IDE if needed

12.2.4.2 Game Fails to Launch

Problem: Clicking "Run" doesn't open the game.

Solutions: 1. Verify SDK path is configured and tested 2. Save all unsaved files before running 3. Check the Ren'Py log for errors (in the project folder) 4. Try launching the game manually from the Ren'Py launcher to verify the project is valid

12.2.4.3 SDK Version Compatibility

Problem: Game runs in the SDK but has errors.

Solution: Ren'IDE works best with Ren'Py 7.5+. Update the SDK if you're using an older version. Download from renpy.org.

12.2.5 Asset Management

12.2.5.1 Images Not Appearing

Problem: Images in `game/images/` don't show in the Images tab.

Solutions: - Refresh the asset list (right-click folder → Refresh, if supported) - Check file extensions (Ren'IDE supports PNG, JPG, JPEG, WEBP) - Ensure images are in `game/images/` or a scanned external directory

12.2.5.2 Audio Playback Issues

Problem: Audio won't play in the built-in player.

Solutions: - Check file format (Ren'IDE supports MP3, OGG, WAV, FLAC) - Check file isn't corrupted (try playing in another audio player) - Restart Ren'IDE

12.2.5.3 Scan Directory Not Refreshing

Problem: Changes in external scan directories aren't reflected.

Solutions: - Right-click the scan directory entry and select "Refresh" - Reopen the project - Re-add the scan directory

---

12.2.6 Code Editor Issues

12.2.6.1 IntelliSense Not Working

Problem: Autocomplete suggestions don't appear.

Solutions: - Press `Ctrl+Space` to manually trigger IntelliSense - Ensure the project analysis is complete (check status bar) - Check that the file is a `.rpy` file (IntelliSense only works for Ren'Py files) - Restart Ren'IDE

12.2.6.2 Syntax Highlighting Missing

Problem: Code appears as plain text, no colors.

Solutions: - Ensure the file has a `.rpy` extension - Restart Ren'IDE - Check Settings → Editor for theme/color settings

12.2.6.3 Snippets Not Triggering

Problem: Typing snippet prefixes and pressing Tab doesn't expand.

Solutions: - Ensure you're in a `.rpy` file - Check the Snippets tab to verify the snippet exists - Try manually triggering IntelliSense (`Ctrl+Space`) and selecting the snippet from the list

---

12.2.7 Canvas Issues

12.2.7.1 Blocks Not Rendering

Problem: Canvas is blank or blocks are missing.

Solutions: - Zoom out to see if blocks are offscreen - Click "Fit-to-Screen" to center all blocks - Restart Ren'IDE - Check browser console for errors (if running in browser mode)

12.2.7.2 Arrows Not Drawing Correctly

Problem: Jump/call arrows are missing or incorrectly drawn.

Solutions: - Save the project and reopen (triggers re-analysis) - Click "Redraw" to recompute layout - Check for invalid jumps in the Diagnostics panel (broken jumps may not draw)

12.2.7.3 Minimap Not Updating

Problem: Minimap shows outdated block positions.

Solutions: - Click the minimap to refresh it - Restart Ren'IDE

12.2.7.4 Go-to-Label Not Finding Labels

Problem: Ctrl+G doesn't show a label you know exists.

Solutions: - Ensure the file with the label is saved - Wait for project analysis to complete (check status bar) - Restart Ren'IDE

12.3 Appendix C: Frequently Asked Questions

12.3.1 General

Q: Is Ren'IDE free? A: Yes, Ren'IDE is free and open source.

Q: Does it work with existing Ren'Py projects? A: Yes. Ren'IDE works with any standard Ren'Py project. Your `.rpy` files remain as `.rpy` files.

Q: Will it lock me into a proprietary format? A: No. Ren'IDE uses standard Ren'Py files. You can open your project in any text editor, and you can open projects created in other editors in Ren'IDE.

Q: Can I use it alongside other editors? A: Yes. Edit in Ren'IDE, or edit in VS Code, Sublime, Atom, etc. Ren'IDE detects external changes and prompts you to reload (if file watching is supported).

Q: Does it replace the Ren'Py SDK? A: No. Ren'IDE is a development environment. You still need the Ren'Py SDK to run and build your game.

12.3.2 Features

Q: Can I edit code directly? A: Yes. Ren'IDE includes a full Monaco code editor (the same engine as VS Code) with syntax highlighting, IntelliSense, snippets, and more.

Q: Can I run my game from Ren'IDE? A: Yes. Configure the Ren'Py SDK path in Settings, then click the "Run" button to launch your game.

Q: Does it support version control (Git)? A: Yes. Ren'IDE works with any version control system. Your project files are standard `.rpy` files that can be committed to Git.

Q: Can multiple people work on the same project? A: Yes. Use version control (Git) to share changes. Commit `project.ide.json` if you want teammates to see canvas layouts and compositions.

Q: Does it support Ren'Py plugins/extensions? A: Ren'IDE doesn't load Ren'Py plugins directly, but any plugins in your project will work when you run the game with the SDK.

12.3.3 Compatibility

Q: What Ren'Py versions are supported? A: Ren'Py 7.5 or later is recommended. Ren'Py 8.x is fully supported. Older versions (7.0-7.4) may work but aren't officially supported.

Q: Does it work on Windows 11? A: Yes.

Q: Does it work on Apple Silicon (M1/M2/M3) Macs? A: Yes. The macOS build is a universal binary that runs natively on both Intel and Apple Silicon.

Q: What Linux distributions are supported? A: Any recent distribution that supports AppImages (Ubuntu 20.04+, Fedora 35+, etc.). FUSE is required for AppImages.

12.3.4 Assets

Q: What image formats are supported? A: PNG, JPG/JPEG, WEBP (and any format supported by web browsers).

Q: What audio formats are supported? A: MP3, OGG, WAV, FLAC, and any format supported by web audio.

Q: Can I use external asset libraries? A: Yes. Use the "Scan External Directory" feature to reference assets without copying them into your project.

Q: How do I organize assets? A: Use subfolders (e.g., `game/images/characters/`, `game/images/backgrounds/`) and tags (custom metadata) for categorization.

12.3.5 Advanced

Q: Can I customize keyboard shortcuts? A: Some versions support shortcut customization in Settings → Keyboard Shortcuts. Check your version's documentation.

Q: Can I create custom themes? A: Ren'IDE includes 10 themes. Custom theme creation isn't currently supported, but you can request it as a feature.

Q: Can I extend Ren'IDE with plugins? A: Ren'IDE doesn't currently have a plugin API, but it's open source, so you can contribute features.

Q: Is there an API for automation? A: Not currently, but you can script file operations (e.g., batch renaming) using standard command-line tools since Ren'IDE uses standard file formats.

12.4 Appendix D: Glossary of Terms

AppImage: A self-contained Linux executable format that runs on most distributions without installation.

Block: A draggable node on the Story Canvas representing a `.rpy` file.

Canvas: A visual workspace (Story Canvas, Route Canvas, or Choice Canvas) for viewing project structure.

Call: A Ren'Py statement (`call label_name`) that transfers control to a label and returns when the label ends.

Character: A Ren'Py construct defined with `define tag = Character("Name", ...)` representing a speaking character.

Choice Canvas: Canvas view showing player-facing menu choices and their destinations.

Composition: A saved layout from the Scene Composer, ImageMap Composer, or Screen Layout Composer.

Diagnostics: Errors, warnings, and info messages detected by analyzing your project code.

DMG: Disk image format for macOS applications.

Drafting Mode: Mode that adds placeholder images and audio for missing assets, allowing the game to run during development.

Electron: Framework for building desktop apps with web technologies (used by Ren'IDE).

Fall-Through: Implicit control flow when one label ends without `jump`, `call`, or `return`, and control passes to the next label.

IntelliSense: Context-aware autocomplete feature that suggests labels, images, characters, etc., as you type.

Jump: A Ren'Py statement (`jump label_name`) that permanently transfers control to a label.

Label: A named point in a Ren'Py script (defined with `label name:`), used as a jump or call target.

Monaco Editor: The code editor engine from Visual Studio Code, used in Ren'IDE for editing `.rpy` files.

Ren'Py: An open-source visual novel engine.

Route: A complete narrative path from the `start` label to an ending label.

Route Canvas: Canvas view showing label-level control flow with nodes for labels and edges for jumps/calls.

SDK (Software Development Kit): The Ren'Py SDK includes the engine, launcher, and tools for building visual novels.

Semantic Tokens: Enhanced syntax highlighting based on the meaning of code (e.g., labels vs. variables).

Snippet: A reusable code template that expands when you type a trigger prefix and press Tab.

Story Canvas: Canvas view showing file-level structure with blocks for `.rpy` files and arrows for jumps/calls.

Story Elements: The right sidebar panel with tabs for Characters, Variables, Images, Audio, Screens, etc.

TextMate Grammar: A syntax highlighting definition format used by many editors (including Ren'IDE).

Unreachable Label: A label that no path from the `start` label leads to (dead code).

12.5 Appendix E: Resources & Links

12.5.1 Official Resources

- **Ren'IDE Website:** https://bluemoonfoundry.com/renide (example URL, replace with actual)
- **GitHub Repository:** https://github.com/bluemoonfoundry/vangard-renpy-ide
- **Issue Tracker:** https://github.com/bluemoonfoundry/vangard-renpy-ide/issues
- **Latest Releases:** https://github.com/bluemoonfoundry/vangard-renpy-ide/releases/latest
- **Demo Video:** YouTube Demo Reel (Beta 4)

12.5.2 Ren'Py Resources

- **Ren'Py Official Website:** https://www.renpy.org/
- **Ren'Py Documentation:** https://www.renpy.org/doc/html/
- **Ren'Py Forum (Lemma Soft):** https://lemmasoft.renai.us/forums/
- **Ren'Py Discord:** Search for "Ren'Py Discord" to find the community server
- **Ren'Py Quickstart:** https://www.renpy.org/doc/html/quickstart.html

12.5.3 Community & Support

- **Ren'IDE Community Discord:** (Add link if available)
- **Ren'IDE Forums:** (Add link if available)
- **Ren'Py Reddit:** r/RenPy

12.5.4 Tutorials & Examples

- **Ren'Py Tutorial Game:** Included with Ren'Py SDK (`the_question` project)
- **Ren'Py Cookbook:** https://www.renpy.org/wiki/renpy/Cookbook
- **Example Projects:** (Add links to example projects if available)

12.6 Appendix F: Credits & Acknowledgments

12.6.1 Development Team

Ren'IDE (Vangard Ren'Py IDE) - **Project Lead:** (Add name/handle) - **Lead Developer:** (Add name/handle) - **Contributors:** See GitHub Contributors

12.6.2 Open Source Libraries

Ren'IDE is built with the following open source technologies:

- **Electron** - Desktop app framework (electronjs.org)
- **React** - UI framework (react.dev)
- **Monaco Editor** - Code editor engine (microsoft.github.io/monaco-editor/)
- **Vite** - Build tool (vitejs.dev)
- **Tailwind CSS** - Utility-first CSS framework (tailwindcss.com)
- **Immer** - Immutable state updates (immerjs.github.io/immer/)
- **Marked** - Markdown parser (marked.js.org)
- **Graphology** - Graph data structure library (graphology.github.io)
- **Sharp** - Image processing (sharp.pixelplumbing.com)

12.6.3 Special Thanks

- **Ren'Py Community** - For creating an amazing visual novel engine and supporting developers
- **Beta Testers** - For feedback and bug reports
- **Visual Novel Developers** - For inspiring this project
- **You** - For using Ren'IDE and supporting visual novel development

12.6.4 License

Ren'IDE is released under the MIT License (or specify actual license).

12.7 Document Metadata

Document Title: Ren'IDE User Guide **Application Version:** 0.7.1 Public Beta 4 **Document Version:** 1.0 **Last Updated:** April 2026 **Total Pages:** Approximately 100-110 pages (when rendered as PDF) **Authors:** (Add author name/handle) **License:** (Add documentation license, e.g., Creative Commons)

12.8 End of User Guide

Thank you for using Ren'IDE! If you have questions, encounter issues, or want to request features, please visit the GitHub issue tracker.

Happy visual novel development!

Ren'IDE - The Visual IDE for Ren'Py Development Version 0.7.1 Public Beta 4